

The Work a Verifier Needs

Transported summaries, certified low-precision heads
and the serving stack of a hybrid model

Samuel Mausberg

30 September 2026

Abstract

A speculative decoder asks its target model for a full vocabulary projection at every verified position, although a greedy verifier needs only a winner and a sampled verifier often needs only the truth of one inequality. We ask what work a verifier actually needs and test two sources of evidence that could let it do less: transporting a drafter’s tile summaries through the shared output head to the target state, as earlier revisions of this manuscript proposed, and evaluating the head at low precision with a rigorous per-row envelope, re-scoring only the rows that can still matter. The setting is Qwen3.5-4B in SGLang on one GH200, where the tied $248,320 \times 2,560$ head is 15% of the weight bytes of a plain decode step and 84% of those of a multi-token-prediction draft step (derived from the checkpoint).

We state the exactness contract a certified head can meet in a real engine: the token the stock head kernel returns at the same batch shape, certified only when a gap condition covers that kernel’s accumulation error and BF16 rounding, with the stock kernel as the fallback. We prove a floating-point envelope for the low-precision pass that holds for Hopper tensor-core accumulation under a published model, or needs no accumulation model for an integer pass; exact greedy selection and exact Gumbel-max sampling under a counter-keyed noise field from one refinement procedure; bounds on the rows that must be re-scored; and conditions under which exact sampling can finish before the scores are known. From the weights alone, transport’s envelope is narrower than an int8 envelope only if draft and target head inputs agree to within about 0.85% in relative ℓ_2 distance (median row, Euclidean envelopes, 64-row tiles); this compares envelope widths, not certification rates or runtime. The greedy certified head is prior art, and our claim is narrower (Section 2.8).

On 6,005 plain-decode head inputs captured from the engine, an int8 head certifies the real-arithmetic winner with 1.3 to 1.6 candidate rows per decision on average, and, assuming the published Khattak–Mikaitis model of Hopper tensor-core accumulation for the stock kernel, the stock contract leaves 0.35% of rows (2.8% of the capture run’s batches of up to 16 requests) to the stock kernel, or 1.40% of rows (10.9% of those batches) under a more conservative model; no certified token differed from the engine’s. The other measurements that decide the hypotheses (the drift of real draft states, head-path runtime, the attribution of time across the stack, and latency-throughput frontiers for an exact stack and for a lossy stack under a stated quality budget) are in progress; each appears as a marked pending item, never as a number.

How to read the evidence in this revision. A *finding* is a theorem proved in the text or a result whose evidence is committed on the repository’s main branch. *Derived* numbers are calculations from the pinned checkpoint, configuration or source code; the formula and inputs are given where they are used. A number followed by [\[under review, PR #N\]](#) rests on evidence in an open pull request. [\[pending: ...\]](#) marks an experiment that has been designed but whose result is not yet committed, with the evidence path it will produce. Hypotheses are labelled as such. Table 2

gives the status of each question; Appendix F lists every claim with its evidence path.

Contents

1	The question	2
2	Where prior work already reaches	5
3	Exactness in a real engine	9
4	Where the time goes	18
5	Transport on real hidden states	19
6	Self-evidence: a certified low-precision head	23
7	Exact sampling and speculative decisions	32
8	Drafting	38
9	Exact repair of long windows	41
10	Stacking every axis	43
11	Serving results	48
12	What did not work, and the limits of what did	50
A	Engine integration notes	52
B	Mechanisms carried over but not tested in this revision	53
C	Mathematical extensions and small falsifiers	54
D	CPU evidence carried over from the earlier revision	55
E	Reproduction	56
F	Claim ledger	56
G	Attribution and disclosure	58

1 The question

Before choosing a faster implementation of an operation, it is worth asking which facts its consumer needs, and whether some of those facts are already implied by work done earlier. In speculative decoding the gap between what is computed and what is needed is concrete. At every verified position the target model projects its hidden state through the output head and produces a score for each of 248,320 tokens. A greedy verifier then asks for one winner. A sampled verifier asks whether a uniform draw falls below a ratio of probabilities, which is a single inequality. A tensor that is convenient to produce is not necessarily the right interface.

This paper asks what work a verifier actually needs, and tests two ways of supplying the missing facts more cheaply than a full projection. The first, which earlier revisions of this manuscript proposed and developed, reuses work the drafter has already paid for. A drafter that projects its own hidden state through the same head can summarize each vocabulary tile (its maximum score and its probability mass). When the target’s hidden state arrives, a Hölder bound transports those summaries to the target, and the verifier projects only the tiles whose bounds leave the decision open. We call this *transport*. The second source produces its own evidence: evaluate the head at low precision, attach a rigorous envelope to every row, and re-score in full precision only the rows whose interval can still change the decision. We call this *self-evidence*. Both keep an exact fallback, so neither changes the decision it certifies; they differ in what the evidence costs and in how tight it is.

The two mechanisms fail for different reasons. Transport’s error term for a row is the product of the row’s deviation from its tile centre and the change between draft and target hidden states. On a trained head both factors can be of the same order as the logits themselves, in which case the transported bound certifies nothing. Self-evidence pays for a full low-precision pass over the head, and its error term is the quantization residual times the hidden-state norm; the difficulty there is to make the envelope cover the floating-point arithmetic the engine actually performs, not an idealized dot product. Which failure dominates is an empirical question about a particular head and a particular engine, and answering it is the core of this revision.

The earlier revision was a research design with executable CPU references and no GPU evidence. This one tests the design in a real engine and reorganizes the paper around what the evidence shows. It keeps the mathematics that survives review, corrects what did not, and adds the theory needed to state exactness against a real kernel rather than against real arithmetic. It also widens the scope to the rest of the serving stack: when the head is only one term of the cost, a paper about avoided work has to say what the other terms are and what can be done about them.

1.1 The setting

The target is Qwen3.5-4B at revision 851bf6e806ef [75, 76]. Its configuration gives hidden width 2,560, a vocabulary of 248,320, and 32 decoder layers, of which 24 use Gated DeltaNet (GDN) linear attention [115] and 8 use full attention (every fourth layer). It has one native multi-token-prediction (MTP) layer, and the output head is tied to the input embedding. The checkpoint is multimodal; its vision tower is loaded by the engine but unused in every experiment here, which are text-only.

Table 1 gives the byte counts that frame the rest of the paper. They are derived from the checkpoint’s tensor sizes and the configuration, not measured. A plain decode step streams every decoder weight once plus the whole head; the embedding lookup reads only one row per request. The head is therefore 15.1% of the weight bytes of a plain step. An MTP draft step runs one small layer followed by the same head, so there the head is 84.1% of the weight bytes. The recurrent state is per request: the configuration stores it in FP32 (`mamba_ssm_dtype`), so each request holds $24 \times 32 \times 128 \times 128 \times 4$ bytes = 50.3 MB of GDN state, and a decode kernel that reads and writes it once per step moves 100.7 MB per request per step. That equals the weight bytes of a step at a batch of 84 requests. Section 4 replaces these calculations with measurements.

The engine is SGLang at commit bd66ce343e4f [127], with CUDA graphs and the overlap scheduler on, the FlashInfer attention backend [117] (FlashAttention 3 is not available on this aarch64 host), and the model revision pinned. The machine is one NVIDIA GH200 with 96 GB of HBM3 (compute capability 9.0)

Component	Bytes	Derivation
Tied head (and input embedding)	1.271 GB	$248,320 \times 2,560 \times 2$ (BF16)
Decoder weights other than the embedding	7.140 GB	sum of tensor sizes in the checkpoint
Weights streamed per plain decode step	8.411 GB	decoder weights plus the head; head share 15.1%
MTP layer	0.241 GB	sum of tensor sizes; a draft step streams it plus the head (84.1% head)
Final decoder layer’s MLP	0.142 GB	$3 \times 2,560 \times 9,216 \times 2$
Vision tower (loaded, unused)	0.667 GB	sum of tensor sizes
GDN recurrent state per request	50.3 MB	$24 \times 32 \times 128 \times 128 \times 4$ (FP32)
Attention KV per context token	32 KiB	$8 \times 4 \times 256 \times 2 \times 2$ (BF16 K and V)

Table 1: Byte counts for Qwen3.5-4B at revision 851bf6e806ef, derived from the checkpoint’s safetensors headers and `config.json`. They are calculations, not measurements; Section 4 reports what the kernels actually move.

and a 64-core Grace CPU. Two drafters are in scope. The native MTP layer is served through SGLang’s NEXTN path, which the engine reports as EAGLE [50]. The public DFlash block drafter for this exact target, `z-lab/Qwen3.5-4B-DFlash` [10, 119], has six layers, drafts blocks of up to 16 tokens conditioned on eight of the target’s intermediate hidden states, and has no head of its own: its drafts are projected through the target’s tied head. It is therefore both the strongest public speculative baseline for this model and a direct test bed for transport. The public DFlash 2 pair for Qwen3.8-27B [33, 34] is a transfer lane; no DFlash 2 checkpoint for the 4B model is public.

1.2 Hypotheses and their status

Table 2 lists the questions the work must answer, what would decide each, and where the evidence stands. The status column is updated as results are committed; a hypothesis that fails stays in the table with the reason.

1.3 Contributions

The contributions of this revision, at the strength the evidence currently supports, are these.

1. *An exactness contract for a certified head in a real engine* (Section 3). The reference is the stock head kernel’s decision for the same hidden state at the same batch shape, not a higher-precision recomputation. A gap condition (Theorem 3.5) states when a certified token must equal the stock token; every other row runs the stock kernel. The section also names five contracts (identical state, greedy outputs, seeded samples, sampling law and empirical quality) against an executable reference, states a general commit rule for decisions from partial information, and classifies skipped work as dead, deferred or soundly enclosed. These are proved results; the stock kernel’s error model on which the gap condition rests is an assumption that is tested, not proved.
2. *A comparison of the two evidence sources* (Section 5). Each envelope is the best its metadata allows (Proposition 5.3). Transport’s envelope is narrower than an int8 self-evidence envelope for row i only if the drift ratio $\rho = \|h^t - h^d\|_2 / \|h^t\|_2$ is below $\|e_i\|_2 / r_c(i)$ (Theorem 5.5). With 64-row tiles of contiguous token indices and the tile mean as centre, that threshold has a median of 0.85% over the rows of this head (weights only; a comparison of envelope widths, not of certification rates or runtime; `evidence/precision/head_constants.json`). The measured drift of real draft states is [\[pending: geometry\]](#).
3. *A floating-point envelope and the certified decisions built on it* (Sections 6 and 7). The envelope covers Hopper tensor-core accumulation under the published model of Khattak and Mikaitis [37], or needs no accumulation model when the pass is an integer product. One refinement procedure gives exact greedy selection and exact Gumbel-max sampling under a counter-keyed noise field, with a bound on the expected

	Hypothesis	Deciding evidence	Status
H1	The head is a large share of decode and MTP-speculation time for this model on GH200.	nsys attribution at several batch sizes (Section 4).	Byte shares derived (Table 1); measured shares [pending: profile, PR #13]
H2	Transport gives useful certificates on real hidden states.	Aligned MTP and DFlash replay: bound widths against logit margins (Section 5).	Crossover derived from weights; measured drift [pending: geometry]
H3	A low-precision head with a rigorous envelope certifies the stock decision while reading fewer weight bytes.	Replay candidate sets, then a GPU kernel against the stock head chain (Section 6).	Envelope and contract proved; plain-decode candidate sets measured (Section 6.4); runtime [pending: kernel]
H4	The certificate improves the served latency–throughput frontier without changing outputs.	Before/after Pareto sweeps with output-equality checks (Section 11).	[pending: bench, integration]
H5	SGLang’s hybrid state stays correct under speculative verification.	Differential tests against plain decoding at every rejection position, aborts and prefix reuse (Section 3.8).	[pending: state]
H6	Other layers of the stack leave measurable headroom.	Profiles and controlled ablations (Sections 4, 10).	[pending: profile, moonshot]
H7	Reformulations and declared approximations multiply gains well beyond tuning.	Bytes per step, quality–speed curves, end-to-end sweeps (Section 10).	Ceilings derived; measurements [pending: moonshot]
P1	A certificate on the decision can skip the final layer’s MLP and the head in target-only decoding.	Oracle-radius replay at the cut, then real bounds (Section 6.8).	Rejected by its kill test (Section 6.8)
P2	Long-window exact repair from a DFlash window beats optimized DFlash substantially.	Perfect-continuation oracle, then real proposals (Section 9).	Hypothesis [pending: repair]
P3	Target-anchored residual repair is much cheaper than recomputation.	Oracle and real-correction kill tests (Section 9).	Hypothesis [pending: repair]
P4	Writing the recurrent state less often, bit-exactly, gives at least 1.10× decode at batch 128.	Bit-exactness check, then paired timing (Section 10).	Hypothesis [pending: moonshot]
P5	Exact computation can be shared across unrelated requests.	Analysis and exact witnesses; measured ceiling of the one structural candidate (Section 10).	Rejected in analysis; ceiling [pending: profile]

Table 2: Questions and the evidence that decides them. H1–H7 are the working hypotheses of the programme; P1–P5 are later proposals with designed kill tests or pre-registered criteria.

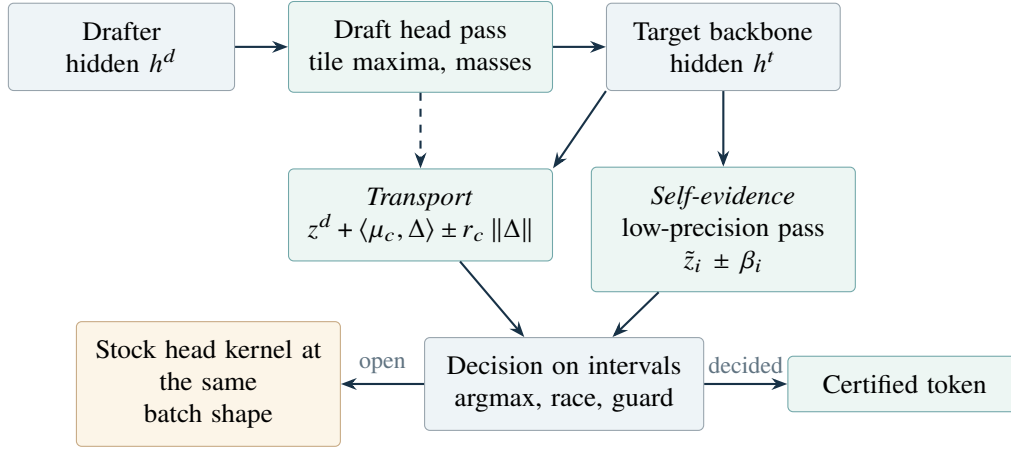


Figure 1: Two sources of evidence for the same decision. Transport reuses the drafter’s head pass and bounds only the change $\Delta = h^t - h^d$; self-evidence pays for a cheap pass over the head at the target state. Either way the decision is taken on intervals, rows or tiles are refined while it stays open, and a row that cannot be certified runs the stock head kernel. The target backbone always runs; the saving, if any, is in the head.

number of re-scored rows. Fixed-noise verification [16], with the stock seeded sampler’s noise field, turns sampled verification into a certified argmax with no partition function; it is the sampling contract adopted for the certified speculative path (design decided, measurement pending). Acceptance guards with enclosed numerators, residual races, a characterization of how much of an exact sample can be emitted before the scores are known, and KL and total-variation bounds for an approximate mode complete the set. Several of these rest on known principles (Gumbel-max, A* Sampling [60], maximal coupling); Section 2 says which parts are new. On 6,005 captured plain-decode states an int8 head certifies the real-arithmetic winner with 1.3–1.6 candidate rows per decision on average, and the interval form of the stock contract leaves 0.35% of rows (2.8% of the capture run’s batches of up to 16 requests) to the stock kernel if the stock kernel follows the Khattak–Mikaitis model of Hopper accumulation, an assumption, and 1.40% of rows (10.9% of those batches) under a more conservative model, against 2–3% of rows for the plain gap condition (Section 6.4).

4. *Measurements on every axis of the serving stack* (Sections 4 and 8–11): attribution and bytes per step, drafting, the certified head’s runtime, repair, recurrent state, and two latency–throughput frontiers, one exact and one lossy under a stated quality budget. These are [pending] in this revision.

The greedy form of the certified low-precision head is not new: several public implementations screen a low-precision copy of an LM head with a bound and re-score the survivors exactly (Section 2.3). Nor is using bounds to obtain an exact Gumbel-max sample [60, 62]. What we claim is narrower, and Section 2.8 states it.

1.4 Reading guide

Section 2 places the work against prior art. Section 3 fixes what “exact” means in this engine. Section 4 measures where the time goes. Sections 5–7 develop and test the two evidence sources and the decisions built on them. Sections 8–10 cover drafting, repair and the rest of the stack, and Section 11 reports the served frontiers. Section 12 collects what did not work and the limits of what did. The appendices hold the engine integration notes, the CPU evidence inherited from the earlier revision, mechanisms not yet tested, reproduction commands, the claim ledger and the attribution statement.

2 Where prior work already reaches

A source audit should remove false novelty before experiments make it expensive to abandon. This section is organized by the mechanism each body of work shares with ours, and it ends with the narrow claim that remains. Descriptions come from the literature review that accompanies this revision (sources/literature_review.md);

performance figures quoted from other work are the authors’ own and were not reproduced.

2.1 The head as a cost

For a small model with a large vocabulary the output head is a large part of each step. FR-Spec profiles EAGLE-2 drafting on Llama-3-8B, with a 128k vocabulary, and finds the head projection at 49% of draft time and the softmax over its output at a further 13% [125]; a trimmed-vocabulary study finds the head at 64% of an EAGLE-3-style drafter’s FLOPs [5]. Vocabulary sizes have grown with compute budgets [93], which is why 4B models now ship 250k-row heads. Table 1 gives the corresponding byte shares for Qwen3.5-4B; Section 4 measures the time shares.

2.2 Screening, exact search and bounds

Computing cheap scores for every row, bounding the error, and re-computing only the rows that could still matter is an old idea in similarity search. The VA-file filters with quantized cell approximations and refines survivors exactly [107]; FEXIPRO combines an SVD transform with scaled-integer upper bounds and exact floating-point refinement [49]; LEMP prunes norm buckets with length and Cauchy–Schwarz bounds [94]; ball and cone trees bound $\max\langle q, p \rangle$ by $\langle q, \mu \rangle + R \|q\|$ [78]. These methods are exact in real arithmetic. A warning comes with them: Maximus and Optimus found that blocked dense matrix multiplication often beats such indices end to end [2], so a pruning method must beat a tuned dense kernel, not a count of floating-point operations.

In language models, SVD-Softmax computes preview logits from leading singular directions and recomputes a fixed top set in full [87], and HiRE uses an int4 or low-rank copy of the head to choose a fixed top- k set that is recomputed in BF16 [46]. Both have the two-pass shape of self-evidence without a certificate: a fixed candidate count carries no guarantee that the true winner is among the candidates. CSV-Decode clusters the head’s rows offline and bounds each cluster by $\langle \mu_c, h \rangle + R_c \|h\|$, opening clusters until the top- k is certified [56]. That is exactly the static tile screen that earlier revisions of this manuscript proposed as an ablation, and it is exact in real arithmetic; its text does not treat floating-point rounding. A public experiment on a Qwen3.8 head of the same vocabulary found that centroid-radius bounds prune almost nothing there [77]. The transport bound differs from these screens only in where it starts: from a score already measured at the draft state, bounding the change $\langle w_i - \mu_c, \Delta \rangle$ rather than the score itself. Bounding an inner product under a bounded displacement of the query is also known, for example in sequential screening rules for the lasso [102] and in cached retrieval with drifting queries [21].

2.3 Certified low-precision heads

The greedy half of the self-evidence mechanism is implemented publicly, several times over.

- sparkpipe keeps an E4M3 shadow of a BF16 head in 32-element groups, attaches a per-group certificate that includes γ_n floating-point terms for CUDA-core FMA accumulation, and re-scores survivors exactly in BF16. It routes the decode, last prefill and MTP argmax rows of Qwen3.8-27B (a 248,320-row head) through this path and reports bit-exact tokens and scores [90]. Sampled requests take the full head read.
- dgpp stores an int8 head as per-row bit planes, bounds every logit from the high planes, and re-reads only surviving rows in the production accumulation order, for MTP draft steps and greedy verification [28]. Its reference is the quantized head itself, and its floating-point term is a fixed relative slack.
- A Metal implementation for Apple silicon screens with an MXFP8 coarse head whose bound covers quantization and both kernels’ rounding, and re-scores candidates with a textual replica of the stock BF16 kernel so that rows are bit-identical [47].
- knlp certifies greedy decoding with an int8 shadow in a PCA basis and a Cauchy–Schwarz bound on the out-of-basis part; its own table reports an argmax match of 0.998 at 14B, so floating point is not fully handled [8].

In retrieval, EAHR bounds int8 inner products by $\|e_q\| \|v\| + \|\hat{q}\| \|e_v\|$ with a floating-point guard and rechecks ambiguous items in FP32 [120]. Du et al. show why the reference must be named: BF16 and FP16 greedy

decoding diverge on 49–100% of prompts in their experiments, a flip is decided at the head by the top-two margin, and recomputing in FP32 when the margin is small (a heuristic threshold, not a certificate) repairs much of it [18]. Measured head sensitivity sets expectations for re-scoring: a row-wise int8 copy of the Qwen3-8B head agrees with the BF16 argmax on 97.56% of positions [40], so an exact re-score must catch the rest.

2.4 Exact sampling, fused sampling and determinism

The Gumbel-max identity [27, 71] makes a categorical sample the argmax of perturbed scores, and its top- k form samples without replacement [42]. A* Sampling uses bounds to instantiate only the relevant perturbations and still returns an exact sample [60]. Musmann, Levy and Ermon draw Gumbels only for a retrieved top set, bound every other score by the smallest retrieved one, and lazily instantiate the few tail perturbations that could win; with an approximate top set they widen the cutoff by the approximation error [61, 62]. That is the direct precedent for exact Gumbel-max sampling from bounded scores. The version used here differs in two engineering respects: a dense low-precision pass bounds every row instead of a sublinear index, and the noise is a fixed field keyed by token, so the bounded race returns the same token as a dense race with the same field, not only a sample with the right law.

FlashSampling fuses Gumbel-max sampling into the head projection, keeping one perturbed maximizer per row and tile so logits never reach memory; it computes every tile, accumulates in FP32 and is exact in distribution [80]. Its noise offset depends on the tiling, so a token’s noise cannot be recomputed on its own. FlashInfer samples top- k , top- p and min- p without sorting and implements chain speculative sampling [113, 117]. SonicSampler fuses the post-logit sampling and verification kernels over a bounded top-128 pool and describes top- p under that pool as theoretically lossy [73]. On determinism, batch-invariant kernels make temperature-0 outputs reproducible across batch compositions [29]; SGLang’s deterministic mode replaces the default matrix multiplication with a persistent Triton kernel and forces the PyTorch sampler [96], which changes the kernel a certificate would have to reproduce. Floating-point nondeterminism in inference has also been characterized and partly mitigated [118].

2.5 Speculative decoding and verification policies

Speculative sampling accepts a proposal $x \sim q$ with probability $\min(1, p_x/q_x)$ and otherwise samples the normalized residual $(p - q)_+$; the output has the target law [9, 48], and the acceptance probability $\sum_i \min(p_i, q_i)$ is that of a maximal coupling. SpecTr and block verification extend the rule to several drafts and to whole blocks [91, 92]. Drafter-invariant speculative decoding couples draft and target through shared Gumbel noise, so the committed token is $\arg \max_i (\log p_i + G_i)$ whatever the drafter proposes [16], and the construction extends to several drafts [79]. Under that coupling sampled verification is an argmax, which is why it matters here (Section 7.3).

Drafters that project through a head pay for it at every draft step: Medusa and the EAGLE family [6, 50–52], native multi-token prediction [17, 24], and DFlash’s block-diffusion drafter [10], whose public 4B checkpoint drafts through the target’s head [119]. DFlash 2 adds a context-gated bilinear selector over candidate lattices [33, 34]; LiLiCorr reranks DFlash candidates and finds that an exact best-path search over a surrogate score decodes worse than the greedy rule [82]. The model card of the 4B DFlash drafter reports, on one B200 with greedy decoding, 3.40–4.60× over autoregressive decoding at concurrency 1 with blocks of 16 and 2.15–2.61× at concurrency 32 with blocks of 8, against at most 2.31× and 1.80× for the MTP layer [119]. These are the authors’ numbers on different hardware; Section 8 measures the drafter here.

Verification-depth policies choose how many drafted tokens to verify. OPT-Tree searches for the tree that maximizes expected acceptance [103]; Sequoia sizes trees for the hardware [13]; SpecDec++ stops drafting with a learned acceptance head [32]. DSpark schedules verification from confidences and a profiled throughput curve [14], D-cut prunes draft blocks against a profiled cost table aligned with the engine’s CUDA-graph shapes [57], and an open SGLang pull request ports the DSpark scheduler to DFlash 2 with greedy benchmarks [85]. For sampled decoding, DSpark states that lossless speculation requires admission decisions that do not depend on future candidate tokens, with a counterexample, and D-cut gives a two-token counterexample and a shifted-confidence

Existing work	Already established or implemented	Question pursued here
CSV-Decode [56]; ball bounds [78]	Cluster centre-plus-radius bounds on head rows; exact top- k in real arithmetic.	Transport starts from draft scores; measure it against this static screen and against self-evidence.
sparkpipe, dgpp, Laguna prune, knlp [8, 28, 47, 90]	Certified greedy screening with a low-precision head and exact re-score, including MTP rows of a 248,320-row head.	Hopper-sound envelope; stock-kernel contract; exact sampling; SGLang measurement.
HiRE, SVD- Softmax [46, 87]	Cheap copy selects a fixed candidate set; exact recompute.	Certified, variable candidate sets; fixed- k' as an ablation.
A* Sampling; lazy Gumbels [60, 62]	Exact samples from bounded scores.	A dense cheap pass and a keyed noise field for pathwise equality with the dense race.
FlashSampling [80]	Gumbel-max fused into the head; every tile computed.	Skip rows, not just logit materialization; dense comparator.
Drafter-invariant decoding [16, 79]	Shared-noise verification; output independent of the drafter.	Certified argmax for sampled verification with no partition function.
DSpark, D-cut, PR 36136 [14, 57, 85]	Confidence- and cost-scheduled verification depth; the non-anticipating condition.	Depth policies under fixed-noise verification, where any policy is exact.
DFlash, DFlash 2 [10, 33, 119]	Block drafting through the target head; public 4B DFlash drafter.	Strongest speculative baseline here; test bed for transport.
ReplaySSM; GDN tree verification [23, 54, 84, 105]	Replay-based rollback; tree verification for GDN hybrids.	Measured state traffic and capacity; state as an axis of the stack.

Table 3: Prior art bounds the novelty claim. Reported performance of other work is not reproduced by this paper.

repair [14, 57]. The counterexample in Section 7.8 illustrates their condition; it is not a contribution of this paper.

2.6 Draft-vocabulary reduction

FR-Spec restricts the draft softmax to a frequency-ranked subset of the head [125], VocabTrim trims by token counts on target-generated text [25], SpecVocab selects a per-step subset with low-rank scores and computes exact draft logits on it [108], DynaSpec routes contexts to clusters of rows [121], and NanoSpec (titled MicroSpec in its first version) builds a small in-context vocabulary without training [12]. FastMTP compresses only the draft vocabulary and keeps full-vocabulary verification [7]. All of these change only the drafter, so the target’s decision is unchanged when the verifier uses the drafter’s actual proposal law, or when verification is target-only. They trade acceptance for draft cost; a certified head returns the full-vocabulary draft token while reading fewer bytes per row, and it also applies on the target side, where truncation does not.

2.7 Hybrid recurrent state under speculation

Gated DeltaNet’s recurrence [114, 115] makes rollback after a rejection a state problem. The Mamba-in-Llama work advances one cached state lazily and recomputes after a rejection [104]; STree, TreeWY, Bole, GDN Tree-Scan and SpecLA verify trees or chains on SSM and GDN hybrids [23, 59, 105, 106, 111], and TreeWY says explicitly that its outputs are not bit-identical to the baseline. ReplaySSM caches recent inputs instead of writing the state at every step, equivalent to ordinary decoding up to floating-point error [54]; SGLang has its own replay fold behind flags that are off by default [84]. State precision matters for quality: DAMP measures FP16 recurrent state as nearly lossless and BF16 as several points worse on a reasoning benchmark for another GDN model [123], and LeapQuant re-quantizes the state once per 16-token window [70].

2.8 The novelty boundary

We do not claim norm or quantization bounds, bound-then-refine search, certified greedy screening of an LM head, top- k merging, Gumbel-max sampling from bounded scores, drafter-invariant verification, the non-anticipating condition for adaptive depth, function scans, recurrent replay or graph-aware batching. Table 3 lists the closest work and the question left for this programme. What remains, as far as the review found as of 30 September 2026 with the coverage limits it records, is:

1. a floating-point envelope for a low-precision head pass that is sound for the accumulation the GPU actually performs (NVIDIA tensor cores align addends to the largest term and truncate [20], and Khattak and Mikaitis model Hopper’s BF16 and FP8 paths in detail [37]; existing certified heads use CUDA-core FMA, replay the production order, or assume FP32 is exact);
2. an exactness contract against the stock kernel at the same batch shape, with a stated gap condition and fallback;
3. extension of the certified head from greedy selection to exact sampling under a fixed counter-keyed noise field (pathwise equal to the dense race) and to fixed-noise verification with the engine’s own seeded noise, whose committed tokens are the seeded sampler’s tokens for the verify pass’s logits; partition brackets for standard acceptance are analysed but not built;
4. a measurement inside SGLang on HBM3 with CUDA graphs, applied to draft steps, verification and plain decoding, against the stock head chain and FlashSampling;
5. for transport, the only mechanism not found in prior work (draft-side tile summaries carried through a shared head to prune the verifier’s head), a measurement of whether it survives real geometry.

Items 1–3 are modest extensions of known principles; their value depends on measurement. The contribution is mostly soundness, integration and measurement, and a well-explained negative result would also be informative.

3 Exactness in a real engine

A certificate is a statement about a specified computation. “Exact” is therefore incomplete until it names the computation whose output is preserved and the sense in which it is preserved. This section fixes both for the engine used here. It separates the contracts a decoder can promise, describes what the stock engine actually computes, states a general rule for committing a decision from partial information, and derives the condition under which a certified token equals the token the stock head kernel would have produced.

3.1 Five contracts, and naming the reference

An exactness claim needs two things: the reference it is measured against, and the sense in which the result equals it. The reference is an executable specification, not an idealized network: the weights, the kernels, the decoding policy, the random-number protocol and the execution shape, together with whether it is stock plain decoding, stock speculative verification or a recorded execution. Mathematical equivalence over the reals does not imply that two GPU reductions choose the same token, because a different batch shape or reduction order can move logits that are close [66, 74], so a certificate must enclose the value the reference computes, casts included. Table 4 lists the contracts a change can claim against such a reference. They are ordered roughly from strongest to weakest, but none implies the next in general.

Small witnesses separate them (`tests/test_contracts.py`, `evidence/contracts/`). Reassociating an FP32 sum flips a greedy winner: $1 + 2^{-24} + 2^{-24}$ summed left to right returns 1 and ties a competing logit of 1, so the first-index rule picks the competitor, while the regrouped sum returns $1 + 2^{-23}$ and wins as it does in real arithmetic. Distinct FP32 values, 1 and $1 + 2^{-10}$, fall in one BF16 rounding cell, so a BF16-output head ties them. Logits (1, 0) and (2, 0) have the same greedy decision but different laws, $(\frac{2}{3}, \frac{1}{3})$ against $(\frac{4}{5}, \frac{1}{5})$ in the base-2 mass model and 0.731 against 0.881 on the first token at temperature one. Two inverse-CDF samplers that list the tokens in different orders have the same law but map the same uniform to different tokens. And a state difference the current query cannot see becomes visible after one recurrent step (Section 3.9), so equal outputs

Contract	What must be equal	Where this paper uses it
Identical internal state	Named logical tensors (recurrent state, attention cache) at named boundaries, bitwise; not layouts or scratch buffers	State replay and write avoidance (Sections 3.9, 10)
Identical greedy outputs	Decisions, ties and stopping, with a valid state from which to continue	The certified head under R-stock (Section 3.4)
Identical seeded samples	The reference’s mapping from seed to output; with an inverse-CDF reference, certify with CDF bounds against the same uniform and fall back without redrawing	Fixed-noise verification against the seeded sampler (Section 7.3)
Exact sampling distribution	A named law: the ideal model’s, the categorical law of the reference’s scores, or the finite random-number implementation’s; speculative correction targets the verifier’s own p	Certified races and residual races (Section 7)
Empirical quality equivalence	Task scores and distances within declared margins, with confidence intervals	The lossy stack (Section 10)

Table 4: Exactness contracts. Each claim in this paper names one of them and its reference.

now do not imply equal state. Equal token sequences, finally, say nothing about latency or about whether the cache that produced them is owned correctly.

For sampled verification, let p be the target law at the reached prefix and q the *actual* proposal law, conditional on the information used to produce the proposal. A softmax buffer from a different selector is not q ; a deterministic proposal is a point mass. The standard one-step correction accepts $X \sim q$ with probability $\min(1, p_X/q_X)$ and otherwise samples $(p - q)_+$ normalized; the accepted mass at i is $\min(p_i, q_i)$ and the residual contributes $(p_i - q_i)_+$, which sum to p_i [48]. The certificates in this paper change how such a comparison is evaluated, never the law used in it.

3.2 What the stock engine computes

At the pinned commit, with the defaults used here, the head is `torch.matmul(h.to(bf16), W.T)` in `LogitsProcessor._compute_lm_head` (`python/sglang/srt/layers/logits_processor.py`). The product is a cuBLAS BF16 GEMM that accumulates in FP32 and returns BF16 logits, which are then copied into an FP32 buffer. cuBLAS chooses its algorithm per shape, so the logits of a row can depend on how many rows are in the batch, and PyTorch’s default permits reduced-precision reductions in BF16 GEMMs. Greedy decoding and greedy verification take `torch.argmax`, which returns the first maximal index. BF16 keeps eight significant bits, so logits in $[16, 32)$ are spaced 0.125 apart; two real logits closer than about one spacing can round to the same BF16 value, and then the smaller token index wins.

Sampling at the pin is already a race. Without top- k , top- p or min- p and without a seed, the sampler returns $\arg \max_i p_i/E_i$ with E_i exponential draws from the Philox generator, which is Gumbel-max in another form (`SGLANG_OPT_USE_GUMBEL_SAMPLE`, on by default). With a per-request seed it returns $\arg \max_i (\log p_i + G_i)$ in FP64, where $G_i = -\log(-\log x_i)$ and x_i is a 32-bit MurmurHash of (seed, position, token) scaled to $[0, 1]$ (`multinomial_with_seed` in `python/sglang/srt/layers/sampler.py`). That is a counter-based noise field: the noise of any token can be recomputed on its own, in any order. Sampled speculative verification by default uses point-mass drafts: a drafted token is accepted when a uniform coin falls below the target probability mass of the siblings tried so far, and a rejection samples the target restricted to the untried tokens by inverse CDF. Two flags change the reference: `--enable-fp32-lm-head` makes the head return FP32 logits, and `--enable-deterministic-inference` replaces the default matrix multiplication with a persistent batch-invariant Triton kernel [96].

3.3 A decision needs a sound set, not a finished forward pass

The same rule underlies every certificate in this paper. Partial computation yields information I about the final logits: an approximate pass with error bounds, a transported summary, an approximate hidden state with a radius. From I we keep a *sound set* $Z(I) \subseteq \mathbb{R}^V$ that contains the reference implementation’s logits. A decision rule $D(z, R)$ maps logits and the randomness fixed by the contract (a noise field, a uniform draw) to a token or an accept bit.

Definition 3.1 (Commit rule). Commit the decision d when $\{D(z, R) : z \in Z(I)\} = \{d\}$. Otherwise acquire more information, which must shrink or replace $Z(I)$ by another sound set, and test again.

If the reference logits $z^\star$ lie in $Z(I)$, a committed d equals $D(z^\star, R)$; that is the whole soundness argument, and it holds whatever produced $Z(I)$. For a box $Z(I) = \prod_j [l_j, u_j]$ the greedy rule commits c when $l_c > \max_{j \neq c} u_j$. A Gumbel race with a fixed field G commits c when $l_c/T + G_c > \max_{j \neq c} (u_j/T + G_j)$, which needs no partition function; the field must stay fixed through every refinement [60]. When the information is about the hidden state rather than the logits, a box is loose, and the directional form below is the right certificate.

Proposition 3.2 (Directional margin). *Let the logits be $z = Wh$ in real arithmetic and suppose $\|h - \tilde{h}\|_2 \leq \varepsilon_h$ for a known $\tilde{h}$. Then c is the strict argmax of Wh for every such h if and only if*

$$(w_c - w_j)^\top \tilde{h} > \|w_c - w_j\|_2 \varepsilon_h \quad \text{for all } j \neq c.$$

This condition is implied by, and can be much weaker than, the box condition obtained from the row bounds $|z_j - w_j^\top \tilde{h}| \leq \|w_j\|_2 \varepsilon_h$.

Proof. For fixed j , $\min_{\|e\|_2 \leq \varepsilon_h} (w_c - w_j)^\top (\tilde{h} + e) = (w_c - w_j)^\top \tilde{h} - \|w_c - w_j\|_2 \varepsilon_h$ by Cauchy–Schwarz, with equality at $e = -\varepsilon_h (w_c - w_j) / \|w_c - w_j\|_2$. So $z_c > z_j$ throughout the ball exactly when the stated inequality holds. The box condition reads $(w_c - w_j)^\top \tilde{h} > (\|w_c\|_2 + \|w_j\|_2) \varepsilon_h$, and $\|w_c - w_j\|_2 \leq \|w_c\|_2 + \|w_j\|_2$. $\square$

The directional certificate is not free: it needs $\|w_c - w_j\|_2$ for every competitor, which for all j costs as much as a head pass. It is useful after a box screen has reduced the competitors to a few rows, and for the stock contract the gap condition of Theorem 3.5 then applies on top of it, as it does for the refinement ladder. It is also a real-arithmetic statement; Section 6 supplies the floating-point terms that turn such statements into enclosures of computed values.

3.4 Contracts for the head and agreement with the stock kernel

Let $\mathbb{B}$ be the BF16 numbers. The stored head is $W \in \mathbb{B}^{V \times D}$ and $h \in \mathbb{B}^D$ is the vector the engine feeds to it. The *real logit* is $z_i = \sum_j w_{ij} h_j$ in exact arithmetic. A binary format with precision p (significant bits) and smallest normal exponent $e_{\min}$ has unit roundoff $u = 2^{-p}$ (BF16 $p = 8$, FP32 $p = 24$, FP64 $p = 53$). Round to nearest satisfies $|\text{fl}(x) - x| \leq u|x| + \mu$ with $\mu = 2^{e_{\min}}$, which also covers flush-to-zero; directed and truncating rounding satisfy the same bound with $2u$. Write $\text{rn}_\mathbb{B}$ for rounding to nearest-even BF16 and $\text{ulp}_\mathbb{B}(x)$ for the BF16 spacing at $|x|$. The stock head returns $L_i = \text{rn}_\mathbb{B}(\hat{y}_i)$, where $\hat{y}_i$ is cuBLAS’s FP32 accumulation of the products.

Definition 3.3 (Greedy and race contracts). **R-real**: $\text{tok} = \min\{i : z_i = \max_j z_j\}$. **R-bf16**: $\text{tok} = \min\{i : \text{rn}_\mathbb{B}(z_i) = \max_j \text{rn}_\mathbb{B}(z_j)\}$. **R-stock**: $\text{tok} = \min\{i : L_i = \max_j L_j\}$. **S-race**: for $T > 0$ and a noise vector $g \in \mathbb{R}^V$, $\text{tok} = \min\{i : z_i/T + g_i = \max_j (z_j/T + g_j)\}$.

R-real and R-bf16 are functions of (W, h) alone, so they do not depend on reduction order or batch shape. R-stock is what an unmodified server returns, and it can depend on both.

Lemma 3.4 (BF16 separation). *If $y_a - y_b > \text{ulp}_\mathbb{B}(\max(|y_a|, |y_b|))$ then $\text{rn}_\mathbb{B}(y_a) > \text{rn}_\mathbb{B}(y_b)$.*

Proof. Rounding is nondecreasing. If both round to v , both lie in the preimage of v . When $|v|$ is not a power of two the preimage has length $\text{ulp}_B(v)$ and both values lie in the binade of v . When $|v| = 2^e$ the preimage is $[v - \text{ulp}_B(v)/4, v + \text{ulp}_B(v)/2]$ (for $v > 0$); two values on one side are within half the spacing of their binade, and values on both sides are within $\frac{3}{4} \text{ulp}_B(v) \leq \text{ulp}_B(y_a)$. At $v = 0$ the preimage has the subnormal spacing. Each case contradicts the hypothesis. $\square$

Theorem 3.5 (Agreement with the stock head). *Let $|\hat{y}_i - z_i| \leq G_i$ for every i and let a be the R-real token. If for all $b \neq a$*

$$z_a - z_b > \delta_{ab} := G_a + G_b + \text{ulp}_B(\max(|z_a|, |z_b|) + \max(G_a, G_b)),$$

then R-stock returns a . If $\text{rn}_B(\hat{y}_i) = \text{rn}_B(z_i)$ for all i , R-stock equals R-bf16.

Proof. $\hat{y}_a - \hat{y}_b > \text{ulp}_B(\max(|z_a|, |z_b|) + \max(G_a, G_b)) \geq \text{ulp}_B(\max(|\hat{y}_a|, |\hat{y}_b|))$, so Lemma 3.4 gives $L_a > L_b$ for every b . The second claim is immediate. $\square$

The contract used in this paper. A certified head claims R-stock at the batch shape it serves: for each row it returns the token the stock head kernel would return for the same hidden state in a batch of the same shape. It certifies a row only when the gap condition of Theorem 3.5 holds against every competitor, with G_i bounding the stock kernel’s pre-rounding error at that shape; for every other row it runs the stock kernel on the batch and takes that kernel’s own argmax and tie rule, so those rows are the stock output by construction. Near $|z| \in [16, 32)$ the margin required is about $0.125 + 2G$.

The procedure has to establish that margin for every competitor, including rows it never re-scores. With enclosures $lo_i \leq z_i \leq hi_i$, define for a pair of rows

$$\hat{\delta}_{aj} = G_a + G_j + \text{ulp}_B(\max(|lo_a|, |hi_a|, |lo_j|, |hi_j|) + \max(G_a, G_j)).$$

A candidate a is certified when every competitor $j \neq a$ satisfies $hi_j < lo_a - \hat{\delta}_{aj}$; then $z_a - z_j > \hat{\delta}_{aj} \geq \delta_{aj}$, because ulp_B is nondecreasing in the magnitude and $|z|$ is bounded by the endpoints, and Theorem 3.5 gives the stock token under the G_i model. A competitor may therefore be excluded only when $hi_j < lo_a - \hat{\delta}_{aj}$, not merely when $hi_j < lo_a$. At level 0 this is checked in bulk: with $G_{\max} = \max_i G_i$, $M = \max_i \max(|lo_i|, |hi_i|)$ and $\delta_{\max} = 2G_{\max} + \text{ulp}_B(M + G_{\max})$, every row with $hi_j < \max_i lo_i - \delta_{\max}$ meets the condition for whichever candidate is finally certified, since that candidate’s re-scored lower bound is at least $\max_i lo_i$ (enclosures are intersected across levels) and $\hat{\delta}_{aj} \leq \delta_{\max}$. The remaining rows are re-scored and checked pair by pair.

The condition rests on G_i , and G_i rests on a model of the stock kernel. For an FP32 accumulation, Lemma 6.1 supplies G_i from the reduction tree; for the tensor-core kernels cuBLAS selects on this GPU, the tree’s per-node error comes from the published model of Hopper accumulation [37], which is not a vendor contract, and a reduced-precision split- K reduction admits no documented bound at all. PyTorch permits such reductions by default, so either the stock baseline disables them or the fallback must cover every shape at which cuBLAS might use them; the exact reference contains a constructed witness in which BF16 partial sums reverse a token that the FP32-model gap condition certifies (tests/test_precision.py, evidence/precision/). The agreement of certified tokens with R-stock is therefore conditional on the stock kernel’s error being within the modelled G_i at the shapes served, and that assumption is tested against the kernel’s outputs rather than proved: on 6,005 captured plain-decode rows no certified token differed from the token the engine returned (Section 6.4) [pending: kernel: measured cuBLAS accumulation error against the models at the served shapes; evidence/certified_head/].

R-bf16 is not the contract, although it is attractive: it is batch-invariant and it uses the same lowest-index rule as the stock argmax. It is not what the engine computes. The stock kernel rounds its own FP32 accumulation, which can fall on the other side of a BF16 rounding boundary from the exactly rounded value. A two-row witness shows it: with $w_0 = (16, 0, 0)$, $w_1 = (16, \frac{1}{16}, 2^{-30})$ and $h = (1, 1, 1)$, the exact logits are 16 and $16.0625 + 2^{-30}$, so R-bf16 rounds them to 16 and 16.125 and picks token 1; a sequential FP32 accumulation loses the 2^{-30} , rounds 16.0625 to even, returns 16 for both rows, and the stock argmax picks token 0 (test_bf16_contract_is_not_stock in

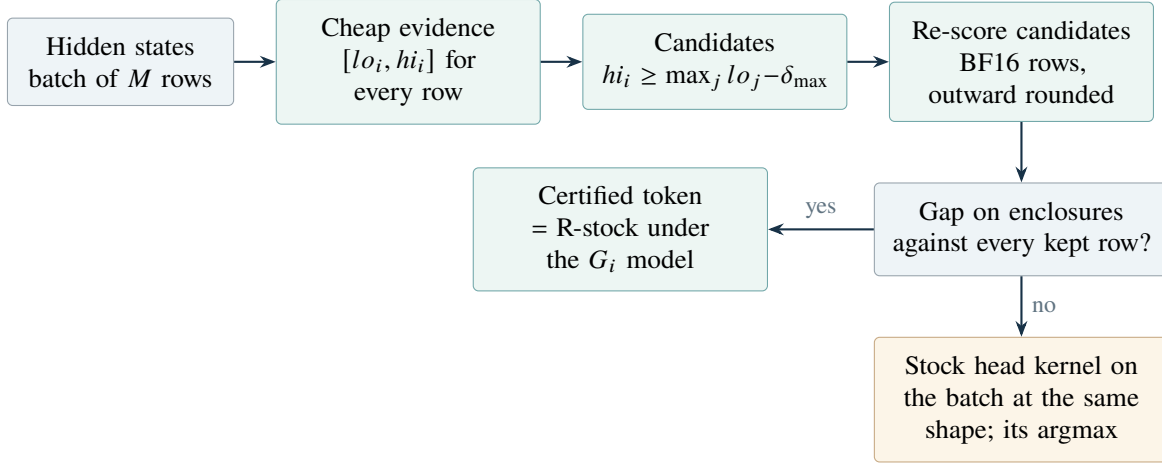


Figure 2: The exactness contract. Cheap evidence (self-evidence or transport) and re-scoring decide R-real for most rows; the gap condition of Theorem 3.5, checked on enclosures against every competitor, converts that decision into the stock kernel’s decision when the margin covers the stock kernel’s modelled accumulation error and BF16 rounding. The candidate filter is widened by $\delta_{\max}$ so that every row it drops also meets the gap. A row that fails runs the stock kernel at the same batch shape, so its token is stock by construction. The kernel implementation is described in Section 3.5.

tests/test_precision.py; the gap condition refuses to certify this row). Using R-bf16 as the reference would report agreement with a computation the engine does not perform. R-real remains useful as a batch-invariant reference for replay studies, and the FP32-logit contract equals the engine run with `--enable-fp32-lm-head`; both are named as such wherever they are used.

3.5 How the fallback runs at the same batch shape

The gap condition needs a bound $|\hat{y}_i - z_i| \leq \gamma \sum_j |w_{ij} h_j|$ on the stock kernel’s accumulator. Two named models supply it. The *conservative* model charges $2D$ roundings of relative size 2^{-23} in any order; it covers any reduction tree, split- K with FP32 partials and truncating adders, and gives $\gamma = 6.1 \times 10^{-4}$ at $D = 2560$. The *blocked* model uses the published model of Hopper BF16 wgmma (16-term blocks, 25 fractional bits, truncation) plus an allowance for FP32 split- K , and gives $\gamma = 1.19 \times 10^{-4}$ [37]; it rests on that hardware model, not on vendor documentation. Neither covers split- K partials stored in BF16, which PyTorch permits by default, so for every batch size the engine uses we record which cuBLAS kernel runs and whether its output changes when reduced-precision reduction is disabled; a shape whose output changed goes to the fallback unconditionally. The same measurement records the largest observed $|\hat{y}_i - z_i| / \sum_j |w_{ij} h_j|$ against both models [pending: kernel: kernel per M , flag test and observed accumulation error; evidence/certified_head/stock_invariance.json].

The decision uses the BF16 rounding boundaries exactly rather than a one-spacing margin. After re-scoring, each candidate has an interval $[\hat{y}_i^-, \hat{y}_i^+]$ for the stock accumulator, and because rounding is monotone its stock logit lies in $[L_i^-, L_i^+] = [\text{rn}_B(\hat{y}_i^-), \text{rn}_B(\hat{y}_i^+)]$.

Proposition 3.6 (Interval form of the stock decision). *Let C be a set of rows such that every row $j \notin C$ has $L_j^+ < \max_{i \in C} L_i^-$, and let $k = \min\{i \in C : L_i^- = \max_{j \in C} L_j^-\}$. If $L_j^+ < L_k^-$ for every $j \in C$ with $j < k$ and $L_j^+ \leq L_k^-$ for every $j \in C$ with $j > k$, then R-stock returns k .*

Proof. For $j < k$, $L_j \leq L_j^+ < L_k^- \leq L_k$, so no earlier index attains the maximum. For $j > k$, $L_j \leq L_j^+ \leq L_k^- \leq L_k$, so a later index can at most tie, and the first-index rule then returns k . Rows outside C have $L_j \leq L_j^+ < L_k^- \leq L_k$ by the choice of C . $\square$

Two tokens whose logits round to the same BF16 value under every admissible accumulation are therefore certified, with the tie broken by index exactly as `torch.argmax` breaks it; only a tie that the accumulation error

could make or unmake is left undecided.

Undecided rows run the stock head. The assumption-free form reruns the same `torch.matmul` call on the whole batch at the same M , so cuBLAS selects the same kernel, and takes its argmax; that fallback needs no error model, and a fallback costs time, never agreement. Its price is the whole stock head. On replayed plain-decode states the gap condition alone leaves 2–3% of rows, and 15–22% of batches, to it; the interval form above leaves 0.35% of rows and 2.8% of the capture run’s batches (up to 16 requests) if the stock kernel follows the blocked Khattak–Mikaitis model, which is an assumption, and 1.40% of rows and 10.9% of those batches under the conservative model (Section 6.4). A cheaper form relies on a property of the stock kernel that has to be measured on this cuBLAS build, namely that a row’s BF16 logits are bitwise the same whether it is computed alone, in any subset of the batch, or against any subset of head rows; a row undecided only because of a near tie among a complete candidate list is then completed by one stock `matmul` over its gathered candidate rows, whose own values and tie rule decide it. That mode’s claim is conditional on the measured invariance [pending: kernel: invariance by batch size; evidence/certified_head/stock_invariance.json]. Inside a CUDA graph each fallback is a conditional node whose predicate a routing kernel sets on the device, so a captured step has a fixed topology and pays for the fallback only when it runs [pending: kernel: undecided rows and batches by error model and fallback mode on replayed decode steps; evidence/certified_head/replay_decisions.json].

3.6 Every divergence has a mechanism

Before a certified head can claim that it adds no divergence, the unmodified engine’s disagreement with itself has to be known. Stock configurations that ought to be equivalent already disagree: plain decoding at different batch sizes, plain decoding against MTP speculation, and speculation at different batch sizes can emit different greedy tokens. Both greedy paths reduce to one rule, `torch.argmax` (first maximal index) over the BF16 head output widened exactly to FP32, so two runs that feed identical logits to it cannot choose different tokens. Every divergence therefore means that the computed logits differed, and the question is where. “A tie” is an observation about where the divergence happened, not an explanation of why.

Differential tests. A fixed set of 320 prompts from five public datasets, tokenized once so that every configuration receives identical token IDs, is decoded greedily for up to 256 tokens with the top five logprobs at every position. The configurations differ only in settings that should leave greedy output unchanged: plain decoding against MTP speculation (one, three and five draft steps with top- k 1, and a small tree with top- k 2), the prefix cache and the overlap scheduler on or off, `--enable-deterministic-inference` and `--enable-fp32-lm-head`, each with one request in flight and with 32. Two runs of a prompt are compared position by position; the first position where the tokens differ is a divergence, the positions before it are the exposure, and the rate per 1,000 tokens is divergences over exposure.

Locating the first difference. A diagnostic tap hashes the exact bits of every module output row by row inside the CUDA graphs and dumps the rows of selected requests with the head input and the logits fed to the argmax; tapped and untapped runs must agree bitwise. For each divergence both configurations are replayed on the prompt and the committed rows walked in order to the first position and module whose output bits differ; before that point every tensor, and therefore every cached attention and recurrent state, agrees. At the divergence the classification asks whether the head inputs agree (then the head GEMM itself produced different BF16 values for the same input) and, if not, whether the exact logits of the two competing tokens, recomputed in FP64 from each run’s head input, are ordered the same way in both runs (a rounding flip: BF16 rounding created the tie or reversal) or differently (an order flip carried by the hidden state).

[pending: state: divergence rate per 1,000 tokens and mechanism class for each configuration pair, the first differing module, and the rate of exact BF16 top-two ties in plain decoding; evidence/state_safety/noise_floor.csv, evidence/state_safety/mechanism_plain_c1_vs_c32.json]

[pending: state: whether deterministic inference makes plain decoding and MTP speculation batch-invariant, and the kernel responsible where it does not]

The practical consequence for this paper is that a certified head is compared with the stock head at the same batch shape and in the same serving configuration, where stock is bitwise reproducible, never with a stock run at another shape or an earlier run of another configuration; otherwise the comparison measures the stock noise floor rather than the certificate.

3.7 Unfinished work is dead, deferred or enclosed

Skipping a computation does not remove its cost unless nothing downstream needs its result. Every skipped result therefore falls into one of three classes. It is *dead* when it has no remaining consumer. It is *deferred* when a later consumer needs it and a reconstruction path exists; the cost is moved, not eliminated, and must be charged where it lands. It is *soundly enclosed* when a certificate bounds it well enough for every consumer that remains. A claimed saving should say which class it belongs to, because early exits and approximations can borrow time from future tokens.

The head is dead work in the relevant sense: once a token is committed, no future computation reads the logits. In target-only decoding of Qwen3.5-4B the same is true of more. The last decoder layer is a full-attention layer (Section 1.1); its attention writes the key–value cache before its MLP runs, and the final MLP, the final norm and the head have no consumer in future state. A certificate that decides the token before that tail could therefore remove about 1.41 GB of the 8.41 GB streamed per step (the head plus the last layer’s MLP, Table 1), and the removal would be permanent. The same cut does not transfer to speculation without re-analysis: the MTP layer and the DFlash drafter consume target hidden states, so work that is dead in plain decoding becomes deferred there. Section 6.8 states the certified-tail proposal and the kill test that rejected it.

3.8 Recurrent state under speculation

Qwen’s hybrid architecture makes state correctness part of verification. Reusable state includes attention KV, recurrent tensors, convolution history, positions and backend metadata. With state orientation $S \in \mathbb{R}^{d_v \times d_k}$, the gated-delta update can be written

$$S_t = \alpha_t S_{t-1} + u_t k_t^\top, \quad u_t = \beta_t (v_t - \alpha_t S_{t-1} k_t), \quad (1)$$

which is the Gated DeltaNet rule $S_t = S_{t-1} \alpha_t (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ [115] with the update vector made explicit. The update depends on the preceding state.

Proposition 3.7 (Replay of valid derived records). *If (α_t, u_t, k_t) were recorded along a causal trajectory from S_0 , applying the first a records to that same S_0 reproduces S_a in exact arithmetic:*

$$S_a = \left(\prod_{j=1}^a \alpha_j \right) S_0 + \sum_{t=1}^a \left(\prod_{j=t+1}^a \alpha_j \right) u_t k_t^\top.$$

Proof. Substitute the expansion at $a - 1$ into (1); the new gate multiplies every preceding term and the new outer product adds the last summand. Induction completes the proof. $\square$

The qualification about S_0 matters: replaying a derived u_t from a different state gives the wrong result even when the token IDs agree. The inherited CPU suite contains a one-dimensional witness (a record made at state zero and replayed from state two gives three, where the genuine update gives one). Replayng raw inputs with the true transition is a different operation. Regrouping the recurrence also does not preserve floating-point results; SGLang’s replay fold constrains its operation order and warp count to be a bitwise clone of the verify kernel [84].

A cycle may emit a correction or bonus token before that token has been consumed into every cache, so the runtime must distinguish the *emitted* prefix from the *materialized* state prefix; treating both as one accepted length

invites off-by-one state errors. Publication of speculative state should check that the request’s slot and epoch are still current, and memory must not be reused while any stream or captured graph can still write it. An epoch check does not by itself prevent a stale kernel from writing reused memory; ownership and delayed reuse are separate obligations.

What SGLang does, and how it is tested (H5). During verification SGLang computes the recurrent state and the convolution window after every draft position into per-position buffers and leaves the live state untouched; after acceptance a fused scatter copies the buffers of the last accepted position (for a tree, the last accepted node) into the request’s slot. The emitted prefix includes the bonus token while the materialized state does not; the bonus token is the first input of the next cycle. Prefix reuse checkpoints the recurrent state every 256 tokens of sequence length into a ping-pong pair of slots, including checkpoints that fall inside a verify cycle, and a freshly allocated slot is cleared by the next forward on the same stream, after any in-flight verification by its previous owner. The MTP draft layer is a full-attention layer, and under greedy verification every committed token is the target’s argmax, so an error in draft-side state could lower acceptance but not change the output. Each point where a speculative cycle touches this state is tested against a reference that isolates it: truncation by `max_new_tokens` and stop tokens at every index inside a verify cycle must reproduce the untruncated run bitwise on the same server; checkpoints taken during speculative decoding, including in the cycle that finished the request, are restored for a continuation and compared with a cold prefill of the same prefix; requests aborted mid-stream with a recurrent pool of exactly four slots force the next request onto the freed slot while the aborted request’s last verification may still be in flight; chunked prefill at 200 and 256 tokens is compared with an unchunked prefill; and the logprob drift of every MTP run against plain decoding is bucketed by the draft position that the previous cycle rejected.

[pending: state: outcome of each targeted test; evidence/state_safety/targeted.json]

3.9 How much recurrent state an exact computation needs

The dense FP32 state is sufficient for exact decoding; nothing yet shows that it is necessary. Two questions should be kept apart: whether less information could be retained, and whether the state has to be materialized as often as it is.

For the first, fix one GDN head of one layer and treat the layer’s inputs as given. Along any permitted history the state evolves as $S_t = S_{t-1}A_t + B_t$ with $A_t = \alpha_t(I - \beta_t k_t k_t^\top)$ and $B_t = \beta_t v_t k_t^\top$, which depend on the inputs and not on the state, and the head reads $o_t = S_t q_t$. A state difference ΔS at time t therefore propagates linearly, to $\Delta S A_{t+1} \cdots A_{t+j}$, and is observed as $\Delta S A_{t+1} \cdots A_{t+j} q_{t+j}$.

Proposition 3.8 (What an exact state must retain). *Let $\mathcal{S}$ be the set of states reachable over permitted histories, $\mathcal{D} = \text{span}\{S - S' : S, S' \in \mathcal{S}\}$, and $\mathcal{N}$ the set of differences ΔS with $\Delta S A_{t+1} \cdots A_{t+j} q_{t+j} = 0$ for every permitted continuation and every $j \geq 0$. Two reachable states produce the same outputs under every permitted continuation if and only if their difference lies in $\mathcal{N}$. If $\mathcal{S}$ contains a relatively open subset of its affine hull, then a linear map that separates every pair of reachable states with different outputs has rank at least $\dim \mathcal{D} - \dim(\mathcal{D} \cap \mathcal{N})$ on $\mathcal{D}$.*

Proof. Because B_t does not depend on the state, the output difference under a continuation is the linear expression above, and it vanishes for every continuation exactly when $\Delta S \in \mathcal{N}$. For the bound, take $d \in \mathcal{D}$ with $\phi(d) = 0$. By the hypothesis there are a reachable S and an $\varepsilon > 0$ with $S + \varepsilon d$ reachable; ϕ gives them the same image, so they must have the same outputs, and $\varepsilon d \in \mathcal{N}$. Hence $\ker \phi \cap \mathcal{D} \subseteq \mathcal{N} \cap \mathcal{D}$, and the rank of ϕ on $\mathcal{D}$ is at least $\dim \mathcal{D} - \dim(\mathcal{D} \cap \mathcal{N})$. $\square$

The hypothesis matters: if only one state is reachable, $\dim \mathcal{D} = 0$ and a representation with no dimensions is exact. Treating every input sequence as permitted is conservative, since in the full model a layer’s outputs influence later tokens; it is the notion that makes a reduction safe for any request. Exact witnesses show that neither $\mathcal{D}$ nor $\mathcal{N}$ is favourable without structure in the keys (tests/test_state_structure.py, evidence/state_structure/). Writes with unit coordinate keys and gates inside $(0, 1)$ reach any matrix, so the reachable set can fill the whole

state space and $\mathcal{D}$ with it. A direction the current query cannot see is not unobservable: with $\alpha = \beta = \frac{1}{2}$, $k = (\frac{3}{5}, \frac{4}{5})$, $q = e_1$ and $\Delta S = ue_2^\top$ for any value vector u , the observed difference $\Delta S q$ is zero now, but one step later it is $\Delta S A q = -\frac{3}{25}u$. Each transition is a contraction, not an erasure: $\det A_t = \alpha_t^d (1 - \beta_t \|k_t\|^2) > 0$ when $\beta_t \|k_t\|^2 < 1$, so no step forgets a direction exactly. Transitions with non-orthogonal keys do not commute, so there is no common basis in which they are diagonal. And because a gated RMSNorm precedes the output projection, the null space of the projection is not unobservable: two outputs that $W_o = (1, 0)$ cannot tell apart, $(1, 1)$ and $(1, 7)$, project to 1 and $\frac{1}{5}$ after normalization.

One structural reduction is exact. If every key lies in the column space of a fixed orthonormal $U \in \mathbb{R}^{d_k \times r}$ and $S_0 = Z_0 U^\top$, then $S_t = Z_t U^\top$ with

$$Z_t = \alpha_t Z_{t-1} + \beta_t (v_t - \alpha_t Z_{t-1} \kappa_t) \kappa_t^\top, \quad \kappa_t = U^\top k_t,$$

the same recurrence in dimension r , since $U^\top A_t (I - UU^\top) = 0$ and the write $\beta_t v_t \kappa_t^\top$ stays in the form exactly when $k_t \in \text{col}(U)$. The keys are produced by a short convolution, a SiLU and a normalization, and SiLU raises rank (SiLU($\log 2 \llbracket [1, 2], [2, 4] \rrbracket$) has rank two), so a low-dimensional input does not imply low-dimensional keys; whether real keys are confined to a subspace is a measurement, to be certified by exact or modular rank rather than an SVD threshold [pending: rank and observability audit of captured q, k, α, β traces]. With U fixed per layer and head and shared across requests, the per-request state is $d_v r$ values instead of $d_v d_k$, a saving for every $r < d_k$; a per-request low-rank factorization $S = AB^\top$ instead stores $d_v r + r d_k$ values, which beats the dense state only below $r = 64$ at $d_v = d_k = 128$ and halves it only at $r \leq 32$ (derived). Floating point adds a further obstruction: an exact rational reduction does not preserve bitwise results. Evaluating the recurrence in the split form $S_t = S_0 P_t + W_t$, with P_t the product of the transitions and W_t the recurrence started from zero, is exact in real arithmetic, yet over 50 steps at width 32 it changed 1,481 of 1,600 FP32 outputs, and 2 of them after rounding to BF16 (evidence/contracts/). Deferring a decay likewise reassociates FP32 roundings (one ulp apart for $\alpha_1 = \alpha_2 = 0.9$, $x = 1.3$) and rounding an exactly rank-one BF16 outer product to BF16 gives a 32×32 matrix of exact rank 28 (tests/test_state_structure.py, evidence/state_structure/).

Errors that do enter the state stay bounded as long as each transition is non-expansive: $(I - \beta k k^\top)$ has spectral norm at most 1 exactly when $0 \leq \beta \|k\|^2 \leq 2$ (its eigenvalues are 1 and $1 - \beta \|k\|^2$), so a perturbation η_t injected at each step obeys $\|E_t\| \leq |\alpha_t| \|E_{t-1}\| + \|\eta_t\|$; with normalized keys and $\beta \in (0, 1)$ the condition always holds.

The second question has a better answer. A decoder that keeps a dense anchor and the recent actual FP32 operands of each step, and replays them in the reference kernel's order, reproduces every state word bit for bit by induction, while reading the dense state once per step and writing it once every m steps. That reduces information movement, not information, and it is what replay-based rollback already does [54, 84]. Section 10 treats it as a lever with a pre-registered test.

3.10 Useful progress and elapsed time

Let a cycle commit R output tokens in elapsed time $T > 0$. Under a stationary regenerative model with finite expectations the long-run rate is

$$\lambda = \frac{\mathbb{E}R}{\mathbb{E}T}, \quad (2)$$

the ratio of accumulated reward to accumulated time, not $\mathbb{E}[R/T]$, and not a queueing-latency model. A change improves the rate exactly when $\mathbb{E}R_1/\mathbb{E}R_0 > \mathbb{E}T_1/\mathbb{E}T_0$. Higher acceptance does not compensate for arbitrary overhead, and lower acceptance can be worthwhile when the saved work exceeds the lost progress. The denominator includes drafting, verification, the head, refinement, completion, state commit, non-overlapped dispatch and control. A fixed-width target pass evaluates all admitted positions even when the first proposal is rejected; overlapped kernels require a critical-path measurement rather than a sum of kernel durations.

Consumer	Required information	Certificate here
Greedy token	Winning index under the reference kernel and tie rule.	Ladder plus gap condition (Sections 3.4, 6.3).
Exact top- k	Winning indices and scores; no partition.	Ladder on membership.
Seeded or fixed-noise sample	Argmax of perturbed scores under the keyed field.	Certified race (Section 7.1).
Acceptance bit	$\exp(z_X)$ against $U_{q_X}Z$; an interval can suffice.	Guards (Section 7.4; analysis). The engine path uses fixed-noise verification instead (Section 7.3).
Residual sample	The $(p - q)_+$ law, including omitted mass.	Residual races (Section 7.5).
Bonus sample	A sample from the target law under the RNG contract.	Certified race.
Top- p sample	Mass above each token relative to Z .	Partition bounds, or fall back.
Arbitrary log-probabilities	Requested logits and the full partition.	None; ordinary path.

Table 5: Consumers of the head and what each needs. The last rows are where a narrow fast path usually loses.

3.11 What each consumer needs

Table 5 lists the information each consumer of the head requires. A fast path is admitted by these contracts, not by a request being labelled “decode”. Grammar masks, repetition penalties and temperature transforms are part of the scores; they cannot be applied after candidates have been discarded.

4 Where the time goes

A component that occupies a fraction f of a step can speed the step up by at most $1/(1 - f)$, so the head’s share of the time bounds what any head certificate can deliver, and the other shares say where the rest of the stack leaves room. This section measures those shares for SGLang serving Qwen3.5-4B in its production configuration: CUDA graphs for prefill and decode, the overlap scheduler, FlashInfer attention and greedy decoding, at batch sizes from 1 to 128 for plain decoding and from 1 to 32 for MTP speculation.

Decode on this GPU is dominated by streaming bytes, so the byte counts of Table 1 give a first picture before any trace. Figure 3 draws the derived bytes of one plain decode step against the batch size at a fixed context. The weights are streamed once whatever the batch; the recurrent state and the attention cache are per request. With FP32 GDN state that the decode kernel reads and writes once per step, the per-request term equals the weights near a batch of 70 at a 512-token context and dominates beyond it. The head’s share of the bytes falls accordingly, from 15% at batch one.

4.1 Attribution method

[pending: profile: workload and profiled windows (fixed concurrency, prefill excluded); nsys session tracing with CUDA-graph node records; step reconstruction from graph-replay correlation IDs; kernel categories and GEMM labelling; idle time split inside and outside replays; profiler perturbation against uncollected twin windows; the measured HBM read peak used as the bandwidth reference; evidence under evidence/profiles/]

4.2 The head in plain decode and in speculation

[pending: profile: the head chain (GEMM, FP32 copy, argmax) as a share of the plain decode step at $B = 1, 8, 32, 128$;

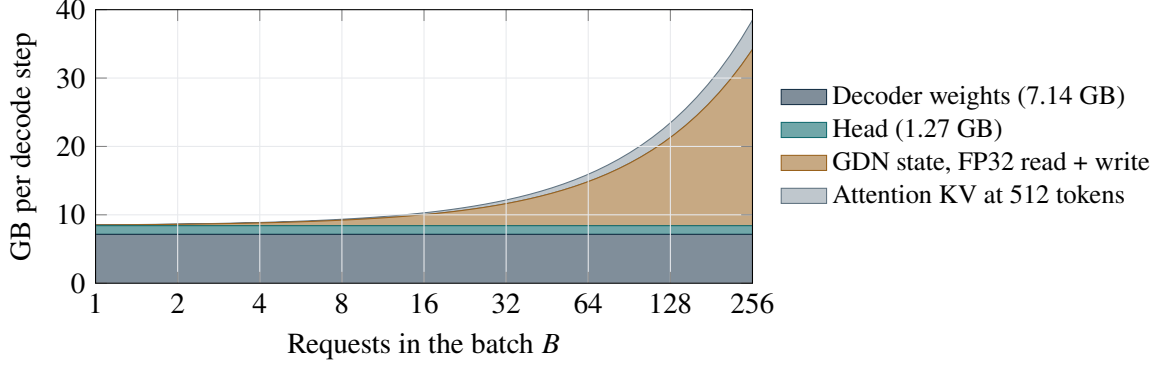


Figure 3: Derived bytes per plain decode step against batch size, from the checkpoint’s tensor sizes and configuration (Table 1): weights and head once per step, 100.7 MB of GDN state and 16.8 MB of attention KV (512 context tokens) per request. A calculation, not a measurement; logits and convolution state are omitted. The measured attribution is [pending: profile, PR #13].

the head microbenchmark of the exact calls SGLang makes at $M = 1$ to 256; the MTP cycle’s target and draft head chains at $B = 1, 8, 32$; evidence evidence/profiles/attribution/, evidence/profiles/head_microbench.json]

Pending figure. Share of a plain decode step and of an MTP cycle by component (target head chain, draft heads, MTP layer, weight GEMMs, GDN state kernels, attention, small kernels, idle) at each batch size.

Data: evidence/profiles/breakdown.csv

4.3 Recurrent state and host gaps

[pending: profile: measured GDN decode-kernel time per request and its implied bandwidth; bytes per step by component with the measured-kernel check; the per-position FP32 state writes of MTP verification; GPU idle time and the host call sites on the critical path of a speculative cycle; evidence evidence/profiles/bytes_per_step.json, evidence/profiles/bytes_per_step_sweep.csv]

5 Transport on real hidden states

Transport was the central mechanism of the earlier revisions. This section states it with the hypotheses that the proofs need, derives the condition under which it can beat the alternative, and reports what happened when real draft and target states were compared.

5.1 Tile geometry and already-paid summaries

Let $W \in \mathbb{R}^{V \times D}$ be the head used for both a draft hidden vector h^d and an aligned target hidden vector h^t , both taken at the head input (after the final norm). Write $\Delta = h^t - h^d$, $z_i^d = \langle w_i, h^d \rangle$ and $z_i^t = \langle w_i, h^t \rangle$. A vocabulary partition C covers all admitted tokens with disjoint tiles. For each tile c choose a centre μ_c and a radius

$$r_c \geq \max_{i \in c} \|w_i - \mu_c\|_\infty, \quad a_c = \langle \mu_c, \Delta \rangle, \quad \epsilon_c = r_c \|\Delta\|_1. \quad (3)$$

The draft pass emits $M_c^d = \max_{i \in c} z_i^d$ and $Z_c^d = \sum_{i \in c} \exp(z_i^d)$; stable implementations store a maximum and a scaled mass. The centres and radii are checkpoint metadata; the maxima and masses belong to one request, position and draft vector.

Theorem 5.1 (Transported tile certificates). *In real arithmetic, with the same head matrix for both passes and both masses taken at the same temperature, every $i \in c$ satisfies*

$$z_i^d + a_c - \epsilon_c \leq z_i^t \leq z_i^d + a_c + \epsilon_c. \quad (4)$$

Consequently

$$\max_{i \in c} z_i^t \leq M_c^d + a_c + \epsilon_c =: U_c, \quad (5)$$

$$e^{a_c - \epsilon_c} Z_c^d \leq Z_c^t \leq e^{a_c + \epsilon_c} Z_c^d. \quad (6)$$

Proof. $z_i^t - z_i^d - a_c = \langle w_i - \mu_c, \Delta \rangle$, whose absolute value is at most $r_c \|\Delta\|_1$ by Hölder's inequality. Maximization gives (5); monotonicity of the exponential and summation give (6). $\square$

In the engine the draft maxima and masses come from BF16-rounded draft logits, so (5) and (6) must be widened by the draft kernel's error before they enclose anything the engine computes; Section 6 gives the terms. The Euclidean form replaces r_c by $r_c^{(2)} \geq \max_{i \in c} \|w_i - \mu_c\|_2$ and $\|\Delta\|_1$ by $\|\Delta\|_2$.

5.2 A summary that supports both selection and mass

For a set of labelled logits, keep its top k items under descending score then ascending token index, and the mass of every discarded item. Two such summaries merge by taking the top k of their retained union and adding the newly discarded mass to the two old tails.

Proposition 5.2 (Top- k plus tail is a compositional summary). *For disjoint finite domains the merge produces the same top- k items and tail mass as summarizing the union, and it is associative and commutative in exact arithmetic under the common total order.*

Proof. An item omitted from a local top- k has at least k locally superior items and cannot enter the union's top- k , so every possible winner is retained. The original tails and the newly discarded items partition the final tail, so summing them gives its mass without subtracting nearly equal totals. Every parenthesization and order represents the same union. $\square$

This is a sufficient-statistic construction from familiar reductions, and floating-point tail masses are not associative. Its value is an interface: one epilogue can serve candidate generation, bounds and completion. Retaining a small set R_c per tile also tightens the mass bounds: recompute its target logits exactly and transport only the rest,

$$\sum_{i \in R_c} e^{z_i^t} + e^{a_c - \epsilon_c} Z_{c \setminus R_c}^d \leq Z_c^t \leq \sum_{i \in R_c} e^{z_i^t} + e^{a_c + \epsilon_c} Z_{c \setminus R_c}^d, \quad (7)$$

which is never looser than transporting the whole tile, at the cost of $|R_c|$ extra row projections.

5.3 Geometry decides

A good draft does not imply a small $\|h^t - h^d\|$. The head can ignore directions in which the states differ, and probabilities ignore a common shift of all logits, yet a crude hidden-space norm charges both. With $W = ((1, 0), (0, 0))$ and $\Delta = (0, L)$ both logits are unchanged for every L , while $r_c \|\Delta\|_1$ grows with L ; coordinate radii remove that particular failure. The bounds form a hierarchy: a scalar radius; coordinate radii with $\epsilon_c = \sum_j r_{cj} |\Delta_j|$; grouped norms

$$\epsilon_c = \sum_g r_{cg} \|\Delta_g\|_2, \quad r_{cg} \geq \max_{i \in c} \|(w_i - \mu_c)_g\|_2; \quad (8)$$

and a low-rank split $w_i = \mu_c + Qu_i + e_i$ with the projected part evaluated exactly and a norm bound on e_i . Richer geometry costs metadata and bound arithmetic, and a learned centre or subspace does not certify the residual radius; that radius must cover the stored weights.

5.4 Greedy and top- k admission

Suppose at least k distinct target rows have been evaluated and τ is the k -th best evaluated score. A tile may be skipped only when $U_c < \tau$; then at least k evaluated items outrank every skipped one. The strict inequality preserves unresolved ties. This establishes exact selection; it does not establish an exact unrestricted softmax, because an omitted logit still contributes to the normalizer even when it cannot win.

5.5 What transport must beat

The alternative source of evidence (Section 6) evaluates a low-precision copy $\bar{w}_i = w_i - e_i$ of each row at the target state. In real arithmetic its error for row i is $\langle e_i, h^t \rangle$, bounded by $\|e_i\|_2 \|h^t\|_2$, where e_i is known offline. Transport's error is $\langle w_i - \mu_c, \Delta \rangle$. The two can be compared exactly.

Proposition 5.3 (No better bound from the same information). *For fixed h and radii $\epsilon_b \geq 0$ over a partition of the coordinates into blocks b , $\sup\{|\langle e, h \rangle| : \|e_b\|_2 \leq \epsilon_b \ \forall b\} = \sum_b \epsilon_b \|h_b\|_2$ and $\sup\{|\langle e, h \rangle| : \|e_b\|_\infty \leq \epsilon_b \ \forall b\} = \sum_b \epsilon_b \|h_b\|_1$. Likewise $\sup\{\langle w - \mu, \Delta \rangle : \|w - \mu\|_\infty \leq r\} = r \|\Delta\|_1$.*

Proof. The inequalities are Cauchy–Schwarz and Hölder. Equality holds at $e_b = \epsilon_b h_b / \|h_b\|_2$, at $e_b = \epsilon_b \text{sign}(h_b)$ and at $w = \mu + r \text{sign}(\Delta)$. $\square$

Both envelopes are therefore optimal for the metadata they use. For errors not aligned with h the realized $|\langle e_i, h \rangle|$ is typically about $\|e_i\|_2 \|h\|_2 / \sqrt{D}$; closing that gap needs more arithmetic, not a better inequality.

Proposition 5.4 (Exclusion conditions). *After the winner a is scored exactly, let $m_i = z_a - z_i$. Row-level transport excludes i if and only if $m_i > r_c \|\Delta\|_1 - \langle w_i - \mu_c, \Delta \rangle$; tile-level transport excludes tile c if and only if $z_a > M_c^d + \langle \mu_c, \Delta \rangle + r_c \|\Delta\|_1$; self-evidence (in real arithmetic, with envelope β_i) excludes i if and only if $m_i > \beta_i - \langle e_i, h \rangle$. The slack terms lie in $[0, 2r_c \|\Delta\|_1]$ and $[0, 2\beta_i]$.*

Proof. Substitute $z_i^d + \langle \mu_c, \Delta \rangle = z_i - \langle w_i - \mu_c, \Delta \rangle$ and $\tilde{z}_i = z_i - \langle e_i, h \rangle$ into the upper bounds. $\square$

Theorem 5.5 (Transport cannot beat self-evidence unless drift is small). *For every tile c and every centre μ_c , $r_c = \max_{k \in c} \|w_k - \mu_c\|_\infty \geq \frac{1}{2} \text{diam}_\infty(c)$ and $r_c^{(2)} = \max_{k \in c} \|w_k - \mu_c\|_2 \geq \frac{1}{2} \text{diam}_2(c)$.*

1. (Hölder family.) *With b -bit round-to-nearest per-row codes, $s_i = \|w_i\|_\infty / (2^{b-1} - 1)$ and no clipping, $|\langle e_i, h \rangle| \leq \frac{s_i}{2} \|h\|_1$, and $\frac{s_i}{2} \|h\|_1 \leq r_c \|\Delta\|_1$ whenever $\|\Delta\|_1 / \|h\|_1 \geq \|w_i\|_\infty / ((2^{b-1} - 1) \text{diam}_\infty(c))$. If some row of c has the opposite sign, or zero, in the coordinate where $|w_{ij}|$ peaks, then $\text{diam}_\infty(c) \geq \|w_i\|_\infty$ and the condition is implied by $\|\Delta\|_1 / \|h\|_1 \geq 1 / (2^{b-1} - 1)$: $1/127$ for $\text{int}8$ and $1/7$ for $\text{int}4$.*
2. (Euclidean family.) *$\|e_i\|_2 \|h\|_2 \leq r_c^{(2)} \|\Delta\|_2$ if and only if $\|\Delta\|_2 / \|h\|_2 \geq \delta_i^* := \|e_i\|_2 / r_c^{(2)}$.*

Proof. For rows k, l of the tile, $\|w_k - w_l\| \leq \|w_k - \mu_c\| + \|w_l - \mu_c\| \leq 2r_c$. Round-to-nearest codes give $|e_{ij}| \leq s_i/2$, and Hölder gives the first inequality. The rest is algebra, and opposite signs at the peak coordinate give $|w_{ij} - w_{kj}| \geq |w_{ij}|$. $\square$

Proposition 5.6 (Transport against the static screen). *If μ_c lies in the convex hull of the tile's rows, then $U_c^T - U_c^{\text{static}} \geq r_c (\|\Delta\|_1 - \|h\|_1)$, where $U_c^T = M_c^d + \langle \mu_c, \Delta \rangle + r_c \|\Delta\|_1$ and $U_c^{\text{static}} = \langle \mu_c, h \rangle + r_c \|h\|_1$. Transport is therefore no tighter than the static screen with the same geometry once $\|\Delta\|_1 \geq \|h\|_1$.*

Proof. $U_c^T - U_c^{\text{static}} = (M_c^d - \langle \mu_c, h^d \rangle) + r_c (\|\Delta\|_1 - \|h\|_1)$, and $M_c^d = \max_k \langle w_k, h^d \rangle \geq \langle \mu_c, h^d \rangle$ for a convex combination μ_c . $\square$

The quantity that decides the comparison is the *drift ratio*

$$\rho = \frac{\|h^t - h^d\|_2}{\|h^t\|_2}, \quad (9)$$

Quantity (tied head, $248,320 \times 2,560$)	p10	p50	p90
Row $\ w_i\ _2$	0.574	0.652	0.746
int8 per-row $\ e_i\ _2 / \ w_i\ _2$	0.81%	0.97%	1.25%
int8, 128-column groups	0.64%	0.68%	0.75%
int4 per-row	14.8%	17.5%	22.6%
64-row tile $r_c^{(2)} / \ w_i\ _2$, token-index order	0.99	1.14	1.31
random tiles (control)	1.03	1.19	1.37
δ^* against int8 per-row	0.67%	0.85%	1.15%
δ^* against int8, 128-column groups	0.51%	0.61%	0.72%
δ^* against int4 per-row	12.2%	15.5%	20.8%
Hölder-family threshold on $\ \Delta\ _1 / \ h\ _1$ against int8 per-row	0.23%	0.30%	0.40%

Table 6: Weight-only constants of the Qwen3.5-4B head (float64 on CPU; quantiles over a one-million-row sample; no hidden states). $\delta_i^* = \|e_i\|_2 / r_c^{(2)}(i)$ is the drift ratio ρ below which transport’s Euclidean envelope for row i is narrower than the Euclidean int8 envelope, with 64-row tiles of contiguous token indices and μ_c the tile mean (Theorem 5.5); the last row is the analogous threshold for the ℓ_∞/ℓ_1 pairing, $s_i \|h\|_1 / 2$ against $r_c \|\Delta\|_1$. Widths are compared, not certification rates or runtime. Derived; evidence/precision/head_constants.json.

measured on the exact head inputs behind the same verified draft token. In the Euclidean family, with 64-row tiles of contiguous token indices and μ_c the tile mean, transport’s envelope for row i is narrower than the per-row int8 self-evidence envelope if and only if $\rho < \delta_i^* = \|e_i\|_2 / r_c^{(2)}(i)$. Table 6 evaluates δ^* on the Qwen3.5-4B head from the weights alone. Its median is 0.85% against per-row int8 and 15.5% against per-row int4. For the manuscript’s original pairing of an ℓ_∞ radius with $\|\Delta\|_1$ (Theorem 5.5, part 1, with the tile radius measured rather than bounded by the diameter), the corresponding threshold on $\|\Delta\|_1 / \|h\|_1$ has a median of 0.30%. The comparison is of envelope widths, not of certification rates, candidate counts or runtime; it uses no hidden states, and it ignores the BF16 rounding of the draft logits that transport also has to carry. The measured drift decides this width comparison, not which mechanism is faster. The Hölder condition of Theorem 5.5 needs $\text{diam}_\infty(c) \geq \|w_i\|_\infty$, which holds for 99.9% of rows with 64-row tiles (evidence/precision/head_constants.json). It says that transport can compete with an int8 copy only if draft and target states agree to better than about one part in a hundred at the head input, which a drafter trained to predict tokens has no particular reason to achieve. The static tile screen is weak on this head for a related reason: the 64-row tile radius exceeds the row norm (median $r_c^{(2)} / \|w_i\|_2 = 1.14$), so $\langle \mu_c, h \rangle + r_c^{(2)} \|h\|_2$ is wider than the plain row bound $\|w_i\|_2 \|h\|_2$, and tiles in token order are barely tighter than random ones. A scalar ℓ_∞ radius paired with $\|\Delta\|_1$ is about four times looser again than the Euclidean pairing for a spread-out Δ (derived from the same constants, $r_c \approx 1.30 \|w_i\|_\infty$ and $\|\Delta\|_1 \approx 0.8\sqrt{D} \|\Delta\|_2$). The enclosures of the two mechanisms can be intersected, so a hybrid is never looser than either; the question is only whether transport’s cheaper evidence excludes enough to pay for itself.

5.6 Capturing the head inputs the engine used

Transport is a statement about two vectors fed to the same head: the draft input h^d that produced a proposal and the target input h^t that decides it. The serving engine exposes neither for the MTP drafter, so a capture-only patch (engine/sglang/patches/0001-head-capture-replay-dumps.patch) stashes the tensor LogitsProcessor hands to the head and joins it, inside the same verify step, with the proposed tokens, the target’s argmax and the accepted lengths. Hidden vectors are stored as BF16 bit patterns, so the replay reads exactly what the head read. The hooks run in Python between forwards, so the capture server runs without CUDA graphs and without the overlap scheduler; it is a correctness configuration, not a performance one. The prompts are 320 public prompts split by prompt into analysis and held-out halves before any fitting, decoded greedily for 384 new tokens.

The pairing follows the verify layout. A verify step for one request holds $[x_0, d_1, \dots, d_S]$ with x_0 the last

committed token; draft token d_k came from the draft input h_k^d and is decided by the target input at verify position $k - 1$. Each pair is labelled accepted, rejected or unreached, and the target input at the last verify position, which produces the bonus token, has no draft anchor at all: a transport-only verifier still streams the whole head for that row.

The pairing was confirmed by recomputation rather than assumed. For plain decoding, the FP64 argmax of the captured head input equals the engine’s token at 99.51% of 105,269 positions, and all 519 disagreements lie within one BF16 spacing of the FP64 winner (the largest gap is 0.118 logits where the spacing is 0.125). The argmax of the exact logits rounded to BF16, with the first-index rule, matches the engine at 99.998%; it is not the engine’s computation (Section 3.4), and the remaining positions are where the two differ. The engine’s own top two logits were equal in BF16 at 1.0% of positions, where the stock decision is set by the tie rule. R-real and R-stock therefore differ at about 0.5% of greedy positions here (evidence/head_geometry/alignment_plain4b.json).

5.7 The measured drift

The decisive measurement evaluates ρ (Equation (9)) on aligned held-out pairs, with the certification rates of every transport variant against the static screen at h^t and against self-evidence on the same states. The bound families are those of Section 5.3 (a scalar ℓ_∞ radius, coordinate radii, grouped ℓ_2 radii with groups of 32 and 128, and low-rank residual radii), each on contiguous, randomly permuted and k -means tiles of 64, 256 and 1,024 rows; with k -means tiles and an ℓ_2 radius the static screen is CSV-Decode’s. A tile is skipped when its transported upper bound falls below the exactly computed target score of the draft token, and the oracle threshold $\max_i z_i^t$ is reported as a ceiling. Two controls bound what any geometry could achieve: oracle radii, the realized per-tile deviation, which is the tightest radius any certified geometry could have; and a drift-scaling replay, $h^d(s) = h^t - s\Delta$, which scales ρ and every radius term by s and asks how far a drafter would have to move before transport certifies a useful share of decisions. For sampled verification the partition interval $\log(Z^+/Z^-)$ and P_7 of Theorem 7.8 are reported at temperatures 1 and 0.7. Head-metric versions of the drift, $\|W\Delta\|_2 / \|Wh^t\|_2$ after removing each position’s mean logit and the share of Δ in the leading right singular subspaces of W , test whether the drift hides in directions the head ignores.

[pending: geometry: the distribution of ρ for the MTP drafter by draft step and for the DFlash drafter by block position, split by outcome, domain and context length; certification rates, partition widths and P_7 by bound family and tiling; oracle-radius and drift-scaling results; evidence/head_geometry/rho_mtp4b_quantiles.csv, evidence/head_geometry/transport_mtp4b.json]

Pending figure. Empirical distribution of the drift ratio ρ on aligned MTP and DFlash head inputs, drawn against the band of δ^* quantiles from Table 6 (0.67%–1.15% against int8, 12.2%–20.8% against int4); ρ to the left of the band means transport’s envelope is the narrower one.

Data: evidence/head_geometry/rho_mtp4b_quantiles.csv

A skipped tile saves time only if the GPU avoids the work. For each mechanism the replay counts the weight bytes read (refined tiles including their union over the real batch, gathered candidate rows, the low-precision pass), metadata and evidence bytes, sequential stages and fallbacks, and converts them to predicted time with the kernel’s measured primitive costs; these are model predictions, and a runtime claim needs a measured kernel [pending: geometry: work counts and predicted time per mechanism; evidence/head_geometry/work_model.json].

6 Self-evidence: a certified low-precision head

A cheap approximation of the head can supply the evidence itself. The approximate pass bounds every row; rows whose interval can still matter are re-scored more precisely; exact arithmetic or the stock kernel completes whatever remains. Certified greedy screening of this form exists in public code (Section 2.3). What this section adds is an envelope that covers the floating-point arithmetic of the pass as the GPU performs it, including Hopper tensor-core accumulation under a published model, a refinement procedure that serves greedy selection and

sampling alike, and bounds on how much re-scoring each needs.

6.1 Accumulation error

Lemma 6.1 (Accumulation trees). *Let $s = \sum_{\ell} p_{\ell}$ be evaluated by a rooted tree whose leaves carry the exact values p_{ℓ} and whose internal nodes v return $\hat{s}_v$ with $|\hat{s}_v - \sum_c \hat{s}_c| \leq u_v \sum_c |\hat{s}_c| + \eta_v$ over the children c of v . Let d be the largest number of internal nodes on a leaf-to-root path, N the number of internal nodes, $u \geq \max_v u_v$, $\eta \geq \max_v \eta_v$ and $du < 1$. Then*

$$|\hat{s}_{\text{root}} - s| \leq \gamma_d \sum_{\ell} |p_{\ell}| + N\eta(1 + \gamma_d), \quad \gamma_d = (1 + u)^d - 1 \leq \frac{du}{1 - du}. \quad (10)$$

A node that multiplies its only child by a constant σ and rounds fits the model after multiplying the leaves below it by σ .

Proof. For a node v and a leaf ℓ below it let $P_{v\ell} = \prod (1 + u_x)$ over the internal nodes x on the path from ℓ up to v , and for an internal node w below v let P'_{vw} be the same product over the nodes strictly above w up to v ($P'_{vv} = 1$). By induction on height,

$$|\hat{s}_v| \leq \sum_{\ell} |p_{\ell}| P_{v\ell} + \eta \sum_w P'_{vw}, \quad |\hat{s}_v - s_v| \leq \sum_{\ell} |p_{\ell}| (P_{v\ell} - 1) + \eta \sum_w P'_{vw}.$$

Leaves satisfy both with equality. For an internal node, $|\hat{s}_v| \leq (1 + u_v) \sum_c |\hat{s}_c| + \eta$ and $|\hat{s}_v - s_v| \leq \sum_c |\hat{s}_c - s_c| + u_v \sum_c |\hat{s}_c| + \eta$; substituting the hypotheses for the children multiplies every P and P' by $(1 + u_v)$ and adds $P'_{vv} = 1$. Finally $P_{v\ell} \leq (1 + u)^d$, $P'_{vw} \leq (1 + u)^{d-1}$, and there are N internal nodes. $\square$

The IEEE model gives $u_v = u$ for a rounded addition, since $|a+b| \leq |a|+|b|$ [30]. Sequential accumulation from zero has $d = n$, pairwise summation $\lceil \log_2 n \rceil$, blocks of B summed as a tree and then accumulated $\lceil n/B \rceil + \lceil \log_2 B \rceil$, and split- K adds the number of partials. A multi-term adder that aligns $k+1$ addends to the largest, truncates each to F fractional bits below its leading bit, adds exactly and rounds once has $u_v = (k+1)2^{-F} + 2^{1-p}$ with truncation, and depth $\lceil n/k \rceil$. Tensor cores behave this way rather than as IEEE adders [20]. Khattak and Mikaitis model Hopper’s BF16 wgmma with FP32 output as $k = 16$, $F = 25$ with truncation [37], so $u_v = 17 \cdot 2^{-25} + 2^{-23}$ and $\gamma_{D/16} \approx 1.0 \times 10^{-4}$ at $D = 2560$; the conservative default $D \cdot 2^{-22} \cdot 1.001 = 6.1 \times 10^{-4}$ covers it about sixfold. This is a published model, not a documented contract, so kernels built on it are validated against exact arithmetic on adversarial inputs. The same model gives FP8 wgmma only 13 fractional bits in 32-term blocks, consistent with the accumulation limits reported for FP8 training [17]; an FP8 tensor-core pass needs periodic promotion to FP32 and its own analysis. On CUDA cores with IEEE round-to-nearest, γ at $D = 2560$ is about 1.7×10^{-6} for a blocked reduction and plays no role.

6.2 The envelope of the approximate pass

Row i is quantized with group scales $s_{ig} > 0$, integer codes q_{ij} and a set O of columns kept exact. The dequantized row is $\bar{w}_{ij} = s_{ig(j)} q_{ij}$ for $j \notin O$ and $\bar{w}_{ij} = w_{ij}$ for $j \in O$; the error $e_i = w_i - \bar{w}_i$ is computed exactly offline. The pass computes $\hat{t}_g = \sum_{j \in g} q_{ij} h_j$, $\hat{v}_g = \text{fl}(s_{ig} \hat{t}_g)$, sums the groups, adds $\sum_{j \in O} w_{ij} h_j$ and returns $\tilde{z}_i$.

Theorem 6.2 (Envelope). *Suppose every product $q_{ij} h_j$ and $w_{ij} h_j$ is exact in the accumulator format (true for int8 or BF16 weights times BF16 activations in FP32; underflow is covered by η) and every node obeys Lemma 6.1 with (u, η) and depth d . Then $|z_i - \tilde{z}_i| \leq \beta_i$ with*

$$\beta_i = Q_i + \gamma_d \|\bar{w}_i\|_2 \|h\|_2 + N\eta(1 + \gamma_d) \max(1, \max_g s_{ig}), \quad (11)$$

where Q_i may be any of $\|e_i\|_2 \|h\|_2$, $\sum_b \|e_{i,b}\|_2 \|h_b\|_2$, $\|e_i\|_{\infty} \|h\|_1$ or $\sum_b \|e_{i,b}\|_{\infty} \|h_b\|_1$ for any partition of the columns into blocks b (in particular O and its complement, so that only the norm of h outside O enters). Round-to-nearest codes without clipping have $\|e_{i,g}\|_{\infty} \leq s_{ig}/2$.

Proof. $z_i - \tilde{z}_i = \langle e_i, h \rangle + (\langle \bar{w}_i, h \rangle - \tilde{z}_i)$. The first term is bounded blockwise by Cauchy–Schwarz or Hölder. The second is the rounding error of evaluating $\langle \bar{w}_i, h \rangle$ by a tree whose leaves are $s_{ig}q_{ij}h_j$ and $w_{ij}h_j$; apply Lemma 6.1 and $\sum_j |\bar{w}_{ij}h_j| \leq \|\bar{w}_i\|_2 \|h\|_2$. $\square$

The componentwise midpoint-radius form $|z - \tilde{z}| \leq |e|^\top |h|$ is tighter than any of these relaxations and costs the same to evaluate when $|e|$ is stored per group [81]; the norm forms are used here because they need one constant per row or block.

Theorem 6.3 (Integer pass). *Let $h = s_h q_h + e_h$ with s_h a power of two and integer codes, so $s_h q_h$ and e_h are exact. Let $I_i = \sum_j q_{ij} q_{h,j}$ be accumulated exactly in 32-bit integers ($D q_{\max}^w q_{\max}^h < 2^{31}$; $2560 \cdot 127^2$ qualifies) and $\tilde{z}_i = \text{fl}(\text{fl}(s_i \text{fl}(I_i)) s_h)$ without overflow or underflow. Then*

$$|z_i - \tilde{z}_i| \leq Q_i + \|s_i q_i\|_2 \|e_h\|_2 + \gamma_2 \|s_i q_i\|_2 \|s_h q_h\|_2.$$

Proof. $z_i = s_i s_h I_i + s_i \langle q_i, e_h \rangle + \langle e_i, h \rangle$. The two roundings give $|\tilde{z}_i - s_i s_h I_i| \leq \gamma_2 s_i s_h |I_i|$, and the last multiply by a power of two is exact. Apply Cauchy–Schwarz to I_i and to $\langle q_i, e_h \rangle$. $\square$

This path needs no model of floating-point accumulation at all, because int8 products accumulated in int32 are exact for this shape [1]; it pays instead for quantizing the activation.

Lemma 6.4 (Computed norms). *In FP32 with round to nearest and $D \leq 2^{14}$, let $\hat{\sigma}$ be any computed sum of the rounded squares $\text{fl}(h_j^2)$ with depth $d \leq 2^{14}$, $\hat{r} \geq (1 - 2^{-16})\sqrt{\hat{\sigma}}$ any square root, and $N_2 = \text{fl}(\text{fl}(\hat{r}(1 + 2^{-10})) + 2^{-50})$. Then $N_2 \geq \|h\|_2$. Likewise, if $\hat{\sigma}_1$ is any computed sum of the $|h_j|$ with depth $d \leq 2^{13}$, then $N_1 = \text{fl}(\text{fl}(\hat{\sigma}_1(1 + 2^{-10})) + 2^{-50}) \geq \|h\|_1$.*

Proof. A rounded nonnegative value, and a rounded addition of nonnegative values, returns at least $(1 - u)$ times its exact value less μ (which also covers flush-to-zero). For N_2 the squares add one rounding to the d levels of the sum, so $\|h\|_2^2 \leq (1 - u)^{-(d+1)} \hat{\sigma} + 3D\mu$ and $\|h\|_2 \leq (1 - u)^{-(d+1)/2} (1 - 2^{-16})^{-1} \hat{r} + 2^{-54}$ for $D \leq 2^{14}$. The computed $N_2 \geq (1 - u)^2 (1 + 2^{-10}) \hat{r} + 2^{-51}$, and $(1 - u)^2 (1 + 2^{-10}) \geq (1 - u)^{-(d+1)/2} (1 - 2^{-16})^{-1}$ because $(d + 1)u/2 \leq 2^{-11} + 2^{-25}$. For N_1 the terms $|h_j|$ are exact, so $\|h\|_1 \leq (1 - u)^{-d} (\hat{\sigma}_1 + D\mu)$, and $(1 - u)^2 (1 + 2^{-10}) \geq (1 - u)^{-d}$ holds for $d \leq 2^{13}$ (then $du \leq 2^{-11}$) but fails at $d = 2^{14}$, where the margin is about -6×10^{-7} ; there N_1 must be inflated by $1 + 2^{-9}$ instead. $\square$

Lemma 6.5 (Outward rounding of the computed interval). *In a format with $p \geq 12$ and round to nearest, let $\tilde{z}$ be representable, let β^* be a real bound with $|z - \tilde{z}| \leq \beta^*$, and let b be computed from representable upper bounds by at most $k \leq 59$ rounded operations ($+$, $\times$, $\sqrt{}$) on nonnegative values along any path of the expression, so that $b \geq (1 - u)^k \beta^* - k\mu$. Set*

$$\hat{b} = \text{fl}(\text{fl}(\text{fl}(b(1 + 2^{6-p})) + \text{fl}(2^{4-p}|\tilde{z}|)) + 2^{e_{\min}+26}), \quad lo = \text{fl}(\tilde{z} - \hat{b}), \quad hi = \text{fl}(\tilde{z} + \hat{b}).$$

Then $lo \leq z \leq hi$, so every comparison between computed endpoints is sound.

Proof. Each rounding of a nonnegative value loses at most a factor $(1 - u)$ and μ , so $\hat{b}(1 - u) \geq (1 - u)^4 (1 + 64u)b + 16u(1 - u)^3 |\tilde{z}| + (1 - u)^2 2^{e_{\min}+26} - 4\mu$. Also $lo \leq \tilde{z} - \hat{b}(1 - u) + u|\tilde{z}| + \mu$. It suffices that $\hat{b}(1 - u) \geq \beta^* + u|\tilde{z}| + \mu$: the $|\tilde{z}|$ terms satisfy $16u(1 - u)^3 \geq u$, the b terms satisfy $(1 - u)^{4+k} (1 + 64u) \geq (1 - 63u)(1 + 64u) \geq 1$ for $u \leq 2^{-12}$, and $2^{26}\mu$ absorbs the absolute terms. The upper endpoint is symmetric. $\square$

For FP32 the constants are $1 + 2^{-18}$, $2^{-20}|\tilde{z}|$ and 2^{-100} . Offline constants ($\|e_i\|_2$, $\|\bar{w}_i\|_2$, $\|w_i\|_2$) are computed exactly and stored rounded up. The exact CPU reference emulates every BF16, FP32 and FP64 rounding on rationals and checks each bound against real arithmetic, including witnesses that the accumulation term, the outward rounding and the norm inflation are each necessary (tests/test_precision.py, evidence/precision/).

6.3 Certified argmax and its cost

Theorem 6.6 (Refinement ladder). *Let levels $\ell = 0, \dots, L$ supply enclosures $lo_i^\ell \leq z_i \leq hi_i^\ell$, exact at level L , and let f_i be nondecreasing maps. Starting from a row set S , at each level let $\tau = \max_{i \in S} f_i(lo_i^\ell)$ and keep $\{i \in S : f_i(hi_i^\ell) \geq \tau\}$; stop when one row remains, when every kept interval is a point, or at level L , and return the smallest kept index. The result is $\min \arg \max_{i \in S} f_i(z_i)$. With $f_i = \text{id}$ this is R-real, with $f_i = \text{rn}_B$ it is R-bf16, and with $f_i(x) = x/T + g_i$ it is S-race. The token does not depend on the precision or reduction order of any level.*

Proof. $[f_i(lo_i), f_i(hi_i)]$ encloses $f_i(z_i)$. A maximizer m is never dropped: for the row j that attains τ , $f_m(hi_m) \geq f_m(z_m) \geq f_j(z_j) \geq f_j(lo_j) = \tau$. A dropped row is strictly below the row attaining τ . If one row remains it is the maximizer. If all kept intervals are points, each kept value is at least τ and at most τ , so all kept rows are maximizers, and every maximizer is kept. The exact level produces points. The returned index is a function of the maximizer set alone. $\square$

The ladder decides R-real (or S-race). For the stock contract its filter is not enough on its own: rows dropped at $hi_i \geq \max_j lo_j$ are only known to lose, not to lose by the gap of Theorem 3.5. Under R-stock the level-0 filter is therefore widened by $\delta_{\max}$ and the gap is checked on enclosures against every kept row (Section 3.4); a row that fails runs the stock kernel (Figure 2). The Lean formalization proves the certificate, its uniqueness across levels and both filter properties in scaled integers (`formal/CertifiedArgmax.lean`).

Proposition 6.7 (Candidate counts). *Let the level-0 intervals have widths $\omega_i = hi_i - lo_i \leq \omega$ (with outward rounding, $\omega_i \approx 2\beta_i$) and let C be the rows kept at level 0.*

1. *Greedy: $C \subseteq \{i : z_i \geq z_{\max} - \omega_i - \omega_a\}$ for any maximizer a , hence $|C| \leq e^{2\omega/T'} / p_{\max}^{(T')}$ for every $T' > 0$, where $p^{(T')} = \text{softmax}(z/T')$.*
2. *Race: if the g_i are i.i.d. standard Gumbel and $p = \text{softmax}(z/T)$, then with $t = 2\omega/T$,*

$$\mathbb{E}|C| \leq \sum_i \frac{p_i e^t}{1 + p_i(e^t - 1)} \leq \min(V, e^t).$$

Proof. (1) $hi_i \leq z_i + \omega_i$ and $lo_a \geq z_a - \omega_a$, so a kept row satisfies $z_i + \omega_i \geq \tau \geq z_a - \omega_a$. Each such row has $p_i^{(T')} \geq p_{\max}^{(T')} e^{-2\omega/T'}$, and the probabilities sum to at most one. (2) In score space the widths are ω_i/T , so $C \subseteq \{i : y_i \geq M - t\}$ with $y_i = z_i/T + g_i$ and $M = \max_j y_j$. Row i belongs to that set if and only if $y_i + t \geq \max_{j \neq i} y_j$. The maximum of independent Gumbels with locations z_j/T is Gumbel with location $\log \sum_{j \neq i} e^{z_j/T}$, and two independent Gumbels with locations a, b satisfy $P(G_a \geq G_b) = e^a / (e^a + e^b)$. Hence the probability is $p_i e^t / (p_i e^t + 1 - p_i)$; sum over i . $\square$

At $T = 1$ and envelope half-width β (so $t \approx 4\beta$), the expected re-scoring set for exact sampling is at most 55 rows for $\beta = 1$, 2,981 for $\beta = 2$ and up to 1.6×10^5 for $\beta = 3$ (Figure 4). Exact sampling through the cheap pass is therefore cheap only if the envelope is about two logits wide or less. Greedy selection has no distribution-free bound: a flat distribution (small $p_{\max}$) needs many rows. The envelope in logits is $\beta_i \approx \|e_i\|_2 \|h\|_2$; with the int8 constants of Table 6 that is about $0.0063 \|h\|_2$, so the norm of the head input decides whether β is a fraction of a logit or several. On captured plain-decode states that norm has a median of 159 (p10 132, p90 169), which puts the int8 row envelope at about one logit (Section 6.4).

6.4 Candidate sets on real hidden states

The certificate was replayed on 6,005 held-out plain-decode head inputs from 160 prompts, captured from the engine (Section 5.6), against the exact logits of the stored BF16 operands; the candidate counts therefore certify the R-real decision. The envelope is Theorem 6.2 with the conservative tensor-core accumulation model ($\gamma = 6.1 \times 10^{-4}$) unless stated. Across every head variant and position no envelope was violated and the exact winner was always a candidate (`evidence/head_geometry/selfevidence_plain4b.json`).

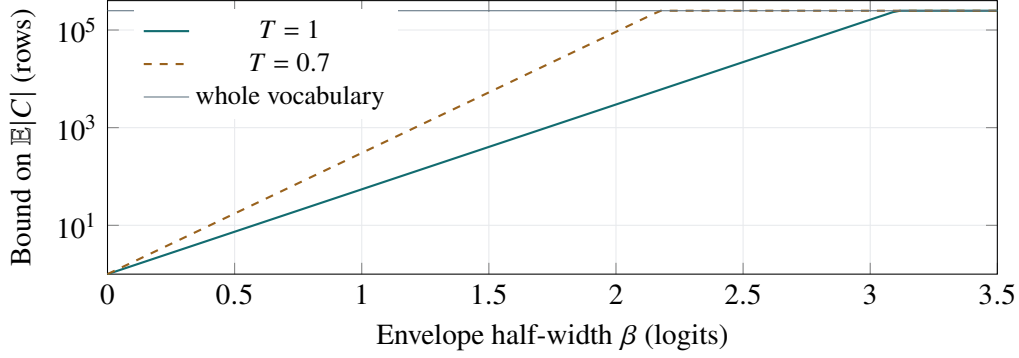


Figure 4: The distribution-free bound $\min(V, e^{4\beta/T})$ on the expected number of rows an exact Gumbel race must re-score (Proposition 6.7, with $\omega \approx 2\beta$). An analytic bound, not a measurement; measured candidate counts on real states are [pending: geometry].

Cheap head	Envelope	Bytes	Mean	p99	Max	1 row	≤ 8 rows
int8, one scale per row	row Cauchy–Schwarz	0.50	1.55	8	30	75.4%	99.3%
int8, 128-column groups	blockwise ℓ_2	0.52	1.34	5	23	81.4%	99.8%
int8, 32-column groups	blockwise ℓ_2	0.54	1.26	5	20	84.0%	99.9%
FP8 E4M3, one scale per row	blockwise ℓ_2	0.51	4.89	60	605	56.2%	88.4%
int4, 32-column groups	blockwise ℓ_2	0.29	20,751	230,238	248,192	0.02%	2.8%
int4, 128-column groups	blockwise ℓ_2	0.27	72,847	248,295	248,320	0%	0%

Table 7: Candidate rows per greedy decision on 6,005 held-out plain-decode positions (248,320 rows), tensor-core accumulation model. “Bytes” is the approximate pass relative to the BF16 head, before candidate rows; “1 row” is the share of positions decided without re-scoring. Measured, evidence/head_geometry/selfevidence_plain4b.csv.

An int8 head certifies almost every decision with one or two rows (Table 7, Figure 5). With one scale per row and the row Cauchy–Schwarz envelope a position needs 1.55 candidate rows on average (median 1, p99 8, maximum 30), and 75.4% are decided by a single candidate; with 128-column groups and the blockwise envelope the mean is 1.34, at 0.52 of the BF16 head’s bytes. FP8 E4M3 keeps only three mantissa bits and needs 4.9 rows on average and 60 at p99. Int4 alone is useless as a certificate: with 128-column groups the median candidate set is 13,991 rows.

The reason int8 suffices is the margin distribution, not the envelope’s tightness. The row envelope has a median half-width of 1.00 logit for int8 per row, about 75 times the median realized error of 0.013, because it assumes the worst alignment of e_i and h ; the head input’s norm has a median of 159, so $\beta \approx 0.0063 \|h\|_2$ is indeed about one logit. But the median top-two margin is 7.3 logits (p10 0.68) and the top token holds a median 99.9% of the mass at $T = 1$, so most decisions sit far outside an int8 envelope (evidence/head_geometry/stats_plain4b.json). Refinements that tighten the envelope a little barely matter: evaluating 16 or 64 columns exactly, a PCA rotation of h , or quantizing W minus its mean row changes the mean candidate count by at most 7% for 1–12% more bytes. Centring is instructive: the mean row points along the bulk of the vocabulary (median cosine 0.39) but not along the rows that compete for the argmax (median cosine -0.11 over each position’s top eight), so it lowers the median error norm over all rows by 14% and raises it by 20% on the rows that matter.

Gumbel races with the same noise field behave like greedy selection: at $T = 1$ and $T = 0.7$ the int8 128-group candidate sets have mean 1.33 and p99 5, well inside the worst-case bound of Figure 4. Over the capture run’s decode batches (up to 16 running requests, 7–10 held-out rows per step on average) the int8 128-group candidates cover 9.5–12.5 distinct rows, so gathered rows cost tens of kilobytes against a 635 MB int8 pass. Re-scoring candidates with IEEE FP32 accumulation on CUDA cores leaves no position undecided; with the tensor-core error model applied to the re-scoring as well, 2.1% of positions still overlap and need FP64. A two-plane cascade

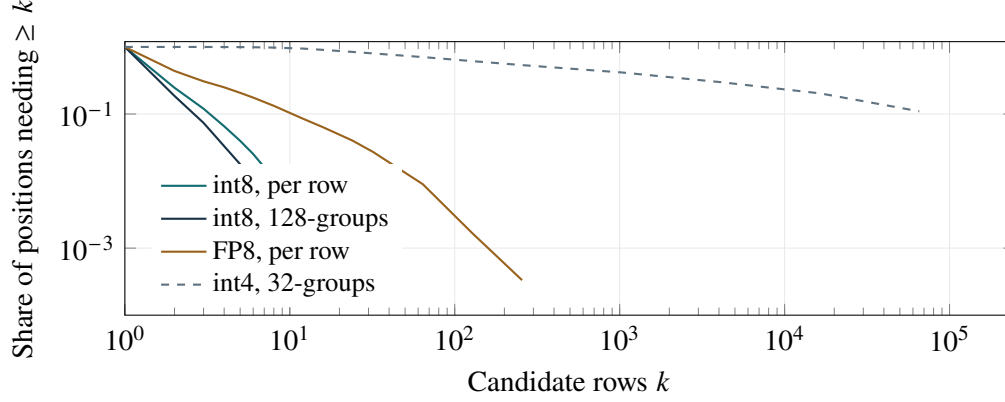


Figure 5: Share of greedy decisions that need at least k candidate rows under each low-precision head, 6,005 held-out plain-decode positions. Measured, `evidence/head_geometry/selfevidence_plain4b_ccdf.csv`.

Stock accumulation model	Gap condition (Thm. 3.5)		Interval form (Prop. 3.6)	
	rows	batches	rows	batches
Conservative tensor-core model, $\gamma = 6.1 \times 10^{-4}$	3.03%	21.6%	1.40%	10.9%
Blocked Hopper model, $\gamma = 1.19 \times 10^{-4}$	2.18%	16.0%	0.35%	2.8%
$\gamma = 10^{-5}$ (sensitivity)	1.97%	14.9%	0.017%	0.13%
IEEE FP32 blocked tree, $\gamma = 1.7 \times 10^{-6}$ (sensitivity)	1.95%	14.8%	0%	0%

Table 8: Share of rows, and of the capture run’s decode batches, that the stock contract leaves to the stock kernel on 6,005 held-out plain-decode positions, by rule and by the error model assumed for the stock kernel’s FP32 accumulation. Batches are the capture run’s, of up to 16 requests. The first two rows model the stock tensor-core GEMM (the blocked row under Khattak and Mikaitis’s published model, an assumption); the last two are sensitivity rows that do not model it. Measured, `evidence/head_geometry/rstock_plain4b.json`.

(an int4 plane for every row, an int4 residual plane only for rows the first cannot exclude) reads 0.32 of the BF16 bytes at batch one and 0.41–0.42 over real batches of 5–16 rows, at the price of an irregular second gather.

How often the stock contract needs the stock kernel. Under R-stock a row is certified only when its token is the stock kernel’s for every accumulator value the error model allows. Table 8 gives, on the same positions, how often each rule leaves a row to the stock kernel, for four accumulation models of that kernel. The gap condition of Theorem 3.5 has a floor near 2% even when the accumulation error is negligible, because it asks for a margin above one BF16 spacing at the winner’s magnitude, which 1.95% of positions lack. The interval form of Proposition 3.6, which rounds each accumulator interval into BF16 buckets and applies the first-index rule, has no such floor: two rows whose logits round to the same BF16 value under every admissible accumulation are decided by index, as the stock argmax decides them. Assuming the stock kernel follows the blocked model of Hopper accumulation ($\gamma = 1.19 \times 10^{-4}$, from Khattak and Mikaitis’s published model [37]), it sends 0.35% of rows and 2.8% of the capture run’s batches (up to 16 requests) to the stock kernel; under the conservative tensor-core model the shares are 1.40% and 10.9%. For every model and rule, no certified token differed from the token the engine returned at the capture’s batch shape, which is consistent with the error models on these 6,005 rows but does not prove that cuBLAS stays within the smallest of them (`evidence/head_geometry/rstock_plain4b.json`).

If the fallback reruns the stock head for the whole batch at the same shape, the batches in Table 8 pay for both heads, and that share grows with the batch: if rows fell back independently with probability p , a batch of B rows would fall back with probability $1 - (1 - p)^B$, about 36% at $B = 128$ for $p = 0.35\%$ and 84% for $p = 1.40\%$. A row-wise fallback avoids that growth but needs the stock GEMM to be bitwise batch-invariant row by row, which

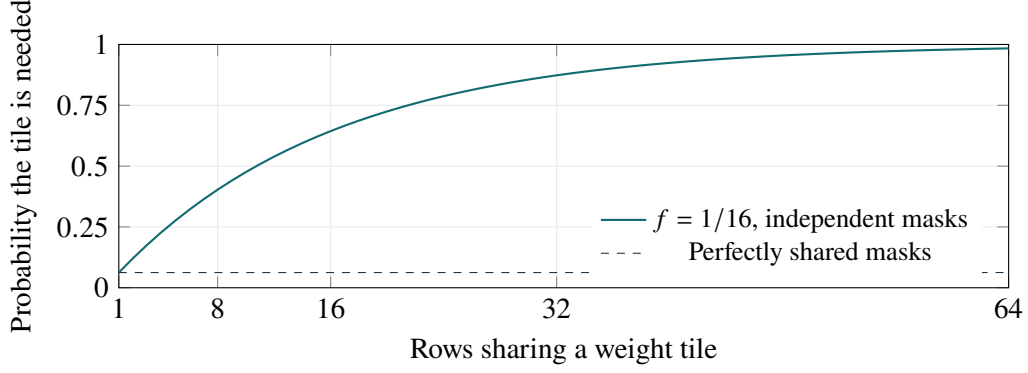


Figure 6: A model of batch-union traffic, not a benchmark. Correlation among masks decides whether per-row sparsity becomes reduced weight traffic.

is [pending: kernel: row-wise batch invariance of the stock head GEMM by batch size]. This is the first measured term of Equation (13), and it leaves an open question. At the capture run’s batch sizes, under the published model, the whole-batch fallback is paid in about 3% of batches; at a serving batch of 128 it would be paid in roughly 36% (blocked model) to 84% (conservative model) of batches, so at serving batch sizes the certified head’s runtime depends on the row-wise mode and on the measured row invariance it requires [pending: kernel: cost of the invariance-checked row-wise fallback at $M = 32$ to 256].

6.5 Sparse rows do not imply sparse weight traffic

For rows i of a microbatch let A_i be the set of retained vocabulary rows or tiles. A kernel that reuses weights across rows reads the union $\bigcup_i A_i$, not the average of the masks. If each row independently retained a tile with probability f , the batch would need that tile with probability

$$1 - (1 - f)^B. \quad (12)$$

At $f = 1/16$ and $B = 32$ this is about 87.3%, so a per-row skip fraction of 93.75% can coexist with nearly a full weight stream. Self-evidence is less exposed to this than transport, because its saving is in the bytes of the cheap pass, which every row shares, and its candidates are few rows rather than whole tiles; the union matters for the re-scoring gather. Figure 6 is an analytic model, not a measurement of mask correlation.

6.6 The certified head as a kernel pipeline

A certified head pays off only if the whole path beats the strongest dense head chain at the served shapes:

$$T_{\text{cheap pass}} + T_{\text{bounds}} + T_{\text{compaction}} + T_{\text{re-score}} + \Pr[\text{fallback}] T_{\text{stock head}} < T_{\text{stock head chain}}. \quad (13)$$

The right-hand side is the stock GEMM plus the FP32 copy and the argmax or top-1 that follow it; a fused dense epilogue that removes the copy and the argmax is a comparator in its own right, as is FlashSampling [80]. Equation (13) is a measurement criterion, not a prediction, and if the head chain is a fraction f of a step, removing it entirely gives at most $1/(1 - f)$.

Figure 7 shows the path for a batch of M head inputs; the implementation is in `src/certified_head/` [pending: kernel: pull request with the implementation and its tests]. Every launch has a grid fixed by M and by buffer capacities, so the whole path, fallback included, is captured once per batch size.

Weights and metadata. Each row is stored as symmetric int8 codes q_i with an FP32 scale $s_i = \text{fl}(\max_j |w_{ij}|/127)$. The error $e_i = w_i - s_i q_i$ is computed offline in FP64, where it is exact, and its group norms,

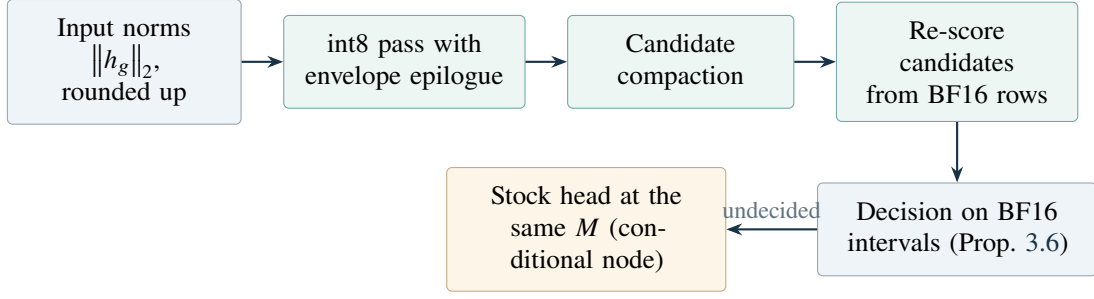


Figure 7: The certified greedy path. Only the approximate pass streams the whole head, in int8 at half the bytes of BF16; its epilogue keeps the four largest upper bounds per tile. Candidate rows are re-read in BF16, and the stock head runs only when the device-side flag is set.

with those of q_i and w_i , are stored as FP64 upper bounds over 128-column groups. From them the head builds one FP32 coefficient per row and group, rounded up,

$$A_{ig} \geq \|e_{ig}\|_2 + \gamma_{\text{acc}} s_i \|q_{ig}\|_2 + \gamma_{\text{ref}} \|w_{ig}\|_2,$$

so that $\sum_g A_{ig} \|h_g\|_2$ bounds the quantization error, the pass’s own accumulation error and the stock kernel’s accumulation error at once (Theorem 6.2 with the reference’s error folded in). With one group per row the metadata is about 1 MB beside 636 MB of codes (derived).

The approximate pass. A Triton kernel tiles the vocabulary and covers all M inputs in one or a few column blocks, so each weight tile is read from HBM once. Codes are converted to BF16 in registers by byte permutation and a BF16 subtraction rather than by conversion instructions, fed to wgmma against the BF16 inputs, and accumulated in FP32; the per-row scale multiplies the accumulator once. The epilogue evaluates the envelope with directed rounding (PTX `fma.rp` and `add.rm`), so the stored bounds enclose the reference’s value without the inflation of Lemma 6.5, and it never writes the $M \times V$ logits. Per input it folds the largest lower bound into one word with an atomic maximum, and per input and vocabulary tile it keeps the four largest upper bounds with their indices and the largest remaining upper bound.

Selection and re-scoring. A compaction kernel applies the threshold to the tile summaries and appends surviving indices to a fixed-capacity list. A tile whose remaining bound reaches the threshold may hold more than four candidates, so all of its rows are appended and the list stays complete; only a list longer than its capacity sends the row to the batch fallback. The re-scoring kernel re-reads the BF16 rows of the candidates, forms each product exactly in FP64 and sums in FP64; its own error is below 10^{-12} relative to $\sum_j |w_{ij} h_j|$, and the interval it stores is widened by the stock kernel’s bound and rounded outward before Proposition 3.6 decides.

Arithmetic by batch size. The int8 pass is bandwidth-bound at small M , but converting codes in registers makes it issue-bound at large M because the conversion repeats for every column block. Two further passes share the epilogue, selection and re-scoring: an integer pass that quantizes each input to int8 and accumulates exactly in int32 (Theorem 6.3), and a BF16 pass over the stored head that keeps only the accumulation terms, a certified dense head that never writes logits and so removes the stock chain’s FP32 copy and separate argmax. The pass is chosen per M from measured times [pending: kernel: per- M times of the three passes; evidence/certified_head/micro_head.json].

Sampling. For the seeded sampler the pipeline carries perturbed scores instead of logits. The stock chain divides FP32 logits by the temperature, takes softmax and log in FP32 and adds the FP64 keyed Gumbel field before the argmax. The normalization is a shift shared by a row, so it cancels from every comparison; what remains is the

rounding of the exponential, the division and the logarithm, which the kernel bounds from CUDA’s documented maximum errors for those functions and adds to the race’s slack (Theorem 7.1), sending undecided rows to the stock sampler [pending: kernel: the written derivation of that slack with its tests, and token-for-token agreement with the stock seeded sampler on replayed steps; evidence/certified_head/].

6.7 Cost of the certified head path

Every timing is a CUDA-graph replay of the complete path (input norms, the int8 pass, compaction, re-scoring, the decision and the conditional fallback) on real decode head inputs, against the stock chain captured the same way. A graph holds 20 back-to-back calls; the median and spread are taken over 50 replays, with the L2 cache warm and with a 256 MB write between calls. Because a fallback costs the stock chain, the expected cost at batch size M is $T_{\text{cert}}(M) + P_{\text{fb}}(M) T_{\text{stock}}(M)$, with $P_{\text{fb}}(M)$ the measured fraction of real batches of size M that contain an undecided row.

[pending: kernel: complete head-path time and variance for $M = 1$ to 256, certified against the stock chain, per error model; stage times and the int8 pass’s effective bandwidth against the measured read peak; primitive costs for the replay’s predictions; evidence/certified_head/micro_head.json, evidence/certified_head/head_primitives.json]

Pending figure. Head-path time against the number of rows M : stock cuBLAS BF16 GEMM plus FP32 copy plus argmax, the certified path without fallback and with the measured fallback rate, and the int8 pass alone as the bandwidth floor, with p10–p90 bands.

Data: evidence/certified_head/head_path_time.csv

Certifying a greedy head from a cheaper copy and re-scoring survivors is not new: sparkpipe screens with an E4M3 shadow on CUDA cores and re-scores in BF16 [90], dgpp uses int8 bit planes with a fixed floating-point slack [28], a Metal implementation re-scores with a replica of the production kernel [47], and knlp reports that on an H100 such a head wins only at batch one under a CUDA graph [8]. What this implementation adds is narrower: a contract stated against a closed-source stock kernel at the engine’s own batch shapes, with that kernel as the fallback rather than a replica; a graph-safe path whose undecided rows cost a conditional stock head; the extension to the engine’s seeded sampler; and a runtime measurement of the complete path against the engine’s actual head chain, which is planned (Section 6.7).

6.8 Beyond the head: a certified tail (rejected)

Section 3.7 observed that in target-only decoding the last layer’s MLP, the final norm and the head have no consumer in future state. The hypothesis, from a later proposal, is that a cheap surrogate of that tail with a sound enclosure could let the commit rule decide the token before the tail runs, removing up to 1.41 GB of the 8.41 GB streamed per step (derived, Table 1). A one-shot surrogate that costs C (surrogate plus certificate) and certifies a fraction p of positions pays only if $C < p T_{\text{tail}}$, and the ceiling for a tail fraction f of the step is $1/(1 - f)$. The directional margin (Proposition 3.2) is the natural certificate when the surrogate predicts the head input with a radius.

The kill test is deliberately optimistic. Record, on about 2,000 positions from 16 prompts of mixed domains, the residual stream at the cut, the reference head input, the logits and the tokens; freeze an int8 tail surrogate; take the oracle radius $\|h - \tilde{h}\|_2$; and apply the directional certificate with no overhead. If the oracle certification rate p_{oracle} satisfies $p_{\text{oracle}} T_{\text{tail}} \leq C_{\text{surrogate}}$, the one-shot design cannot pay and is dropped; otherwise real bounds are compared against the dense tail and against the certified head alone, which is the decisive ablation. The cut does not transfer to speculation without re-analysis, because the MTP layer and the DFlash drafter consume the target’s hidden states. The kill test rejects the design (evidence/head_geometry/tail_killtest.json). On 2,064 positions from 16 held-out prompts, with the head input from a reference BF16 forward pass teacher-forced on the engine’s greedy outputs, an int8 per-row copy of the final MLP moves the head input by a median 0.94% in

relative ℓ_2 norm. With that realized error as an oracle radius and no overhead, the directional certificate decides 85.4% of positions with the exact head and 71.8% when the head is int8 as well, while the int8 head alone, with the exact MLP, decides 79.3% without any re-scoring. The MLP is 0.142 of the tail’s 1.413 GB, so an int8 surrogate can save at most 0.071 GB per token, and it lowers the rate at which the certified head decides on its own. Certifying the head alone is the part worth building.

7 Exact sampling and speculative decisions

Greedy selection is the easy consumer. This section turns the same evidence into sampled decisions: a token drawn from the target law, an accept or reject bit for a drafted token, and the residual or bonus sample that follows. The evidence may come from transport or from self-evidence; every statement below takes enclosures of the logits or of their exponentials as input and does not care which mechanism produced them.

7.1 A certified race under a keyed noise field

Theorem 7.1 (Certified race). *For any fixed noise vector g and $T > 0$, the ladder of Theorem 6.6 with $f_i(x) = x/T + g_i$ returns the S -race token, which is the token of a dense race with the same g and tie rule. If the g_i are i.i.d. standard Gumbel and independent of z , the token has law $\text{softmax}(z/T)$.*

Proof. Theorem 6.6 and the Gumbel-max identity [27, 71]; ties have probability zero under continuous noise. $\square$

Using bounds to avoid evaluating most perturbed scores is the principle of A* Sampling [60] and of lazy Gumbel sampling from a retrieved top set [62]. What the fixed field adds is pathwise equality with the dense race, which a lazily instantiated field does not give. Top- k sampling applies the ladder to membership (drop i when hi_i is below the k -th largest lo) and races over the certified set; equality then also needs the engine’s tie semantics at the k -th position. Min- p admits i when $z_i \geq z_{\max} + T \log(\min-p)$, which intervals decide without a partition function. Top- p admission depends on the mass above each token relative to Z , needs partition bounds, and is usually cheaper to hand to the ordinary path.

Proposition 7.2 (RNG contract). *(i) Seeded equality between a certified and a dense procedure holds when both evaluate the same function of the same noise values with the same tie rule. This requires a counter-based field, a pure function of (seed, request, absolute position, draw purpose, token) that the certified path can evaluate for any token in any order; a stream consumed in evaluation order does not qualify, and in the exact reference assigning a stream to rows in candidate order changed the winner in 247 of 300 cases (evidence/precision/precision_tests.json). (ii) Equality in distribution needs i.i.d. noise with the correct marginal, independent of the proposal draw, the acceptance uniform and the scores; a residual race also needs independence from the acceptance event. (iii) Fixed-noise verification (Section 7.3) needs the noise of a position to be the same whether the position is drafted or decoded plainly, so it must be keyed by absolute position, not by cycle or batch slot. (iv) A hash with 2^{32} levels makes the law exact only for the idealized continuous field; the stock seeded sampler shares that approximation.*

The stock seeded sampler (Section 3.2) already has the form (i) requires. FlashSampling’s noise offset depends on the tiling [80], so it cannot serve as a certified path’s field without change.

7.2 How much of a sample can be emitted before the scores are known

A race needs the winner’s interval to separate. A different construction can emit a token with some probability before any candidate is separated, and then finish the rest of the law after more computation. Let $\mathcal{P}(I)$ be the set of target laws compatible with the information I held so far.

Proposition 7.3 (Common mass). *Consider procedures that, before acquiring more information, emit token i with probability r_i (the same for every $p \in \mathcal{P}(I)$) and otherwise continue until p is known. Such a procedure outputs*

exactly p for every $p \in \mathcal{P}(I)$ if and only if $r_i \leq \inf_{p \in \mathcal{P}(I)} p_i$ for every i , and then it may finish by sampling the residual $(p - r)/(1 - \alpha)$ with $\alpha = \sum_i r_i$. The largest immediate probability is

$$\alpha^*(I) = \sum_i \inf_{p \in \mathcal{P}(I)} p_i.$$

Proof. If the procedure is exact, the final probability of i is at least the probability r_i of emitting it immediately, for every compatible p ; so $r_i \leq \inf_p p_i$. Conversely, if $r_i \leq \inf_p p_i$ then $p - r \geq 0$ and sums to $1 - \alpha$ for every compatible p , so the residual is a law, and the output law is $r + (1 - \alpha) \cdot (p - r)/(1 - \alpha) = p$. $\square$

For two candidate laws $\mathcal{P} = \{p, q\}$, $\alpha^* = \sum_i \min(p_i, q_i) = 1 - \text{TV}(p, q)$, the overlap of a maximal coupling, which is exactly the acceptance probability of speculative sampling [9, 48]. The proposition says the same idea applies to any set of laws the evidence leaves open, and that nothing can do better without more information.

A two-token example shows that exact sampling can finish where greedy selection cannot. Suppose the evidence only says that the probability of token 1 lies in $[0.49, 0.51]$. Then $\alpha^* = 0.49 + 0.49 = 0.98$. Draw U uniform on $[0, 1]$: emit token 1 if $U < 0.49$, token 0 if $U \geq 0.51$, and otherwise compute p and emit token 1 exactly when $U < p$, reusing the same U . The output has law p , and 98% of draws never need the scores; the greedy decision, by contrast, is undecided because the interval contains $1/2$. For two tokens this procedure is inverse-CDF sampling with bounds on the CDF. In general it gives equality in distribution, not seeded equality with a dense race.

Proposition 7.4 (Bounded logit error). *Let $p = \text{softmax}(z/T)$ and $q = \text{softmax}(\tilde{z}/T)$, and suppose $\max_i (z_i - \tilde{z}_i) - \min_i (z_i - \tilde{z}_i) \leq \Delta$. Then $p_i \geq e^{-\Delta/T} q_i$ for every i . Consequently, with $\alpha = e^{-\Delta/T}$:*

1. *sampling q with probability α , and otherwise computing p and sampling $(p - \alpha q)/(1 - \alpha)$, samples p exactly;*
2. *always sampling q gives $\text{TV}(p, q) \leq 1 - \alpha$.*

Proof. Write $d_i = z_i - \tilde{z}_i$ with $m \leq d_i \leq M$ and $M - m \leq \Delta$. Then $p_i/q_i = e^{d_i/T} / \sum_j q_j e^{d_j/T} \geq e^{m/T} / e^{M/T} \geq e^{-\Delta/T}$. Part 1 is Proposition 7.3 with $r = \alpha q$. For part 2, $\text{TV}(p, q) = \sum_i (q_i - p_i)_+ \leq \sum_i (1 - \alpha) q_i = 1 - \alpha$. $\square$

With self-evidence the range is at most twice the largest envelope half-width, so $\alpha \geq e^{-2 \max_i \beta_i/T}$. The trade-off against the race is different: this sampler needs the cheap pass's full softmax and, with probability $1 - \alpha$, the whole exact head, whereas the race re-scores a candidate set whose expected size grows like $e^{4\beta/T}$ and returns the seeded token.

Proposition 7.5 (Approximate mode: KL and sequence total variation). *Suppose $\min_c \|z - \tilde{z} - c\mathbf{1}\|_\infty \leq \varepsilon$ at temperature one (at temperature T replace ε by ε/T). Then $\text{KL}(p \| q) \leq \varepsilon^2/2$ and $\text{KL}(q \| p) \leq \varepsilon^2/2$, so $\text{TV}(p, q) \leq \varepsilon/2$. If an approximate decoder samples q_t in place of p_t at every position and the bound holds with ε_t at every reachable prefix, the laws P and Q of the first N tokens satisfy $\text{KL}(P \| Q) \leq \frac{1}{2} \sum_t \varepsilon_t^2$ and*

$$\text{TV}(P, Q) \leq \frac{1}{2} \left(\sum_{t=1}^N \varepsilon_t^2 \right)^{1/2}.$$

Proof. Let $d = z - \tilde{z} - c\mathbf{1}$ with $\|d\|_\infty \leq \varepsilon$; the shift c changes neither law. Then p is q tilted by d : $p_i = q_i e^{d_i} / \mathbb{E}_q[e^d]$. Let $\psi(\lambda) = \log \mathbb{E}_q[e^{\lambda d}]$, so $\psi(0) = 0$ and $\text{KL}(p \| q) = \mathbb{E}_p[d] - \psi(1) = \psi'(1) - \psi(1) = \int_0^1 \lambda \psi''(\lambda) d\lambda$. Here $\psi''(\lambda)$ is the variance of d under the tilted law $q^{(\lambda)}$, at most $(2\varepsilon)^2/4 = \varepsilon^2$ because d takes values in an interval of length 2ε . Hence $\text{KL}(p \| q) \leq \varepsilon^2/2$; the reverse divergence follows by exchanging the roles of p and q . Pinsker's inequality gives $\text{TV} \leq \sqrt{\text{KL}/2} \leq \varepsilon/2$. For sequences, the chain rule gives $\text{KL}(P \| Q) = \sum_t \mathbb{E}_P[\text{KL}(p_t \| q_t)] \leq \sum_t \varepsilon_t^2/2$, and Pinsker's inequality again gives the total-variation bound. $\square$

To guarantee a sequence total variation of at most 0.01 over $N = 1,024$ tokens with a uniform bound, the proposition needs a logit error, up to a common shift, of at most $\varepsilon = 6.25 \times 10^{-4}$, far tighter than any int8 envelope in Section 6. An approximate head is therefore a quality trade-off to be measured (Section 10); these bounds cannot certify it as harmless over long outputs.

7.3 Fixed-noise verification

Theorem 7.6 (Fixed-noise verification). *Let $G_{t,i}$ be keyed by (seed, absolute position t , token i) and i.i.d. standard Gumbel across (t, i) . Define $Y_t = \min \arg \max_i (z_{t,i}(Y_{<t})/T + G_{t,i})$. A speculative decoder that accepts a drafted token at position t if and only if it equals $\arg \max_i (z_{t,i}(x_{<t})/T + G_{t,i})$ at the drafted prefix, and emits that argmax at the first mismatch, outputs Y for every drafter and every depth policy, and Y has the target’s autoregressive law.*

Proof. By induction on positions, every accepted token equals the argmax at a prefix equal to $Y_{<t}$, so it is Y_t ; at the first mismatch the prefix is $Y_{<t}$ and the emitted token is Y_t . The drafter only changes how many positions one target pass verifies. Given $Y_{<t}$, $G_{t,\cdot}$ is independent of $z_t(Y_{<t})$, so $Y_t \mid Y_{<t} \sim \text{softmax}(z_t(Y_{<t})/T)$. $\square$

This is the drafter-invariant construction of Daliri, Musco and Suresh [16]. For the certified head it matters because verification becomes a perturbed argmax, which Theorem 7.1 certifies with no partition function and no residual sampler. The theorem treats z_t as a function of the prefix alone. In the engine the verify pass computes the logits at a different batch shape from plain decoding, and Section 3.6 shows that stock configurations at different shapes can disagree, so what a fixed-noise verifier reproduces token for token is the seeded sampler applied to the verify pass’s own logits; equality with seeded plain decoding needs the verify-shape and plain-decode logits to agree as well, for example under batch-invariant kernels.

Proposition 7.7 (Acceptance under fixed-noise coupling). *If the drafter proposes $\arg \max_i (\log q_i + G_i)$ and the target accepts if and only if it equals $\arg \max_i (\log p_i + G_i)$, then*

$$P(\text{accept}) = \sum_i \left(\sum_j \max\left(\frac{p_j}{p_i}, \frac{q_j}{q_i}\right) \right)^{-1}, \quad \frac{1}{2} \sum_i \min(p_i, q_i) \leq \sum_i \frac{p_i q_i}{p_i + q_i} \leq P(\text{accept}) \leq \sum_i \min(p_i, q_i).$$

Proof. Both choose i if and only if $G_i > \max_{j \neq i} (G_j + m_j)$ with $e^{m_j} = \max(p_j/p_i, q_j/q_i)$, an event of probability $(1 + \sum_{j \neq i} e^{m_j})^{-1}$. The bounds follow from $\max(x, y) \leq x + y$, from $\max(x, y) \geq x$ and $\geq y$ summed over j , and from $p + q \leq 2 \max(p, q)$. $\square$

Per position, fixed-noise acceptance is at most the $\min(1, p/q)$ rate $1 - \text{TV}(p, q)$ and at least half of it; the two agree when $p = q$. Whether the cheaper certificate pays for any lost acceptance is an empirical question [pending: acceptance under fixed-noise coupling against standard verification on real MTP and DFlash drafts at $T = 1$ and $T = 0.7$].

The sampling contract adopted here. The certified speculative path uses fixed-noise verification with exactly the stock seeded sampler’s noise: $G_{t,i} = -\log(-\log x_{t,i})$ in FP64, with $x_{t,i}$ the 32-bit MurmurHash of (seed, absolute position t , token i) scaled to $[0, 1]$, as in `multinomial_with_seed` (Section 3.2). A drafted token is accepted if and only if it equals the target’s keyed sample at its position, and at the first mismatch that sample is emitted. The output is equal in law to target sampling, and it is drafter-invariant: the same keyed noise and the same verifier logits give the same tokens for any drafter and any depth policy. Three further consequences follow. For SGLang’s default point-mass drafts the per-position acceptance probability is unchanged: the stock rule accepts a draft d when a uniform coin falls below $p(d)$, and the keyed rule accepts it when d wins the race, which also has probability $p(d)$. With a seed, every committed token is the stock seeded sampler’s token for the verify pass’s own stock logits at that position (Theorem 7.6 under R-stock at the verify shape), provided that every row whose race the certificate cannot decide under the stock rounding of log softmax runs the stock sampler. The output equals seeded decoding without speculation only where the verify-pass and plain-decode

logits agree, which the engine does not guarantee because the two use different kernels and shapes; the divergence analysis of Section 3.6 is the measurement that will say how often they differ; stock speculative sampling, by contrast, is equal to non-speculative decoding only in law, because its coins and its inverse-CDF residual consume randomness differently. Without a seed the output equals stock in law. Top- p requests fall back to the stock path. The acceptance guards below are analysis and are not built: under the stock-token contract the stock inverse-CDF residual would still need a dense head row per verify step. The design is decided; its measurement is [pending: kernel: token-for-token agreement with the stock seeded sampler on the verify pass's logits for replayed steps, and acceptance against the stock speculative sampler].

7.4 Acceptance guards from incomplete mass

For one reached proposal $X = x$ write $w_x = e^{z_x}$ and $Z = \sum_i e^{z_i}$, and draw $U \in [0, 1)$ with the reference convention. For $q_x > 0$, acceptance is equivalent to

$$Uq_x Z < w_x, \quad (14)$$

and the $\min(1, \cdot)$ need not be evaluated because $U < 1$ already handles $p_x \geq q_x$.

Theorem 7.8 (Acceptance guards with enclosures). *Let $w_x \in [N^-, N^+]$ and $Z \in [Z^-, Z^+]$ with $N^- > 0$, let $0 < q_x \leq 1$ be the actual proposal mass and $U \in [0, 1)$. Then $Uq_x Z^+ < N^-$ implies acceptance, $Uq_x Z^- \geq N^+$ implies rejection, the two never both hold, and replacing an enclosure by a tighter one cannot reverse a certified decision. For U uniform and independent of the enclosures, neither guard fires with probability*

$$P_\gamma = \min(1, N^+/(q_x Z^-)) - \min(1, N^-/(q_x Z^+)).$$

If $N^- = N^+ = w_x$ and every row outside a re-scored set R has a mass enclosure whose upper and lower ends differ by a factor at most κ , then with $\pi = \sum_{i \notin R} p_i$,

$$P_\gamma \leq \frac{p_x}{q_x} \cdot \frac{(\kappa - 1)\pi}{1 - \pi(1 - 1/\kappa)}.$$

Proof. The implications are transitivity with (14); both firing would give $N^+ \leq Uq_x Z^- \leq Uq_x Z^+ < N^-$. A tighter enclosure stays on the same side of the true values. The unresolved uniforms form $[N^-/(q_x Z^+), N^+/(q_x Z^-)) \cap [0, 1)$. For the bound, $P_\gamma \leq (w_x/q_x)(Z^+ - Z^-)/(Z^+ Z^-)$; $Z^+ - Z^- \leq \sum_{i \notin R} e^{z_i}(\kappa - 1)$ because each lower mass is at most e^{z_i} ; and $Z^- \geq Z - \sum_{i \notin R} e^{z_i}(1 - 1/\kappa)$ because each lower mass is at least e^{z_i}/κ . Divide by Z . $\square$

These guards are analysis in this revision: the engine path adopted above uses fixed-noise verification, which needs no partition function, and a certified version of the stock rule would still pay for a dense residual row. Transported tile masses (Theorem 5.1) and exponentiated self-evidence intervals (Theorem 6.2) both supply $Z^\pm$. With self-evidence $\kappa = e^{2\hat{\beta}}$ times the outward rounding of the exponential; at $\beta = 1$ ($\kappa \approx 7.4$), keeping P_γ below 10% at $p_x/q_x \approx 1$ requires the unre-scored mass π to be below about 1.5%. After an adaptive refinement chosen using U , the probability statement no longer applies, although the guards remain sound pointwise. The Lean formalization proves the guards with enclosed numerators and their exclusivity (`formal/CertifiedArgmax.lean`).

7.5 Residual and bonus work

A rejected proposal still requires a sample from $(p - q)_+$, and knowing that rejection occurred does not supply one. When all drafted tokens are accepted, a bonus token needs a target sample. A certificate that decides acceptance cheaply and then completes the whole head for the residual saves little; Section 7.6 shows this happening. Two facts about the engine and one construction reduce the problem.

First, SGLang's default sampled verification uses point-mass proposals (Section 3.2). With q a point mass at x , the residual is the target restricted to the untried tokens, a race over $V \setminus \{x\}$ that needs no partition function. Second, under fixed-noise verification there is no residual at all. Third, for a proposal with small support there is an exact bounded race.

Let $Q = \{i : q_i > 0\}$. With unnormalized target weights w_i and partition Z , the residual can be sampled from weights

$$r_i = (w_i - Zq_i)_+, \quad \frac{r_i}{\sum_j r_j} = \frac{(p_i - q_i)_+}{\sum_j (p_j - q_j)_+}, \quad (15)$$

and $r_i = w_i$ outside Q . Given $Z^- \leq Z \leq Z^+$, positive-part monotonicity gives $(w_i - Z^+q_i)_+ \leq r_i \leq (w_i - Z^-q_i)_+$ inside Q .

Theorem 7.9 (Bounded residual race). *Assume valid enclosures of the target weights and of Z , the actual proposal law, a nonzero residual, and a priority field $\pi_i = e^{G_i} > 0$ that is fixed and independent of the proposal draw, the acceptance uniform and the rejection event. A refinement procedure that returns only an interval-certified ordered maximizer of $r_i\pi_i$, or completes the dense race, returns the same token as the dense race. With i.i.d. standard Gumbel G_i it samples the residual law (15) exactly.*

Proof. The enclosures of r_i multiplied by $\pi_i > 0$ enclose the priorities. A certified maximizer dominates every possible competitor, including the actual one, and a completed evaluation yields the dense ordered maximizer. The distributional statement is Gumbel-max applied to the nonzero residual weights. $\square$

Theorem 7.10 (Residual races with self-evidence). *Replace the enclosures above by $r_i \in [(N_i^- - Z^+q_i)_+, (N_i^+ - Z^-q_i)_+]$ for $q_i > 0$ and $r_i \in [N_i^-, N_i^+]$ for $q_i = 0$. The conclusions of Theorem 7.9 hold. For a point-mass proposal at x , $r_x = 0$ as soon as $Z^- \geq N_x^+$, and the residual is a race over $V \setminus \{x\}$ that needs no partition bound. SGLang’s target-only rule (accept sibling j if $UZ < \sum_{l \leq j} w_{x_l}$; otherwise sample the target restricted to untried tokens) is certified by the guards with a cumulative numerator and an exclusion race.*

Proof. Positive-part monotonicity gives the enclosures; the rest is the proof of Theorem 7.9. For the point mass, $w_x - Zq_x = w_x - Z \leq 0$. $\square$

The engine samples that residual by inverse CDF, so a race matches it only in distribution. For a support chosen after the draft pass, retaining the top $K + 1$ perturbed entries per tile tightens the outside-support maximum after excluding up to K support tokens; it is not needed for validity, since $r_i \leq w_i$ makes the plain tile maximum an upper bound. For a bonus sample the construction is simpler still: a certified race over the target scores with the bonus position’s field, with no partition function.

7.6 A cheap decision is not a cheap verification

The inherited CPU experiment makes the last point concrete. It constructs two families of 128×8 integer heads with 16-row tiles and 36 paired fixtures each: one clusters rows around separated centres, the control uses unstructured rows. For each fixture $h^t = h^d + d \cdot v$ with drift scale $d \in \{0, 1, 2, 4\}$; proposals are sampled from the anchor head’s law, and all probability arithmetic is exact rational. The experiment counts distinct target rows projected, which is work, not time.

At zero drift the summaries describe the target exactly and the acceptance decision needs only its candidate row, an identity check rather than a forecast. At nonzero drift the unstructured head defeats greedy screening. More subtly, large drift makes rejection easy to certify, which lowers the rows needed for the decision while destroying acceptance itself, and completing the residual then erases the saving (Figure 8). At drift four the clustered family decides with 11.4% of the rows but needs 96.5% once the dense residual is charged. The experiment does not refute transport on trained heads; it refutes the inference that a cheap acceptance decision means cheap verification. The residual races above are the answer it motivates, and they remain to be measured.

7.7 A demand frontier over a fixed protocol

For a chain of L proposals each head row is accepted, rejected or unresolved. The first certified rejection at position r proves that no row after r can affect the cycle’s output, while earlier unresolved rows still matter because one of them may be the true first rejection.

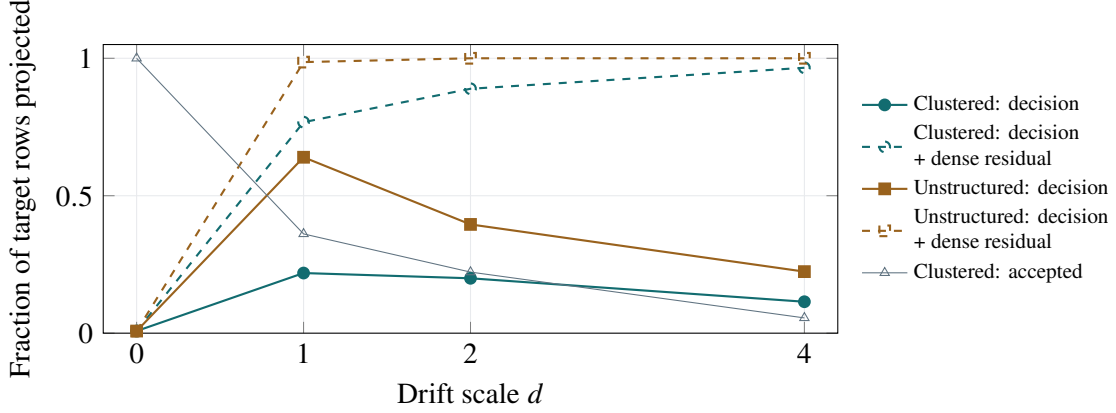


Figure 8: Work counts on constructed heads, not timings (data/synthetic_drift.csv, evidence/synthetic_drift.json). Solid lines count the rows needed to decide acceptance; dashed lines add a dense residual completion after each rejection. The thin line is the fraction of proposals accepted in the clustered family. Evidence production, bounds, bonus sampling and the backbone are excluded.

Proposition 7.11 (Safe suppression after a certified rejection). *If every issued certificate equals the decision of a fixed reference verifier on its row, the first true rejection is no later than the first certified rejection. Suppressing head work strictly after the latter does not change the reference output, provided completion and state commit use the actual first rejection.*

Proof. At the certified rejection the reference decision is rejection; an earlier unresolved row may also reject, and no later row can be the first rejection. All accepted outputs, the replacement token and the state prefix are determined by rows up to the first rejection. $\square$

The statement is for chains. SGLang’s trees share one coin across siblings and choose the next coin by the accepted node, so a tree frontier must follow the tree’s paths. The target backbone has already run over every admitted position, so the frontier saves head work, not transformer layers.

7.8 When adaptive verification changes the sampling law

Avoiding work whose result is already determined differs from choosing which sampled proposals count. DSpark states that lossless speculation requires admission decisions that do not depend on future candidate tokens, and D-cut gives a counterexample and a repair [14, 57]. A two-token illustration: let $p = q = (1/2, 1/2)$, draw $X \sim q$, discard it and draw afresh from p if $X = 0$, and admit it to the ordinary verifier if $X = 1$. Since $p = q$ an admitted proposal is always accepted, and the output law is

$$\Pr(Y = 0) = \frac{1}{4}, \quad \Pr(Y = 1) = \frac{3}{4}. \quad (16)$$

The acceptance formula was unchanged; admission selected proposals by their realized value. A mandatory progress floor on the first token does not help, since the same construction applies at the second position. Three settings avoid the issue: greedy decoding; target-only verification with point-mass proposals, where there is no proposal law to bias; and fixed-noise verification (Theorem 7.6), under which every depth policy yields the same output. The demand frontier above changes only the amount and order of arithmetic used to decide a fixed protocol, so it is exact pointwise.

7.9 Measurements

[pending: geometry, kernel: Gumbel-race candidate counts through the int8 pass on real states at $T = 1$ and $T = 0.7$; mass-enclosure ratios and P_7 for standard acceptance on real MTP and DFlash drafts; the fraction of decisions resolved before any re-scoring]

8 Drafting

How much a verify pass is worth depends on the drafter in front of it: a cycle commits as many tokens as the target accepts plus one, and its cost includes the draft. The native MTP layer of Qwen3.5-4B drafts sequentially, paying one small layer plus the full 1.27 GB head per drafted token, and its acceptance decays with depth. A block drafter proposes a whole block in one forward pass. This section describes the public block drafter for this target, how SGLang runs it on a hybrid model, and what is measured about it.

8.1 The public block drafter

DFlash has a public drafter for exactly this target, `z-lab/Qwen3.5-4B-DFlash` at revision `9a1996cc`, under Apache-2.0 [10, 119]; `modal-labs/Qwen3.5-4B-DFlash` carries byte-identical weights. Read from the checkpoint rather than the card, it is a six-layer Qwen3-style transformer of width 2,560 (32 query and 8 key-value heads of dimension 128, MLP width 9,216) with no embedding or head of its own: it embeds with the target’s table and projects through the target’s tied head. It conditions on the target’s hidden states after layers 1, 5, 9, 13, 17, 21, 25 and 29, concatenated per token and projected by one linear map and an RMS norm to a single context vector. Five layers use sliding-window attention (window 4,096) and the last uses full attention. It has 634 million parameters, 1.27 GB in BF16, almost exactly the size of the target’s head (derived).

A cycle in SGLang’s DFlash worker [86] works as follows (code reading at the pin). The block is the last committed token followed by fifteen copies of a mask token (id 248,077). Each draft layer’s keys and values for the committed context come from the projected target features, which the worker writes into the drafter’s own cache as tokens are committed. One forward pass over the sixteen positions yields sixteen hidden states; positions 1–15 go through the target head, and their argmaxes are the drafted tokens. The target verifies the block in one pass, the worker accepts the longest drafted prefix that matches the target’s argmax, commits it plus the target’s token at the first mismatch, and appends the committed tokens’ features to the drafter’s cache. The sliding layers are causal inside the block and the full layer is not; SGLang infers this from the layer types because the configuration sets `no_is_causal`, which matches the attention pattern of the DFlash training recipe in SGLang’s SpecForge repository (`sgl-project/SpecForge`).

On the hybrid target the verify pass has a cost the drafter’s parameters do not show. SGLang verifies a chain on the GDN layers by keeping the recurrent state after every block position, so that the state after the accepted prefix can be committed without recomputation. Each state is 50.3 MB per request, so a 16-token block writes about 805 MB per request per cycle, and each running request reserves space for sixteen of them (derived). That bounds block-16 capacity on this machine, and it is a property of the verifier, not of the drafter.

The card’s launch line targets a B200 with TRT-LLM target attention, FA4 draft attention and piecewise-compiled prefill graphs. None carries over unchanged: TRT-LLM attention is Blackwell-only, FA3 and FA4 are unavailable on this aarch64 host, and the piecewise prefill graph fails to capture for this target at the pinned commit. The drafter is served with FlashInfer for target and draft attention, the FlashInfer GDN kernels, the default prefill graph and the card’s overlap planning stream, with decode, verify and draft CUDA graphs and the overlap scheduler on [\[pending: drafter: launch record and graph-capture state for every run; evidence/drafter/launch/\]](#).

8.2 What a cycle costs

Counting weight bytes gives the shape of the trade before any timing. At batch one a DFlash cycle reads the drafter once (1.27 GB), the target head once for the fifteen drafted positions (1.27 GB), and the target once to verify (8.41 GB): drafting costs about 30% of the verify pass’s weight traffic and proposes fifteen tokens. An MTP chain of three steps reads the MTP layer and the full head three times (about 4.5 GB, 54% of the verify pass) and proposes three (derived). Activation traffic, the per-position GDN states, attention over the context and host time are not in these counts [\[pending: drafter: split of the DFlash cycle at batch 1, 4, 16 and 64 into draft forward, draft head, verification, GDN commit, cache update and host time; evidence/drafter/cycle_split.csv\]](#).

Block size	All	Chat	Code	GSM8K	MATH-500
16	6.98	4.26	7.87	7.16	7.81
8	5.34	3.80	5.76	5.45	5.84

Table 9: Committed tokens per verify cycle τ for the public DFlash drafter on GH200, averaged over requests as the model card does (greedy, thinking on, concurrency 1; 16 to 32 requests per domain). Pooled over tokens instead, block 16 gives 6.18 overall. Measured, `evidence/drafter/acceptance_summary.csv`.

8.3 Acceptance by block position

Let h_j be the number of verify cycles in which exactly j drafted tokens were accepted, $N = \sum_j h_j$, and $S(k) = \sum_{j \geq k} h_j / N$ the fraction of cycles that accept at least k . The conditional acceptance of block position k is $\alpha_k = S(k)/S(k-1)$, and a cycle commits $\tau = 1 + \sum_{k \geq 1} S(k)$ tokens on average. SGLang reports h per request, so α_k and τ are measured rather than modelled. The card reports mean accepted lengths of 5.9–7.7 at block 16 on a B200 for its five benchmarks, against 3.3–3.5 for MTP with three steps [119]; these are the authors’ numbers on other hardware.

The measurement uses a panel of 32 MATH-500 problems and the first 16 chat, 16 code and 16 GSM8K prompts of the benchmark’s earlier confirmation split, decoded greedily with thinking on, stopping naturally at up to 2,048 new tokens, at concurrency 1, with blocks of 16 and 8 (`evidence/drafter/`). The runs used an engine trace hook that synchronizes every cycle on a shared GPU, so acceptance and token counts are valid and timings are not; the panel contains benchmark test problems and measures acceptance only, never quality.

A block of 16 commits 7.0 tokens per verify cycle on average and a block of 8 commits 5.3 (Table 9); the MATH-500 value, 7.8, sits near the 7.48 the card reports for that benchmark [119], but that is a cross-hardware reference, not a reproduction: the card used a B200 and a 4,096-token cap and does not state how many problems it used, where this panel uses a GH200, a 2,048-token cap and 32 sampled problems. Acceptance depends strongly on the domain: chat commits about four tokens per cycle and code or mathematics about seven to eight. The per-position rates of Figure 9 show why a longer block still commits more tokens per cycle; whether it pays depends on the cost of a cycle, which is pending. Over all domains the conditional acceptance dips at the second position (0.80) and then rises toward 0.9, so the share of cycles that reach position 15 is still 13%; the rise is a selection effect, since cycles that get that far are in predictable stretches of text, not a sign that the drafter is better deep in the block. Chat is lower throughout (0.69 at positions 2 and 3). Two further observations bound what training could change. Acceptance, pooled over tokens, is lower in the answer after the thinking text than in the thinking itself for chat (3.3 against 4.2 tokens per cycle) and higher for code and mathematics, so no single segment dominates. And the first rejected draft in a cycle equals the target’s token at the previous position in about 6% of cycles, a repetition-type failure, while in 94% it is some other token (`evidence/drafter/trace_segments_b16.csv`, `evidence/drafter/trace_segments_b16_failures.json`).

[pending: drafter and bench: τ against tuned MTP on the same prompts, and the split of a cycle into draft, verification and host time from an untraced run]

8.4 Output equality and limits

Greedy speculative verification is exact in real arithmetic, but the verify pass computes the target’s logits at a different batch shape from plain decoding, so the stock noise floor of Section 3.6 applies [pending: drafter: divergences of DFlash (blocks 8 and 16) against plain decoding per 1,000 tokens, classified by mechanism, against the plain-decoding floor; `evidence/drafter/equality.json`]. Four limitations are candidates: acceptance that decays with block position or by domain; drafting cost dominated by the full-vocabulary head for fifteen positions; the verifier’s per-position state writes, which bound block size at high concurrency; and draft states far from the target’s in the head’s metric, which decides whether transport can use DFlash drafts (Section 5.7). The public drafter’s training data are not published, so none of these limitations can be traced to its data. A custom

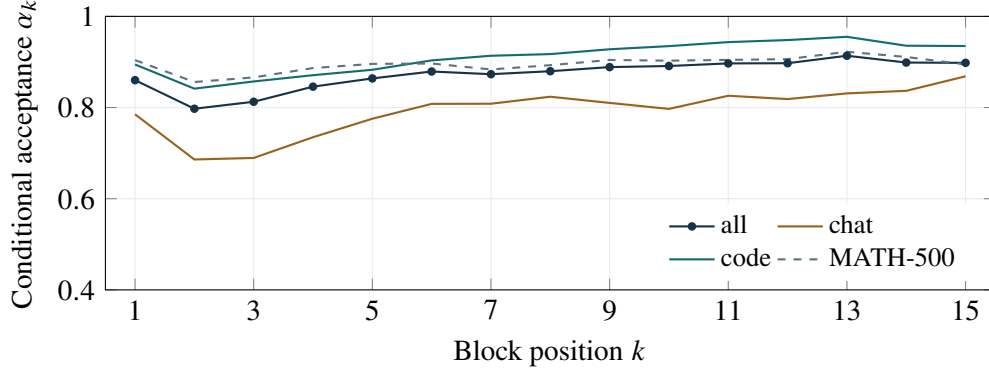


Figure 9: Conditional acceptance $\alpha_k = S(k)/S(k-1)$ of the public DFlash drafter by block position, block size 16, greedy with thinking on. Measured, evidence/drafter/acceptance_by_position.csv.

drafter is worth training only against one of them, judged by total serving time: fine-tuning on the target’s own outputs for acceptance, adding DFlash 2’s selector [33] for acceptance, or an auxiliary loss on $\|h^t - h^d\|_2^2 / \|h^t\|_2^2$ if transport’s required drift reduction proves reachable [pending: drafter: which limitation the measurements support, and the decision on training].

8.5 Training a selector for rate (P6, hypothesis)

A later proposal keeps the DFlash backbone and the verifier frozen and trains only a small predecessor-conditioned selector over the backbone’s $K = 16$ candidates per position, $q_i(b | a) = \text{softmax}_{b \in C_i} [\ell_i(b) + s_{\theta,i}(a, b)]$, so that one coherent path is proposed; DFlash 2’s selector architecture [33], whose training objective the announcement does not state, is the baseline, not a novelty. The objective is emitted tokens per unit time, $\mathbb{E}[\sum R]/\mathbb{E}[\sum C]$ with $R = L + 1$ per cycle, trained as $R - \lambda C$ with λ set to the measured rate. If λ^* is the best achievable ratio in Equation (2), then $\mathbb{E}R - \lambda^* \mathbb{E}C \leq 0$ for every policy, with equality exactly for the policies that attain the ratio, because $\mathbb{E}C > 0$; so maximizing the difference at the right λ maximizes the rate. Two identities make the accepted-length term trainable exactly.

Proposition 8.1 (Prefix value of a sampled draft policy). *Let the draft path Y be sampled position by position, $Y_i \sim q_i(\cdot | Y_{<i})$, and let L be the number of accepted drafted tokens. Against a greedy target with continuation g , $\mathbb{E}L = \sum_k \prod_{i \leq k} q_i(g_i | g_{<i})$, and $\nabla_{\theta} \mathbb{E}L = \sum_j (\sum_{k \geq j} S_k) \nabla_{\theta} \log q_j(g_j | g_{<j})$ with $S_k = \prod_{i \leq k} q_i(g_i | g_{<i})$. Against a sampled target with the standard rule, with $A_k = \prod_{i \leq k} \min(1, p_i(Y_i)/q_i(Y_i))$,*

$$\mathbb{E}L = \mathbb{E}_Y \left[\sum_k A_k \right], \quad \nabla_{\theta} \mathbb{E}L = \mathbb{E}_Y \left[\sum_j \mathbf{1}\{q_j(Y_j) < p_j(Y_j)\} \left(\sum_{k \geq j} A_k \right) \nabla_{\theta} \log q_j(Y_j) \right].$$

Proof. Greedy: $L \geq k$ exactly when $Y_i = g_i$ for $i \leq k$, which has probability S_k , and $\mathbb{E}L = \sum_k \Pr(L \geq k)$; differentiate the product. Sampled: given the path, position k is reached and accepted with probability A_k , so $\mathbb{E}L = \mathbb{E}_Y \sum_k A_k$. Its gradient has a score part and a pathwise part. In the score part, $\mathbb{E}[A_k \nabla \log q_j(Y_j)]$ vanishes for $k < j$, because A_k depends only on $Y_{<j}$ and the score has conditional mean zero, leaving $\sum_j \nabla \log q_j \sum_{k \geq j} A_k$. In the pathwise part, $\nabla \min(1, p_j/q_j)$ is zero where $q_j < p_j$ and $-\min(1, p_j/q_j) \nabla \log q_j$ where $q_j \geq p_j$, which contributes $-\mathbf{1}\{q_j \geq p_j\} \sum_{k \geq j} A_k \nabla \log q_j$. The two parts add to the stated expression. $\square$

Freezing the acceptance probabilities as a fixed reward keeps only the score part and so gives a biased gradient. The proposition is a direct calculation and we claim no novelty for it. Acceptance-aware training already optimizes the greedy expected accepted length directly, with a window total-variation loss for sampling [112]; BV Loss trains on the exact expected acceptance length under block verification [39]; D-PACE, PARD-2 and VAT weight a cross-entropy by position or by the chance of reaching it [3, 26, 110]; variational speculative decoding optimizes

a bound on sequence acceptance [129]; TIGER uses the realized accepted length as a reinforcement-learning reward [101]; and DBLAST drafts blocks with explicit dependence between positions [36]. The only trained DFlash selector we know of, LiLiCorr’s, uses cross-entropy over candidates [82]. What remains for P6 is the combination: a DFlash 2-style selector over a frozen backbone, trained for rate rather than for accepted length, with the sampled-target gradient above; whether that pays is the experiment. The verification width must be chosen from committed-prefix features before proposals are drawn, for the reason of Section 7.8; choosing a draft-tree budget from a profiled verify-cost curve at inference, as CaDDTree does for DFlash marginals [122], and confidence-scheduled width, as in DSpark [14], are the existing ways to set it.

The cheapest test needs no training. From ordinary greedy runs, log the candidate sets C_i and the completed greedy continuation g , and compute $U = \max\{k : g_i \in C_i \text{ for all } i \leq k\}$, an upper bound on the accepted length of any selector over those frozen candidates; if U barely exceeds DFlash’s actual acceptance there is little to gain. Only then is the rate objective compared, at fixed width, with the strongest matched objective (VAT’s weighting ranks first in its own ablation [26]; D-PACE-style weighting and position-weighted cross-entropy are the other controls) using the same backbone, candidates, selector size, features, data and budget. The pre-registered effect size is $1.25\times$ end to end over the strongest matched baseline, rejected if the upper confidence bound falls below it; a gain that vanishes against an identical selector trained with another objective supports the selector, not the objective [pending: drafter: candidate-support screen U against measured acceptance, then the matched-objective comparison].

9 Exact repair of long windows

A block drafter proposes a whole block in one forward pass and the target verifies it in one pass; the public DFlash card reports a mean accepted length of 7.48 tokens per verify pass at block size 16 on MATH-500 on a B200 [119]. While decoding is limited by weight traffic, a verify pass over a wider block costs little more than one over a narrow block, so the question is whether the target can commit far more than about seven tokens per pass without giving up exactness. Two proposals are tested with oracle experiments designed to kill them cheaply: long-window repair of a DFlash window by the target’s own predictions (P2), and target-anchored residual decoding (P3), which repairs a window by updating the target’s previous computation with fixed low-rank corrections and then audits it exactly. Both keep exactness by the same device: only tokens that an exact audit pass of the target has verified are committed, approximate states never enter the committed cache, and the noise of a position is never resampled. Both are hypotheses in this revision.

Prior work. Reusing the verifier’s work on rejected positions is an active line. ReTrace conditions the next DFlash block on the rejected trajectory [53], DFlow feeds the verifier’s hidden states at rejected positions back to the drafter [124], Carryover Drafting reuses those states as extra draft context [41], and CURE repairs uncertain positions of block-parallel drafts locally [55]; token recycling reuses candidate tokens from earlier steps [58], and CARD and H-Spec restructure parallel speculation around cached queries and drafter state [35, 128]. Jacobi-style iteration has been applied to translation, to feedforward computation, to image generation with probabilistic acceptance and to causal language models trained for it [31, 83, 88, 89, 95], and speculative speculative decoding overlaps drafting with verification [45]. P3’s anchored update is closest to computation reuse across similar inputs, which DeltaCNN, Eventful Transformers and CacheBlend apply to video frames, vision tokens and cached retrieval contexts [19, 72, 116]. None of these, as far as we found, commits a long window through an exact audit of a cheap residual evaluator of the target itself; that combination is what the kill tests below examine.

9.1 Setting and cost model

Greedy decoding solves a triangular system: with $F_i(y_{<i})$ the target’s argmax after the committed prefix and $y_1, \dots, y_{i-1}$, the continuation satisfies $y_i = F_i(y_{<i})$. A verify pass evaluates F at every position of a candidate block from the committed prefix and commits the longest prefix with $y_i = F_i(y_{<i})$ plus the target’s token at the first mismatch. The commit is exact for any candidate, because every committed token is the target’s own

decision on a verified prefix and the committed recurrent state and attention cache are those the target computed for that prefix. The Jacobi map $y_i^{(r+1)} = F_i(y_{<i}^{(r)})$ fixes at least one more position per application and reaches the solution within the block length; this is Jacobi iteration applied to decoding [83, 89], and lookahead and consistency-trained decoders pursue related fixed-point iterations [22, 44]. For sampling, fixed per-position random inputs held across repairs give the target law, as in Theorem 7.6.

Let DFlash commit A_D tokens per cycle of cost C_D (draft, verification, state commit and control). A repair scheme that commits A_R tokens per attempt at cost $C_R = C_{\text{init}} + C_{\text{anchor}} + k C_{\text{repair}} + C_{\text{audit}} + C_{\text{state}} + C_{\text{control}}$, averaged over all attempts including failures and fallbacks, speeds decoding up by $S_{\text{decode}} = A_R C_D / (A_D C_R)$. Prefill and scheduling take a fraction f of a request’s time that repair does not change, so end to end $S_{\text{e2e}} = 1/(f + (1 - f)/S_{\text{decode}})$; a 5× end-to-end gain needs $S_{\text{decode}} = 5.44$ at $f = 2\%$ and 9 at $f = 10\%$ (derived). Two oracle bounds use a perfect candidate of width B (the reference continuation, or a forced full acceptance). With $V(B)$ the target verify pass and $C_{\text{state}}(B)$ the work needed to continue from the accepted boundary,

$$S_a(B) = \frac{B C_D}{A_D (V(B) + C_{\text{state}}(B))}, \quad S_b(B) = \frac{B C_D}{A_D (C_{\text{anchor}}(B) + C_{\text{audit}}(B) + C_{\text{state}}(B))}. \quad (17)$$

S_a is an ideal drafter paying one verify pass per block of B tokens; S_b is P3’s two-pass design with free repair, $C_{\text{anchor}}(B)$ a full pass that also writes the anchor cache and $C_{\text{audit}}(B)$ a full pass. If S_b cannot reach the target end to end, the two-pass design is dead whatever the repair does; if even S_a cannot, better drafting alone cannot reach it with this verifier. A concrete repair pays only if $A_R/C_R > A_D/C_D$, and it is rejected outright if a lower bound on its cost gives $C_R^{\text{lower}}/(B + 1) \geq C_D/A_D$, since no attempt commits more than $B + 1$ tokens.

9.2 What a repair must achieve

Local low rank does not imply useful repair of a whole model. Three things must hold separately: the fixed cheap operators stay accurate after candidate tokens change, not only near the anchor; their errors stay below the decision margins after attention, nonlinearities, the recurrent state and later layers; and each sweep resolves enough positions to pay for itself and for the audit. A sufficient condition links these quantities. Let m_i be the exact top-two logit margin at position i , ε_i a bound on the logit error the approximate evaluator makes at i when its inputs are exact, b_j a bound on the error in the current guess’s effect at an earlier position j ($b_j = 0$ once j is correct), and C_{ij} the influence of position j on the logits at i . If $2(\varepsilon_i + \sum_{j < i} C_{ij} b_j) < m_i$, position i is correct after the next sweep, by the triangle inequality. Two consequences follow. A persistent approximation error η in a recurrence with contraction factor $a < 1$ leaves an error floor of about $\eta/(1 - a)$. And a faithful evaluator can advance only as fast as exact Jacobi iteration, which on a copy chain $F_i(y) = y_{i-1}$ gains one position per sweep even with exact rank-one operators; a non-faithful map can jump ahead, as consistency-trained models do [44], but that would be better drafting, not repair. Expected accepted length is best read through hazards: with h_i the conditional failure rate at position i given an accepted prefix, $\mathbb{E}A = \sum_a \prod_{i \leq a} (1 - h_i)$. The decisive control is a small target-conditioned drafter given the same anchor information and matched in compute, memory traffic, training data, verifier width and target calls; if it does as well, the supported claim is better target-conditioned drafting, not cheap repair of a trajectory. This is analysis that fixes the measurements, not a result.

9.3 Perfect continuations at width B

$V(B)$ and $C_{\text{state}}(B)$ are measured inside SGLang’s DFlash worker in the serving configuration (FlashInfer attention and GDN kernels, CUDA graphs, overlap scheduling, concurrency 1) at block widths $B = 16, 32, 64, 128, 256$, forcing every verify pass to accept the whole block; the committed tokens are then meaningless, but a pass’s cost does not depend on token values. CUDA events around each phase of the cycle give GPU times without host synchronization. SGLang’s GDN verify kernels write one FP32 state per verified position and the commit copies the accepted one back; the replay protocol, which keeps per-token inputs and folds the accepted prefix at commit, and a microbenchmark that separates the per-position state writes from the verify arithmetic are timed alongside. Feeding the reference continuation instead of forcing acceptance shows how often a verify pass at width B rejects the plain-decoding continuation; such rejections are the stock noise floor of Section 3.6, not drafting errors.

[pending: repair: $V(B)$, $C_{\text{state}}(B)$, draft cost and cycle period per width, acceptance of the reference continuation per width, and $S_a(B)$, $S_b(B)$; evidence/repair/stage_a_timing.json, evidence/repair/stage_a_oracle.csv]

9.4 Jacobi repair from DFlash windows

The second test asks how close exact repair from real DFlash windows gets, with no reference tokens. Every DFlash block $y^{(0)}$ is verified, and then k further full target passes run from the same committed prefix: *recycle* sweeps apply the Jacobi map, and *keep* sweeps replace only the first mismatching token by the target’s token and keep the rest of the draft. The pass on the original draft is then re-run and committed, so the trajectory is plain DFlash and each sweep’s accepted length is measured on real windows. Re-running a verify pass on the same prefix needs the committed GDN convolution window restored first, because the verify kernel shifts it in place, and the probe checks that the re-run reproduces the first pass’s argmax exactly. A sliding variant drafts each new block from the previous pass’s suffix predictions and fills the rest with DFlash.

Pending figure. Accepted drafts against the number of sweeps, recycle and keep, per block width.

Data: evidence/repair/jacobi_progress.csv

9.5 Anchored residual evaluation

For every linear operator $z = Wx$, P3 keeps the anchor $(\bar{x}, \bar{z} = W\bar{x})$ from a full pass over a DFlash block and repairs a changed block with $\tilde{z} = \bar{z} + (WU)U^T(x - \bar{x})$, where the orthonormal basis U is fixed per operator input and WU is precomputed, so a repair sweep streams WU instead of W ; a basis computed from the current correction would stream W again. Nonlinearities, normalizations, the causal convolution, the GDN recurrence from the exact committed state and attention over the exact prefix run exactly on the repaired values. Bases are the leading principal components of each operator’s exact input changes on a development split (split by request), at ranks 16 to 128. Each held-out block is replayed twice against the original anchor, $y^{(1)} = F(y^{(0)})$ and $y^{(2)} = F(y^{(1)})$, and at every position the test records whether $\arg \max \tilde{z}_i = \arg \max z_i$, the ratio $R_i = 2 \|\tilde{z}_i - z_i\|_\infty / m_i$ ($R_i < 1$ guarantees the same decision), the first disagreement, and the change from the first replay to the second. A free-running repair then iterates, audits every iterate exactly, and reports the accepted prefix after each sweep and the hazards h_i .

[pending: repair: decision agreement, R_i , hazards and progress per rank; evidence/repair/residual_eval.json]

9.6 Verdicts

[pending: repair: kill-test verdicts for P2 and P3 against the criteria of Section 9.1, and which of the three P3 conditions fails first; evidence/repair/README.md]

10 Stacking every axis

The certified head and a better drafter each attack one term of the verifier’s cost. When the head is a sixth of a plain step’s bytes and the recurrent state dominates at high concurrency, a paper about avoided work also has to ask what the whole stack can gain when every term is attacked, which of those gains keep the target’s decisions, and what the others cost in quality. This section starts from bytes and floating-point operations per token, which, under a stated execution model and an assumed tensor-core rate, bound what each lever can deliver, and then lists the levers, how they compose, and the two stacks they form.

10.1 Where the ceilings are

A decode step streams the linear weights once whatever the batch size, $W = 8.41$ GB, and everything else is per request: 100.7 MB of FP32 recurrent state read and written, and 32 KiB of attention KV per context token

Lever stack	MB per request per step	Ceiling (tokens/s)
Plain, FP32 state	114.0	23.7k
16-bit state	63.6	34.6k
Replayed state writes, FP32	66.8	33.6k
Replayed state writes, 16-bit state	40.0	44.0k
Replayed state writes, int8 state	26.7	52.1k
Replayed, 16-bit state, FP8 weights and KV	34.6	61.7k
Replayed, int8 state, FP8 weights, KV and head	21.2	78.9k
MTP (accepted length 3), stock verification, FP32	–	19.5k
MTP (accepted length 3), replayed verification, FP32	–	34.3k

Table 10: Derived large-batch throughput ceilings from Equation (18), which assumes that per-request kernels do not overlap the weight GEMMs, with the HBM read bandwidth taken as 3.79 TB/s and the tensor-core rate assumed at 70% (BF16) or 60% (FP8) of the datasheet peak. An engine with that overlap is bounded only by Equation (19). Not measurements; evidence/moonshot/ceilings.json.

(Table 1). With F the floating-point operations per token (two per weight), s the per-request bytes per step, BW the measured HBM read bandwidth and P an achievable tensor-core rate, and with each layer’s weight GEMMs and its per-request kernels (recurrent state, attention) running in turn, as they do at the pin, the step time at batch B is bounded below by

$$t(B) \geq \max\left(\frac{W}{BW}, \frac{BF}{P}\right) + \frac{Bs}{BW}, \quad (18)$$

since in that order the weight stream and the GEMM arithmetic can overlap each other while the per-request traffic adds to them. An engine that overlapped the memory-bound per-request kernels with the batch GEMMs would be bounded only by

$$t(B) \geq \max\left(\frac{W + Bs}{BW}, \frac{BF}{P}\right), \quad (19)$$

whose large-batch ceiling is $\min(BW/s, P/F)$; Equation (18) is the bound for the engine as it runs, not for every engine on this machine. P is taken as 70% of the BF16 and 60% of the FP8 datasheet peaks, assumptions stated so that the ceilings can be recomputed. Two consequences follow before any measurement. Under Equation (18), as B grows the throughput ceiling approaches $1/(F/P + s/BW)$, set by the per-request state and the arithmetic rather than the weights, so halving the state’s bytes or writing it less often raises it while a faster head does not; and even with every byte removed, the arithmetic term caps an FP8 engine, so no single lever gives an order of magnitude at high concurrency. At batch one the weights bound the step, and the draft head and the target’s weight bytes are the terms to cut.

Table 10 and Figure 10 evaluate the floor for stacks of levers (evidence/moonshot/ceilings.json). The bandwidth input is the profile workstream’s measured read peak of 3.79 TB/s, taken here as an input whose evidence is [pending: profile], and P is assumed as above; the ceilings are derived, not measured. At batch one the floor is 2.25 ms per token (445 tokens/s), of which the weights alone account for 2.22 ms; plain decoding measured 281 tokens/s at concurrency one in a single run, 63% of that rate (evidence/moonshot/ceilings.csv, evidence/moonshot/lever_sweeps_quick.csv). At large batch the plain engine’s ceiling under Equation (18) is 23.7k tokens/s, because each request moves 114 MB per step and needs 8.4 GFLOP. Halving the state bytes, writing the state less often and moving weights and cache to FP8 raise the ceiling to between 34k and 79k tokens/s; none reaches ten times the plain engine’s, because 8.4 GFLOP per token cap an FP8 engine near 140k tokens/s at the assumed rate even with every byte removed (about 235k at the datasheet FP8 peak, of which the assumed rate is 60%; derived from the same constants).

[pending: moonshot: repeated decode-step and served measurements to $B = 1024$ per state and weight format]

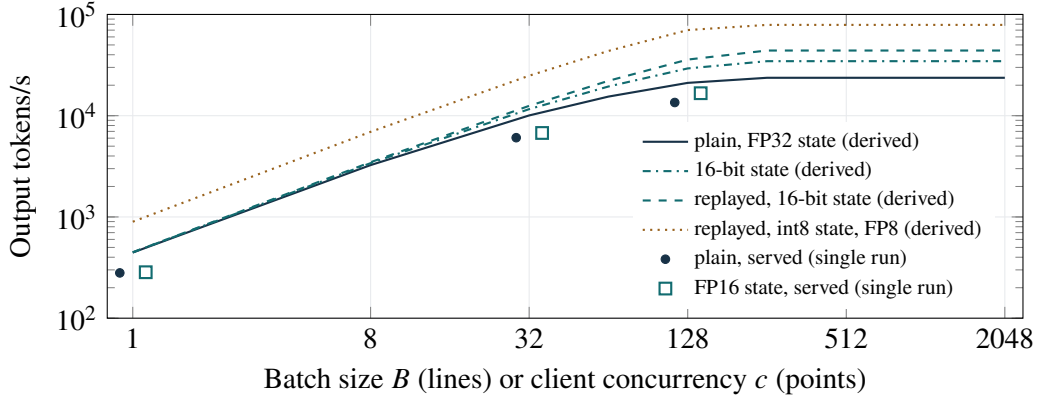


Figure 10: Derived throughput ceilings per lever stack against batch size under the layer-serial model of Equation (18) and the assumed tensor-core rate (lines, `evidence/moonshot/ceilings.csv`) and served output throughput of single benchmark runs at client concurrency 1, 32 and 128 (points, `evidence/moonshot/lever_sweeps_quick.csv`; the two series are offset slightly left and right of their concurrency so that both stay visible). The gap between line and point at a given concurrency is overhead above that layer-serial floor, and at high concurrency it includes the server’s streaming path.

10.2 Speculation and the recurrent state

The hybrid architecture changes the economics of speculation. A verify pass over D draft tokens must be able to roll back to any accepted prefix, and SGLang’s Triton verify kernel does so by writing one full FP32 state per draft position and copying the accepted one back at commit. Per request and cycle that is one state read, D state writes and a read and write at commit: 352 MB for $D = 4$, three and a half plain steps (derived from the code path). With an accepted length of about three, speculation spends more state bytes per output token than plain decoding while saving weight bytes, so at high batch, where the state dominates, its bandwidth advantage can vanish. SGLang’s replay path for speculation stores each draft token’s inputs and folds the accepted prefix into the state at commit, reading and writing the state once per cycle, which would make speculation cheaper in state bytes per token than plain decoding; trees do not get this benefit yet, because the replay path handles only chains and a tree keeps one full state per node [23, 59].

State precision is the cheapest lever and the most dangerous. DAMP reports reasoning accuracy of 85.5 with FP32 recurrent state, 84.6 with FP16, 79.7 with BF16 and below 30 with FP8 or int8 on another GDN model [123], and LeapQuant recovers near-FP32 accuracy with 8-bit state by re-quantizing once per 16-token window [70]. SGLang accepts FP16 state (`--mamba-ssm-dtype float16`).

[pending: moonshot: MTP with and without replayed verification at concurrency 128 to 1024; logit-probe KL, divergence rate and GSM8K for FP32, FP16, BF16 and FP8 state with and without replay; `evidence/moonshot/lever_sweeps_hc.csv`, `evidence/moonshot/lever_quality.csv`]

Writing the recurrent state less often. Rounding-preserving live replay (Section 3.9) is exact and cuts dense-state traffic per step from $2D$ (read and write) to $D + D/m$ when the state is flushed every m steps, 0.625 of the original at $m = 4$, for a few percent more storage (derived). If the recurrent kernel is a fraction f of a decode step, a kernel twice as fast gives at most $1/(1 - f/2)$, removing it entirely $1/(1 - f)$, and the $m = 4$ traffic cut alone $1/(1 - 0.375f)$ (Amdahl). From bytes alone, f would be about 0.55 at batch 128 and a 512-token context (12.9 GB of FP32 state against 8.41 GB of weights and 2.1 GB of attention KV; derived, Table 1), which would give 1.38 $\times$, 2.2 $\times$ and 1.26 $\times$; the measured time share is [pending: profile: GDN kernel share of a batch-128 step]. The pre-registered test decodes 512 tokens greedily at batch 128 from 2,048-token prompts with FP32 state and no speculation, first compares every state word and GDN output of the replayed and dense paths (one differing word rejects bit-exactness), and then times both: the claim is at least 1.10 $\times$ end-to-end decode throughput, rejected if the upper end of the paired 95% confidence interval falls below 1.10 $\times$. SGLang’s own replay path qualifies only

if it passes the bit-exactness check. [pending: moonshot: bit-exactness check and paired timing]

Sharing computation across requests. Factoring hidden or recurrent state across unrelated requests does not survive analysis. An exact factorization with a dense residual costs the full projection, so it saves only if $r(d + B) + z < Bd$ with a zero or sparse residual; SiLU and gating raise rank; full-rank pairwise state differences force private factors at least as large as the dense state; one gated-delta step takes four identical states to batch rank four (tests/test_state_structure.py, evidence/state_structure/); online factorization costs of order B^2 times the update; and reassociation breaks equality with the reference. The one exact candidate is structural: the first GDN layer’s input projections (12,352 outputs) depend only on the current token, so they could be read from a precomputed per-token table (5.71 GiB for the whole vocabulary) with convolution and recurrence kept private. Its ceiling is about 0.75% of the dense projection work per token (derived), and at batch 128 even a 99% table hit rate leaves about 72% of batches with a miss. The pre-registered criterion rejects it unless the stage’s measured share of a decode step allows a 1% end-to-end gain [pending: profile: measured share of the layer-0 input projections, PR #13].

10.3 Levers by axis

Table 11 lists the levers, each built as a separate server flag or environment variable so that any subset composes. A lever is *exact* when it leaves the target’s arithmetic unchanged, so every emitted token is the target’s decision at its own batch shape under the stock-kernel contract, with its measured noise floor; drafting levers are exact because the target verifies every token. A lever is *lossy* when it changes the target’s arithmetic or decisions, and it carries a measured quality cost. SGLang’s replay path for plain decoding belongs to the lossy group as implemented: its decode kernel reconstructs the state on tensor cores from BF16-cast operands and persists that rounding at every flush (code reading at the pin); the bit-exact alternative is the rounding-preserving replay above.

The first single-run measurements on the benchmark harness (plain arm with the prefix cache on, capacity 128) separate the levers by where they act (evidence/moonshot/lever_sweeps_quick.csv). FP16 recurrent state raised served throughput at concurrency 128 from 13.5k to 16.7k tokens/s (1.24×) and changed nothing at concurrency one, which is what a per-request byte term predicts; its quality cost is [pending: moonshot: GSM8K and logit-probe agreement for FP16 state]. FP8 KV did nothing at these short contexts (0.99×). FP8 W8A8 weights could not use their CUTLASS GEMM: the one in the installed aarch64 sg1-kernel aborts at run time because that binary was not compiled for sm_90a, as its logged error states, and the Triton route showed no consistent gain (0.96–1.06× across concurrencies), a negative result until a different GEMM route is wired. These are single runs without repeats. A first engine-only pass of decode steps at batch 512 took 26.9 ms with FP32 state, 23.0 ms with BF16 and 22.5 ms with FP16 (evidence/moonshot/decode_ceiling_try1.csv); it is noisy at small batch sizes and is to be repeated.

Two levers need engine changes. SGLang’s hot-vocabulary draft head (--speculative-token-map, after FR-Spec [125]) does the wrong thing on Qwen3.5 as read at the pin: the MTP layer’s head is the tied embedding, and set_embed_and_head ignores the truncated head in that case, so the drafter scores all 248,320 rows while the worker remaps indices as if it had scored the hot subset; the patch unties a reduced head for the drafter only. The second change adds a relaxed greedy rule for chains, accepting a draft token whose target logit is within g of the maximum; SGLang’s existing relaxed thresholds act only when sampling, and the rule is lossy by construction, so its quality curve is the measurement [99] [pending: moonshot: the patches and their tests].

10.4 Interactions and the two stacks

Speedups from separate levers do not multiply, and quality losses need not add. Three interactions are predictable from the mechanism. A lossy target lowers acceptance, because the drafter was trained against the BF16 target; the published int4 recipe fine-tunes its drafter against the int4 target for exactly this reason [38]. A cheaper verifier makes drafting relatively more expensive, which raises the value of a cheap draft head. And every lever that shrinks per-request bytes also raises capacity, so its measured gain depends on whether the server was allowed to

Axis	Lever	Class	Status
Progress per verify pass	MTP and DFlash drafting, deeper chains, trees where the GDN path allows, n-gram drafting	exact	[pending]
	Relaxed greedy acceptance within g logits	lossy	[pending]
Bytes per verify pass	Certified int8 head (Section 6)	exact	[pending]
	Replayed verification of the recurrent state	exact*	[pending]
	FP8 W8A8 weights, FP8 KV; int4 target with a matched drafter [38]	lossy	FP8 weights: no consistent gain; FP8 KV: no gain (single runs); int4 [pending]
Bytes per draft step	Hot-vocabulary or low-precision draft head	exact	[pending]
Concurrency and state	One state slot per request (lifts the 133-request cap)	exact	[pending]
	FP16 state; replayed decoding; rounding-preserving replay (P4)	lossy, lossy, exact	FP16: 1.24× at $c = 128$ (single run, quality pending); others [pending]
System overheads	Host gaps and synchronizations, fusion, attention backend per shape	exact	[pending]

Table 11: Levers by axis. *The replay fold is written as a bitwise clone of the verify kernel’s state update, but the verify outputs use a chunked reformulation whose rounding differs, so it is exact only up to the batch-shape noise floor until the differential tests confirm more.

use that capacity. Each pair is therefore measured in the baseline, A , B , $A+B$ pattern rather than by multiplying isolated speedups.

The *exact stack* admits only levers that keep the target’s decisions: capacity and scheduling changes, drafting levers, replayed verification if the differential tests confirm it, and the certified head. The *lossy stack* admits everything, under a quality budget declared before measuring: at most 1.0 point of GSM8K accuracy below the reference on the full test split with thinking on (paired, with an exact McNemar test), teacher-forced top-1 agreement of at least 98% and mean top-20 KL divergence of at most 0.01 nats on a fixed probe set, each reported next to the reference’s own run-to-run noise. The combination’s quality is measured on the combined configuration, never summed from its parts. Proposition 7.5 explains why the budget is empirical: a per-token logit error small enough to certify a long output as close in total variation is far below what any low-precision lever delivers.

[pending: moonshot: the interaction matrix (speed at concurrency 1 and at peak, acceptance and quality for each measured pair) and both frontiers from one server launch per arm; evidence/moonshot/interactions.csv, evidence/moonshot/frontier_exact.csv, evidence/moonshot/frontier_lossy.csv]

10.5 What did not work

So far: FP8 W8A8 weights, whose CUTLASS GEMM in the installed aarch64 `sgl-kernel` aborts at run time because that binary was not compiled for `sm_90a`, and whose Triton fallback showed no consistent gain in single runs; FP8 KV cache, which saves nothing at short contexts; BF16 recurrent state, which moves the same bytes as FP16 and is worse on quality in published measurements on another GDN model [123]; and 2:4 sparsity, which the engine does not support at the pin (its compressed-tensors loader raises an error for 2:4 schemes; code reading, `python/sglang/srt/layers/quantization/compressed_tensors/compressed_tensors.py`). An MTP server with BF16 state at 512 concurrent requests also failed to capture its verify graph (Section 11.3).
 [pending: moonshot: remaining levers that fail or regress, with their mechanisms]

11 Serving results

Every speedup in this paper is measured against a server, not a kernel. This section defines that measurement: what runs, on which prompts, how the latency-throughput frontier is computed, how the baselines are made strong before anything is compared with them, and which quality check a change must pass.

11.1 Setup and workload

All runs use the machine and engine of Section 1.1: one GH200, PyTorch 2.13 with CUDA 13.0 forward-compatibility libraries, FlashInfer 0.6.18 and SGLang at the pin, serving the BF16 target at its pinned revision. Target attention uses FlashInfer, and the vision encoder, which text requests never reach, uses Triton. The client is AIPerf [64].

Speculative acceptance depends on what the model is writing, so random-token prompts would measure the wrong thing. The benchmark uses a frozen set of 1,857 public prompts in three domains: chat (the first turns of the 80 MT-Bench questions [126] and English root prompts from OASST1 [43]), code (the 164 HumanEval problems [11] and MBPP test tasks with their assertions [4]) and mathematics (GSM8K training questions with a request to box the final answer [15]; the GSM8K test split is reserved for the quality check, so no workload prompt is a quality problem). Each source is pinned to a Hugging Face revision and the builder reproduces the files byte for byte. Each domain is cut into disjoint warm-up (43 prompts), tuning (192) and confirmation (384) sets, and every split interleaves the domains so that any prefix of it is balanced, which matters because low-concurrency points use only a prefix. Configurations are chosen on the tuning split and reported on the confirmation split. Templated prompts are short: the confirmation split has a mean of 87.9 prompt tokens (median 78, 99th percentile 283), with chat shorter (mean 47) than code (130) and mathematics (86) (`bench/workloads/mixed-v2/manifest.json`).

Requests use the model’s chat template with thinking enabled (the template’s default) and no reasoning parser, so the thinking text is streamed and counted as output. Decoding is greedy. The throughput panel fixes the output at 512 tokens with `ignore_eos`, and every point checks that each response has exactly that length. A fixed length is a faithful sample of the model’s behaviour only while the model has not chosen to stop, so a natural-stopping run on 576 tuning prompts measured where greedy outputs end. Only 3.8% stopped before 512 tokens and 0.7% had entered a detected repetition loop by then (a 256-token window with fewer than 30% distinct 8-grams), so almost all of the fixed-length panel is text the model would have produced anyway. The same run shows why greedy decoding is not used for the quality check: 142 of the 576 outputs (25%; 36% in mathematics, 32% in chat, 6% in code) ran to the 16,384-token limit, a degeneration Qwen’s model card warns about for greedy thinking [75]. The run used the previous workload version, whose mathematics prompts came from GSM8K test, and other processes used up to 24 CPU cores during it, so it is a length measurement, not a timing (`evidence/bench/workload/natural_lengths_tune.json`).

11.2 Metrics

For request i at a sweep point let n_i be the number of generated tokens the server reports in the `usage.completion_tokens` field, s_i the time the client sends it, f_i the arrival of its first non-empty streamed chunk and e_i the interval from s_i to its last non-empty chunk. Over the N measured requests the frontier plots

$$x = \frac{1}{N} \sum_{i=1}^N \frac{n_i}{e_i}, \quad y = \frac{\sum_i n_i}{\max_i (s_i + e_i) - \min_i s_i}, \quad (20)$$

so x is the mean per-request output rate *including* the time to first token, and y is output tokens per second on the one GPU. Two companions are reported and never substituted for them. The decode rate $x_{\text{dec}} = \frac{1}{N} \sum_i (n_i - c_i) / (e_i - (f_i - s_i))$, where c_i is the exact token count of the first chunk from per-chunk usage (a speculative step streams several tokens per chunk), excludes queueing and prefill; one token per chunk is never assumed. The steady-state throughput counts only the tokens streamed between the first and the last send, when the client holds its full concurrency, and so excludes the drain at the end of a point. Each point also records time-to-first-token, inter-token and end-to-end latency percentiles, failures, the per-request speculative statistics the server attaches to each stream (verify steps, accepted drafts, and accepted length as output tokens per verify step with the bonus token included), and deltas of the server’s counters, among them the fraction of decode passes that replayed a CUDA graph.

11.3 Protocol and baselines

A server arm is a model revision plus a declarative set of launch flags. The harness starts it once, verifies it, and sweeps client concurrency $c \in \{1, 2, 4, 8, 16, 32, 64, 128\}$ against it in closed loop. Before trusting a launch it checks that decode CUDA graphs were captured for every batch size up to the server’s capacity (the target decode graph, or the target-verify, draft-decode and draft-extend graphs under speculation), that prefill graphs were captured, that the overlap scheduler is running (the scheduler reports it, no fallback is logged, and a stack sample of the scheduler process shows its overlap loop), and that the recurrent-state cache did not reduce the capacity.

Capacity needs care on this model. The attention cache is not the constraint: 32 KiB per token, and the default memory split leaves room for about a million tokens. The recurrent state is: each running request holds a 50.3 MB FP32 state per slot, the prefix cache with the overlap scheduler reserves several slots per request, and speculative verification adds one intermediate state per draft token. SGLang also resets speculative servers to 48 running requests, and logs it, unless the capacity is given [\[pending: bench: capacity limits per arm at default settings from the server logs\]](#). Feasibility probes show what launches: plain decoding serves 256 concurrent requests with the prefix cache and 1,024 without it, MTP speculation serves 256 without the prefix cache, and an MTP server with BF16 state at 512 fails at verify-graph capture on a FlashInfer workspace overflow. These probes establish what runs, not performance (evidence/bench/probes/). Every arm is therefore launched at a capacity of 128 running requests, the top of the client sweep, with the state cache sized for it and the remaining static memory given to the attention cache and the verify states.

Each point sends $\max(64, 8c)$ measured requests, the first prompts of the confirmation split in order, after one wave of warm-up requests at the same concurrency, and the prefix cache is flushed before every point so no measured prompt is served from cache. Repeats launch the server afresh and alternate the order of concurrencies, ascending then descending, so that drift does not align with concurrency. Timed runs hold the exclusive GPU lock, and because the scheduler, tokenizer manager and client are single-threaded CPU loops, each point also waits until other processes use fewer than two cores and records the foreign load it saw.

The speedup denominator is the strongest existing configuration, not a default. Arm A is ordinary autoregressive decoding. Arm B is native MTP speculation, whose draft steps reuse the target’s tied head. Arm C is the public DFlash drafter (Section 8). On the tuning split the search covers chain depth (one to five steps, and seven without the prefix cache), draft trees (top- k 2 and 4, which run on this hybrid model, with accepted lengths of 3.45 and 3.56 at concurrency 1 in the feasibility probes), attention and GDN kernel backends, the verify attention mode,

SGLang’s replayed GDN verification, its batch-adaptive depth, the prefill chunk size and graph backend, the prefix cache and the streaming frontend [pending: bench: chosen MTP and DFlash configurations with the tuning evidence; evidence/bench/tuning/].

11.4 Quality check

Output length and throughput say nothing about whether a change preserved the model. The quality check is the full GSM8K test split (1,319 problems) scored with the GSM8K grader of the `sgl-eval` package, version 0.1.2 (a zero-shot prompt, the last boxed answer, symbolic equality) with thinking on, natural stopping at 16,384 tokens, and Qwen’s recommended thinking-mode sampling for precise tasks (temperature 0.6, top- p 0.95, top- k 20) under a fixed request seed. Two runs are compared problem by problem: the accuracy difference, an exact McNemar test on the discordant problems, and the share of identical final answers. Sampled accuracy tests equality in distribution for a lossless change and prices a lossy one; it cannot show that greedy outputs are unchanged, which the output comparisons of Section 3.6 address.

[pending: bench: GSM8K accuracy for arms A, B and C with intervals and paired tests; evidence/bench/quality/]

11.5 Frontiers

Pending figure. Output tokens per second per GPU (y) against output tokens per second per user (x , time to first token included) on the confirmation split for arms A, B and C at concurrency 1 to 128, mean of three launches with the range shown; later the certified head, the exact stack and the lossy stack.

Data: `evidence/bench/confirm/frontier.csv`

[pending: bench: frontier table (x , y , time to first token and accepted length per arm and concurrency, with speedups over arm A and over the tuned speculative arm) and run-to-run variance across three launches and five within-launch repeats; evidence/bench/confirm/frontier.csv, evidence/bench/variance/]

[pending: bench, integration: before/after frontiers for the certified head in MTP drafting, verification and plain decoding, with output-equality checks at the same batch shape (H4)]

11.6 Measurement hazards

Two effects outside the GPU shaped this protocol. At high concurrency plain decoding can be limited by the streaming path rather than the GPU; speculative arms stream several tokens per chunk and are less exposed, so a frontend bottleneck would inflate their measured advantage [pending: bench: frontend diagnosis at concurrency 64 to 256 on a quiet host; evidence/bench/frontend/]. And the GPU lock does not keep CPU-heavy jobs off the host, which is why every point records foreign CPU load and waits for a quiet machine.

12 What did not work, and the limits of what did

12.1 What the review of the earlier revision overturned

Several claims of the earlier revision did not survive review, and they are recorded here rather than silently removed.

- The counterexample showing that sampled verification depth chosen from realized proposals biases the output law was presented as a contribution. DSpark and D-cut state the condition with their own counterexamples [14, 57]; it is now an illustration (Section 7.8).
- “Progressive precision” was proposed without citation. Certified greedy screening with a low-precision head and exact re-scoring is implemented publicly [8, 28, 47, 90], and the bound-then-refine pattern is classical [49, 107]. The claim is narrowed accordingly (Section 2.8).

- The static tile screen proposed as an ablation is CSV-Decode’s mechanism [56], and on this head the tile radius exceeds the row norm, so the screen is looser than a per-row bound (Table 6).
- Statements about the engine were out of date at the pin: plain and seeded sampling are already races, seeded noise is counter-keyed, and default sampled verification uses point-mass drafts, whose residual needs no partition function (Section 3.2).
- “Exact” was defined against real arithmetic. It is now defined against the stock kernel at the same batch shape, and the evidence that a certified token equals the stock token is conditional on a modelled error bound for that kernel (Section 3.4).
- The Lean file could not be compiled when the earlier revision was written. Lean 4.19.0 now elaborates it and a second file for the certified decisions (formal/STATUS.md, evidence/precision/lean_check.log).

12.2 Failure criteria

Table 12 pairs each hypothesis with the fastest test that could damage it and the response if it fails. A failed hypothesis stays in the paper with its mechanism; the programme stays ambitious by testing stronger alternatives, not by weakening the meaning of correctness or changing the comparison after the fact.

Hypothesis	Fastest damaging test	Response if it fails	Status
Transport is tighter than self-evidence and the static screen	Aligned real-head replay: ρ against δ^* (Section 5.5)	Drop transport as the head’s evidence source; keep it only if an intersected hybrid pays for its metadata	Crossover derived from weights; ρ [pending]
Certificates save target work	Count rows, re-scoring and completion, not only decision cost	Reject acceptance-only speed claims; measure residual races separately	Cheap decision $\neq$ cheap verification on constructed heads (Section 7.6) [pending]
Certified tokens equal stock tokens	Same-shape token equality on captured states; measured cuBLAS error against the modelled G_i	Widen G_i or send more rows to the stock fallback	[pending]
Fewer bytes read means less time	Head-path time against the stock chain at each served shape	Keep the dense head at shapes where it wins	[pending]
Per-row sparsity reduces weight traffic	Batch unions of candidates and measured HBM traffic	Group compatible rows or stay dense	[pending]
The head is a large share of the step (H1)	nsys attribution	Move effort to the dominant term (state, host gaps)	Bytes derived; time [pending]
Speculative state is equivalent (H5)	Every rejection position, aborts, prefix reuse	Correct the state path before timing	[pending]
Adaptive sampled depth stays exact	Exhaustive tiny laws with candidate-dependent stopping	Non-anticipating policies or fixed-noise verification	Settled by prior art and Theorem 7.6

Hypothesis	Fastest damaging test	Response if it fails	Status
Recurrent state compresses exactly	Exact witnesses; rank audit of captured keys	Reduce state writes instead of state size	Exact witnesses (evidence/state_structure/); audit [pending]
Sharing across requests pays	Measured share of the one structural candidate	Reject	Rejected in analysis; share [pending]
Results transfer	New prompts, a second model or GPU	Narrow the claim and explain the dependence	[pending]

Table 12: Failure criteria and their current status.

12.3 Scope

The measurements concern one model, one engine commit and one GPU, and the experiments are text-only on a multimodal checkpoint. The agreement of certified tokens with the stock kernel rests on a published model of Hopper tensor-core accumulation and on the absence of reduced-precision split- K reductions in the kernels the engine selects; both are tested, neither is a vendor guarantee. The exact CPU references test the certificates on synthetic heads with small vocabularies and widths; they are tests, not proofs, and the Lean files cover the decision logic and scaled-integer bounds, not IEEE rounding, exponentials or probability. The head constants of Table 6 use no hidden states, so they decide nothing about certification rates on their own. Every end-to-end result is pending in this revision.

12.4 What the completed paper should establish

The question was what work a verifier actually needs. The mathematics now says precisely what evidence suffices for each consumer, how to state exactness against the kernel an engine actually runs, and when each of the two evidence sources can be tighter than the other. The measurements will say which of them survives real hidden states, real kernels and real batching on this model, and what the rest of the serving stack contributes when every axis is pushed. A useful outcome includes a well-explained negative one: transport failing on real geometry, or a certified head that reads half the bytes yet loses to the stock chain at the shapes that matter, would each rule out a tempting argument with a mechanism attached.

A Engine integration notes

The source audit uses SGLang commit `bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824`, a frozen research pin rather than the latest revision. The observations below come from reading that code; the integration is proposed engineering work unless a later section reports it measured.

Path under python/sglang/	Observation and proposed integration
srt/layers/logits_processor.py	<code>_compute_lm_head</code> is the single hook through which plain decoding, the MTP drafter and the verifier compute logits: <code>torch.matmul</code> with BF16 output by default, <code>torch.mm(..., out_dtype=float32)</code> under <code>--enable-fp32-lm-head</code> . A certified head replaces this call for admitted consumers only; log-probability requests and unsupported transforms keep the ordinary path.
srt/models/qwen3_5_text.py	The Qwen3.5 head is tied to <code>embed_tokens</code> , which is not a <code>ParallelLMHead</code> , so SGLang’s quantized-head paths do not apply; a low-precision copy must be separate and the BF16 head stays resident for re-scoring and fallback.
srt/models/qwen3_5_mtp.py	<code>set_embed_and_head</code> skips the head when embeddings are tied, so a reduced draft head from <code>--speculative-token-map</code> does not reach the MTP drafter while indices are still remapped [pending: moonshot: test and patch].
srt/layers/sampler.py	Unseeded sampling is an exponential race; seeded sampling is a Gumbel race keyed by (seed, position, token) in FP64 (<code>multinomial_with_seed</code>). The certified race uses the same field.
srt/models/dflash.py	DFlash projects draft hidden states through the target head for its candidates; this is the draft-side summary producer for transport.
kernels/ops/attention/fla/gdn_replayssm_spec_fold.py	The replay fold is written as a bitwise clone of the verify kernel and constrains operation order and warp count; its <code>accept_lens</code> include the bonus token in that backend’s indexing [84].

B Mechanisms carried over but not tested in this revision

The earlier revision developed a broader programme around the head. Nobody is testing these parts in this revision, so they are condensed here with their proved statements; they are hypotheses, not results.

The DFlash 2 selector. At the inspected implementation adjacent candidate scores have the form

$$S_t(a, b) = U_t(b) + \sum_{r=1}^{d_s} A(a)_r H(h_t)_r B(b)_r, \quad (21)$$

with a gate that depends on the draft hidden state [33, 86]. A greedy walk needs only the realized row $S_t(x_{t-1}, \cdot)$, which reduces score work from $O(BLK^2 d_s)$ to $O(BLK d_s)$ at the price of a dependent sequence of small contractions. Alternatively the lattice can emit, for every predecessor, its winning successor $f_t(a) = \arg \max_b S_t(a, b)$, after which the path is $x_t = (f_t \circ \dots \circ f_1)(x_0)$.

Proposition B.1 (Finite-map execution). *Given the same row winners, serial map walking and prefix composition produce the same path. A work-efficient scan of length L over K -entry maps uses $O(LK)$ map work and $O(\log L)$ composition stages, excluding the score producer.*

Proof. The t -th prefix composition applies exactly the first t transitions to the anchor, and function composition is associative. A work-efficient tree scan uses $O(L)$ compositions of $O(K)$ indexed evaluations each, with logarithmic depth. $\square$

Function scanning is standard; the question would be its profitable placement. The inherited CPU reference implements the simpler $O(LK \log L)$ Hillis–Steele scan and checks path equivalence only.

Prefix value. On a fixed lattice with a substochastic surrogate $r_t(a, b)$ for the probability of matching the next target token, $J_r(x) = \sum_{j=1}^L \prod_{t \leq j} r_t(x_{t-1}, x_t)$ is maximized by $V_t(a) = \max_b r_t(a, b)(1 + V_{t+1}(b))$ with $V_{L+1} = 0$. If the estimate $\hat{r}$ also lies in $[0, 1]$ and edge errors are uniformly bounded by ϵ_t , telescoping gives $|J_r - J_{\hat{r}}| \leq \delta = \sum_t (L - t + 1)\epsilon_t$ and optimizer regret at most 2δ . LiLiCorr’s finding that an exact best-path search over a different score decodes worse than the greedy rule [82] is the relevant warning.

A compiler interface for evidence and effects. The earlier revision proposed typed interfaces (head summaries, decision demands, state leases, evidence with an enclosure contract) so that illegal compositions become visible: a top- k summary cannot satisfy a residual-sampling demand, a lease cannot be reused after cancellation, and a real-arithmetic enclosure cannot discharge a bitwise requirement without a numerical bridge. Search over implementations [109] would operate inside that boundary. No part of it is implemented.

Scheduling and structural tuning. If predicted progress $F_i(d)$ is known and cost depends only on the total optional depth m , the snapshot problem is solved by

$$G_i(m) = \max_{0 \leq d \leq L_i} \{G_{i-1}(m - d) + F_i(d)\}, \quad \max_m \frac{G_B(m)}{C(m; z)}, \quad (22)$$

which is exact only under that cost hypothesis. Graph tiers make cost discontinuous: with rewards $(1, 1.9, 2.7)$ and costs $(1, 2, 2)$ at depths $(0, 1, 2)$, depth one looks worse than zero while depth two is best, so stopping at the first local decline fails. Autotuning a kernel that mutates state must restore that state between trials [98].

GPU execution families. A certificate path can run as a two-stage mask-then-compact pipeline, as a bounded number of guarded refinement rounds inside a CUDA graph, or as a persistent work queue; ordinary inter-block spin barriers are not a legal substitute for global synchronization. Graph-safe designs need fixed buffers, device-resident masks and stable addresses, and a guarded stage that does nothing still costs a launch [100]. Nsight Systems establishes the critical path and Nsight Compute the traffic of selected kernels, whose replay and cache control change what is measured [67]. Hopper’s asynchronous warp-group MMA requires `sm_90a`, and Blackwell’s data-centre and consumer targets expose different instruction sets and limits [63, 65, 68]; implementation backends range from Triton to TileLang and CuTe [69, 97, 98].

C Mathematical extensions and small falsifiers

Transformed logits. If a transform adds a known per-token correction b_i with tile bounds $b_c^- \leq b_i \leq b_c^+$, then $z_i^d + a_c - \epsilon_c + b_c^- \leq z_i^t + b_i \leq z_i^d + a_c + \epsilon_c + b_c^+$, and the mass bounds multiply by $e^{b_c^\pm}$. A mask that deletes unknown members cannot preserve the unmasked lower mass; a safe lower bound may collapse to zero.

One anchor mass cannot follow a temperature change. The unnormalized masses $(1, 3)$ and $(2, 2)$ both sum to four, while their squares sum to ten and eight, so a mass at a new temperature cannot be reconstructed from the old sum alone.

A conditional rate certificate. If positive intervals enclose expected progress and time for a candidate and a baseline, $R_1^-/T_1^+ > R_0^+/T_0^-$ (with $T_0^- > 0$) certifies $\lambda_1 > \lambda_0$ in (2). Benchmark confidence intervals are not deterministic enclosures, so the inequality exposes what must be estimated rather than certifying a speedup.

Agreement at an anchor proves nothing away from it. Let $h^d = (1, 0)$ and $h^t = (1, 1)$. Rows with the same first coordinate have identical anchor logits while their second coordinates make the target logits arbitrarily different, so an anchor maximum or mass alone cannot certify the target; a geometric bound, more projection or exact completion is necessary.

RNG tests. Exhaustive tiny-law tests should cover point-mass proposals, zero support, $p = q$, acceptance at the comparison boundary, all-accept cycles, first-position rejection and candidate-dependent stopping, with proposal draws, acceptance uniforms and residual noise aligned across the compared procedures.

D CPU evidence carried over from the earlier revision

The earlier revision’s exact references use bounded integer head weights and hidden vectors, with unnormalized mass 2^z represented exactly in rational arithmetic; this is an exponential model with logits scaled by $\log 2$ and avoids trusting floating exponentials. Eighteen test methods in the decision reference and three in the race reference pass, with four local tests of the benchmark client; Table 14 lists the generated finite instances inside them. The counts are not independent confirmations of anything about a trained model, and the race tests use fixed positive rational priorities, so they establish pathwise equivalence of the bounded and dense algorithms, not continuous Gumbel randomness. Seeds, commands and results are in `evidence/decision_tests.json`, `evidence/race_tests.json` and their logs.

Check	Executed count	Scope
Top- k plus tail composition	250	Exact union and associative merge.
Transported tile bounds	300 heads; 1,048 tiles	3,787 weight rows enclosed.
Greedy certified screening	250	Same winner under a deterministic total order.
Retained-tail tightening	565 tiles	Exact mass enclosed; bounds no looser.
Acceptance interval guards	1,600	Certified bits agree with complete mass.
Unresolved interval measure	160	Exact agreement with Theorem 7.8.
Lazy acceptance	500	With and without retained seeds; finite completion.
Coupled one-step sampling	200	Same output with exact residual fallback.
Rejection-correction identity	300 pairs	Includes zero-probability support entries.
Demand frontier	1,092 partial states	5,460 complete decision assignments.
Exclusion-aware race summaries	300	Exact maxima after support exclusions.
Target priority race	250	Same token as a dense priority race.
Sparse-support residual race	250	Exact bounded and dense race equivalence.
Finite-map execution	400	Scan, walk and composition agreement.
Gated-delta replay	150 trajectories	868 accepted-prefix replay checks.
Lease reuse	300 schedules	Sequential epoch and base-prefix guards.

Table 14: CPU evidence inherited from the earlier revision, on `main`. Counterexamples for biased depth selection, graph-cost cliffs, batch unions, invalid numerical bounds, support truncation and stale namespaces are also included. No GPU implementation is exercised.

The theory workstream adds an exact reference that emulates BF16, FP32 and FP64 rounding on rationals and checks every bound and decision of Sections 6 and 7 against real arithmetic: 20 test methods and 39,761 generated checks, including 5,600 accumulation-bound checks over eight summation models, 420 certified argmax cases (72 exact ties), 360 races, 300 acceptance decisions with adversarial uniforms, witnesses that each envelope term is necessary, and the R-bf16 and split- K witnesses of Section 3.4 (`tests/test_precision.py`, `evidence/precision/`). These are tests on synthetic heads, not proofs and not GPU measurements.

Formal status. The earlier revision’s Lean file (`formal/DecisionGuards.lean`, nine declarations for scaled-integer guards, monotone refinement, composition, replay and publication guards) could not be compiled when it was written. Lean 4.19.0 now elaborates it, together with `formal/CertifiedArgmax.lean`, which proves the certificate and its uniqueness, the candidate filter, the guards with enclosed numerators and their exclusivity, the residual interval, the Hölder bound and the sequential γ_n bound in scaled integers, using only Lean’s built-in axioms (`formal/STATUS.md`, `evidence/precision/lean_check.log`). IEEE rounding itself, the bridge from floating-point values to these integers, exponentials and every probability statement remain outside the checker.

E Reproduction

The CPU evidence on main is reproduced from the repository root with Python 3.13 and NumPy:

```
python -m venv .venv && . .venv/bin/activate
python -m pip install -r requirements-cpu.txt
python scripts/verify_artifact.py      # CPU tests, synthetic drift, Lean attempt
```

The paper builds with `latexmk -pdf paper.tex` in `paper/` (pdfLaTeX and BibTeX; no shell escape). Figures that read committed data files regenerate on each build. GPU experiments go through the repository’s GPU lock and are documented with their exact commands in `RUNBOOK.md` and in the `README.md` of each `evidence/` directory.

F Claim ledger

Claim	Status	Evidence
Head is 15.1% of plain-step weight bytes, 84.1% of an MTP draft step’s	Derived	Checkpoint safetensors headers (Table 1)
GDN state 50.3 MB per request (FP32)	Derived	<code>config.json</code> (Table 1)
Stock head, sampling and verification behaviour at the pin	Code reading	Section 3.2; SGLang <code>bd66ce343e4f</code>
Gap condition (Theorem 3.5) and R-bf16 witness	Proved	Section 3.4
Certified tokens equal stock tokens at the served shapes, with the kernel’s measured error	Pending	kernel; <code>evidence/certified_head/</code>
Commit rule, directional margin, dead/deferred/enclosed accounting	Proved / definition	Section 3
Transport theorem, summary merge, admission rule	Proved	Section 5
Transport–self-evidence crossover and static-screen comparison	Proved	Theorem 5.5, Proposition 5.6
δ^* median 0.85% (int8, ℓ_2 , 64-row tiles), 0.30% (ℓ_∞/ℓ_1)	Derived	<code>evidence/precision/head_constants.json</code>
Measured drift ρ and transport certification	Pending	geometry; <code>evidence/head_geometry/</code>

Claim	Status	Evidence
Envelope with Hopper accumulation model; integer pass; outward rounding	Proved (model assumed)	Section 6; Khattak and Mikaitis [37]
Refinement ladder; candidate-count bounds	Proved	Theorem 6.6, Proposition 6.7
Plain-decode alignment of captured head inputs; candidate sets per cheap head; R-stock fallback share by rule and model; certified tokens equal the engine's on 6,005 rows	Finding	evidence/head_geometry/alignment_plain4b.json, evidence/head_geometry/selfevidence_plain4b.json, evidence/head_geometry/rstock_plain4b.json
Candidate sets on MTP and DFlash states; head-path runtime	Pending	geometry, kernel
Certified race; RNG contract; fixed-noise verification; coupling bounds	Proved	Section 7
Sampling contract: fixed-noise verification with the stock seeded noise; committed tokens equal the seeded sampler's tokens for the verify pass's logits	Design decided; pending	kernel; Section 7.3
Common mass, bounded-range sampler, KL and sequence TV bounds	Proved	Propositions 7.3–7.5
Acceptance guards with enclosures; residual races	Proved	Theorems 7.8, 7.9, 7.10
Cheap decision is not cheap verification (constructed heads)	Finding (CPU)	data/synthetic_drift.csv, evidence/synthetic_drift.json
Exact references (inherited)	Finding (CPU)	evidence/decision_tests.json, evidence/race_tests.json
Exact floating-point reference; Lean elaboration	Finding (CPU, Lean)	evidence/precision/, formal/STATUS.md
Attribution, bytes per step, host gaps (H1, H6)	Pending	profile; evidence/profiles/ (PR #13)
Stock divergence mechanisms and state safety (H5)	Pending	state; evidence/state_safety/
DFlash acceptance by block position and domain on GH200 (correctness runs, no timings)	Finding	evidence/drafter/
DFlash cycle timing and comparison with tuned MTP	Pending	drafter, bench
Certified tail (P1)	Rejected by its kill test	evidence/head_geometry/tail_killtest.json
Long-window repair (P2), anchored residual decoding (P3)	Hypotheses	Kill tests pending

Claim	Status	Evidence
What an exact recurrent state must retain (Proposition 3.8); key-subspace reduction	Proved	Section 3.9
State-structure witnesses ($-3/25$, reachability, SiLU rank, batch rank, RMSNorm, lazy decay, BF16 rank)	Finding (CPU)	evidence/state_structure/
Contract witnesses (reassociation, BF16 cell, law versus argmax, law versus seeded token, regrouped GDN recurrence, transition norm)	Finding (CPU)	evidence/contracts/
Write avoidance for the recurrent state (P4); cross-request sharing (P5)	Pending; P5 rejected in analysis	moonshot, profile
Derived throughput ceilings per lever stack (layer-serial execution, assumed tensor-core rate)	Derived	evidence/moonshot/ceilings.json
First lever runs: FP16 state $1.24\times$ at $c = 128$ (quality pending); FP8 weights no consistent gain ($0.96\text{--}1.06\times$), FP8 KV no gain (single runs)	Finding (single runs)	evidence/moonshot/lever_sweeps_quick.csv
Repeated lever measurements, quality, interaction matrix, exact and lossy frontiers (H4, H7)	Pending	moonshot, bench; evidence/bench/

G Attribution and disclosure

Samuel Mausberg is the author of this paper and reviews its claims and implementation. The earlier revisions of the manuscript, its derivations and its reference code were prepared with GPT-6 Astra Pro (OpenAI) from research materials, public primary sources and local CPU execution. This revision was produced with Claude Code, using Claude Opus 5.5 agents for the literature review, theory, measurements and writing, on the earlier draft; the literature, theory and evidence changes this paper cites were approved by an independent reviewing agent before they were merged, and all of the work is subject to the author’s review. No AI system is an author. The confidential assignment that motivated the work is not reproduced or quoted, and no model weights are redistributed.

References

- [1] Ahmad Abdelfattah, Jack Dongarra, Massimiliano Fasi, Mantas Mikaitis, and Françoise Tisseur. Analysis of Floating-Point Matrix Multiplication Computed via Integer Arithmetic, 2025. URL <https://arxiv.org/abs/2506.11277>. arXiv:2506.11277v3, 8 May 2026 (v1 12 Jun 2025).
- [2] Firas Abuzaid, Geet Sethi, Peter Bailis, and Matei Zaharia. To Index or Not to Index: Optimizing Exact Maximum Inner Product Search. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1250–1261, 2019. doi: 10.1109/ICDE.2019.00114. URL <https://arxiv.org/abs/1706.01449>. arXiv:1706.01449v3, 15 Mar 2019 (v1 5 Jun 2017).
- [3] Zihao An, Taichi Liu, Ziqiong Liu, Dong Li, Ruofeng Liu, and Emad Barsoum. PARD-2: Target-aligned parallel

- draft model for dual-mode speculative decoding, 2026. URL <https://arxiv.org/abs/2605.08632v1>. arXiv:2605.08632v1, 9 May 2026.
- [4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
 - [5] Ofir Ben Shoham. Balancing Coverage and Draft Latency in Vocabulary Trimming for Faster Speculative Decoding, 2026. URL <https://arxiv.org/abs/2603.05210>. arXiv:2603.05210v1, 5 Mar 2026.
 - [6] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 5209–5235. PMLR, 2024. URL <https://arxiv.org/abs/2401.10774>. arXiv:2401.10774v3, 14 Jun 2024 (v1 19 Jan 2024).
 - [7] Yuxuan Cai, Xiaozhuan Liang, Xinghua Wang, Jin Ma, Haijin Liang, Jinwen Luo, Xinyu Zuo, Lisheng Duan, Yuyang Yin, and Xi Chen. FastMTP: Accelerating LLM Inference with Enhanced Multi-Token Prediction, 2025. URL <https://arxiv.org/abs/2509.18362>. arXiv:2509.18362v1, 16 Sep 2025.
 - [8] Luis Chamberlain. Certified LM-head decode: read a quarter of the output head, losslessly. `mcgrof/knlp, docs/lm-head-decode.md`, first committed 23 June 2026 (commit 1dd92463092d), 2026. URL <https://github.com/mcgrof/knlp/blob/main/docs/lm-head-decode.md>. MIT licence. Read 30 September 2026.
 - [9] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating Large Language Model Decoding with Speculative Sampling, 2023. URL <https://arxiv.org/abs/2302.01318>. arXiv:2302.01318v1, 2 Feb 2023.
 - [10] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2602.06036v2>. arXiv:2602.06036v2, 28 May 2026 (camera-ready). Code: <https://github.com/z-lab/dflash>.
 - [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>. 58 authors.
 - [12] Zhiyang Chen, Daliang Xu, Yinyuan Zhang, Chenghua Wang, Mengwei Xu, and Yun Ma. NanoSpec: Accelerating speculative decoding using minimalist in-context vocabularies, 2026. URL <https://arxiv.org/abs/2605.26444v2>. arXiv:2605.26444v2, 1 June 2026. Version 1 (8 April 2026) was titled “MicroSpec: Accelerating Speculative Decoding with Lightweight In-Context Vocabularies”.
 - [13] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ea1f5f0878d43ff4fb8bf64ef4a2326c-Abstract-Conference.html. arXiv:2402.12374v3, 5 Jul 2025 (v1 19 Feb 2024). Proceedings title: “Sequoia: Scalable and Robust Speculative Decoding”.
 - [14] Xin Cheng, Xingkai Yu, Chenze Shao, Jiashi Li, Yunfan Xiong, Yi Qian, Jiaqi Zhu, Shirong Ma, Xiaokang Zhang, Jiasheng Ye, Qinyu Chen, Chengqi Deng, Jiping Yu, Damai Dai, Zhengyan Zhang, Yixuan Wei, Yixuan Tan, Wenkai Yang, Runxin Xu, Yu Wu, Zhean Xu, Xuanyu Wang, Muiyang Chen, Rui Tian, Xiao Bi, Zhewen Hao, Shaoyuan Chen, Huanqi Cao, Wentao Zhang, Anyi Xu, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. DSpark: Confidence-scheduled speculative decoding with semi-autoregressive generation, 2026. URL <https://arxiv.org/abs/2607.05147v1>. arXiv:2607.05147v1, 6 July 2026.
 - [15] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.

- [16] Majid Daliri, Christopher Musco, and Ananda Theertha Suresh. Coupling without Communication and Drafter-Invariant Speculative Decoding. In *2025 IEEE International Symposium on Information Theory (ISIT)*, pages 1–6, 2025. doi: 10.1109/ISIT63088.2025.11195663. URL <https://arxiv.org/abs/2408.07978>. arXiv:2408.07978v4, 20 Aug 2025 (v1 15 Aug 2024).
- [17] DeepSeek-AI. DeepSeek-V3 Technical Report, 2024. URL <https://arxiv.org/abs/2412.19437>. arXiv:2412.19437v2, 18 Feb 2025 (v1 27 Dec 2024). The arXiv record lists DeepSeek-AI and about 200 named authors.
- [18] Gaoyuan Du, Anam Nawaz Khan, Rex Zhou, Xiaoyang Liu, Deepayan Chakrabarti, Fnu Suyu, and Xueping Li. Greedy decoding is not precision-invariant: Cross-precision output divergence in LLM inference. *Transactions on Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2609.26621v1>. arXiv:2609.26621v1, 22 September 2026; accepted by TMLR per the arXiv comment.
- [19] Matthew Dutson, Yin Li, and Mohit Gupta. Eventful transformers: Leveraging temporal redundancy in vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 16911–16923, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/html/Dutson_Eventful_Transformers_Leveraging_Temporal_Redundancy_in_Vision_Transformers_ICCV_2023_paper.html. Verified on the CVF open access page.
- [20] Massimiliano Fasi, Nicholas J. Higham, Mantas Mikaitis, and Srikara Pranesh. Numerical behavior of NVIDIA tensor cores. *PeerJ Computer Science*, 7:e330, 2021. doi: 10.7717/peerj-cs.330.
- [21] Ophir Frieder, Ida Mele, Cristina Ioana Muntean, Franco Maria Nardini, Raffaele Perego, and Nicola Tonellotto. Caching Historical Embeddings in Conversational Search. *ACM Transactions on the Web*, 18(4), 2024. doi: 10.1145/3578519. URL <https://arxiv.org/abs/2211.14155>. arXiv:2211.14155v1, 25 Nov 2022.
- [22] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 14060–14079. PMLR, 2024. URL <https://arxiv.org/abs/2402.02057>. arXiv:2402.02057v1, 3 Feb 2024.
- [23] Sneha Murthy Ghantasala. TreeWY: Speculative Verification for Gated DeltaNet Hybrids, 2026. URL <https://arxiv.org/abs/2608.20961>. arXiv:2608.20961v1, 21 Aug 2026.
- [24] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & Faster Large Language Models via Multi-token Prediction. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 15706–15734. PMLR, 2024. URL <https://arxiv.org/abs/2404.19737>. arXiv:2404.19737v1, 30 Apr 2024.
- [25] Raghavv Goel, Sudhanshu Agrawal, Mukul Gagrani, Junyoung Park, Yifan Zao, He Zhang, Tian Liu, Yiping Yang, Xin Yuan, Jiuyan Lu, Chris Lott, and Mingu Lee. VOCABTRIM: Vocabulary Pruning for Efficient Speculative Decoding in LLMs, 2025. URL <https://arxiv.org/abs/2506.22694>. arXiv:2506.22694v2, 3 Jul 2025 (v1 28 Jun 2025). ICML 2025 Workshop on Efficient Systems for Foundation Models, according to the arXiv comment.
- [26] Geonmo Gu, Byeongho Heo, HeeJae Jun, Yoohoon Kang, Sangmin Lee, Sangdoo Yun, and Dongyoon Han. Verification-aware training for speculative decoding, 2026. URL <https://arxiv.org/abs/2608.30135v1>. arXiv:2608.30135v1, 31 Aug 2026.
- [27] E. J. Gumbel. *Statistical Theory of Extreme Values and Some Practical Applications*. Number 33 in National Bureau of Standards Applied Mathematics Series. National Bureau of Standards, 1954. Bibliographic record checked; the text was not read.
- [28] Stephen Hawkins. Head bit-plane exact argmax: read 25–33 % fewer head bytes on every draft/greedy head read, all families. GitHub issue 69, HawkBearPig/dgpp, opened 29 September 2026; implementation in `src/kernels/packq_head.cu`, 2026. URL <https://github.com/HawkBearPig/dgpp/issues/69>. Apache-2.0. Issue, comments and code read 30 September 2026 at commit a23d8b1. The issue is filed by GitHub user HawkBearPig; the commit author is Stephen Hawkins.

- [29] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, September 2025. URL <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>. Published 10 September 2025. Code: https://github.com/thinking-machines-lab/batch_invariant_ops.
- [30] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, second edition, 2002. doi: 10.1137/1.9780898718027.
- [31] Lanxiang Hu, Siqi Kou, Yichao Fu, Samyam Rajbhandari, Tajana Rosing, Yuxiong He, Zhijie Deng, and Hao Zhang. Fast and accurate causal parallel decoding using Jacobi forcing. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2512.14681>. Verified on the arXiv abstract page (v1); ICML 2026 per OpenReview record ORy2hyPd4G.
- [32] Kaixuan Huang, Xudong Guo, and Mengdi Wang. SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths. In *Second Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2405.19715>. arXiv:2405.19715v3, 11 Jul 2025 (v1 30 May 2024). Venue from the arXiv comment.
- [33] Inco AI. DFlash 2: Keep drafting parallel. Inco AI blog, August 2026. URL <https://inco.ai/blog/dflash2/>. Published 18 August 2026; read 30 September 2026.
- [34] Inco AI. Qwen3.8-27B-DFlash2 model card. Hugging Face, revision `015e795645c74b1a0eeef3b570031fb62e769bc5`, 2026. URL <https://huggingface.co/incoai/Qwen3.8-27B-DFlash2/tree/015e795645c74b1a0eeef3b570031fb62e769bc5>. Created 18 August 2026; read 30 September 2026.
- [35] Weifan Jiang, Krishna Teja Chitty-Venkata, Megan Flynn, Reed Meyerson, Zhenting Qi, Tianyu Wu, Eldar Kurtic, Minlan Yu, and Alexandre Marques. H-Spec: Parallel speculative decoding without a drafter-side KV cache, 2026. URL <https://arxiv.org/abs/2609.24197>. Verified on the arXiv abstract page (v1).
- [36] Amirmohammad Karimi, Chao Gao, and Negar Hassanpour. DBLAST: Dependent block drafting for stochastic speculative decoding, 2026. URL <https://arxiv.org/abs/2608.05448v1>. arXiv:2608.05448v1, 5 Aug 2026.
- [37] Faizan A. Khattak and Mantas Mikaitis. Accurate Models of NVIDIA Tensor Cores. *ACM Transactions on Architecture and Code Optimization*, 23(3):1–25, 2026. doi: 10.1145/3830409. URL <https://arxiv.org/abs/2512.07004>. arXiv:2512.07004v4, 11 Jun 2026 (v1 7 Dec 2025).
- [38] Jaeyeon Kim, Jewon Lee, and Bo-Kyeong Kim. Quantize the Target, Quantize the Drafter: Efficient Inference with Qwen3.5-4B, 2026. URL <https://arxiv.org/abs/2607.04244>. arXiv:2607.04244v2, 7 Jul 2026 (v1 5 Jul 2026). Efficient Qwen Competition, ICML 2026 AdaptFM workshop, according to the arXiv comment. Checkpoints nota-ai/Qwen3.5-4B-QAD-W4A16 and nota-ai/Qwen3.5-4B-DFlash-GPTQ-W4A16 (Apache-2.0).
- [39] Suyoung Kim, Jahyun Koo, Hyeonjin Kim, Inhyeok Bang, Seunghyun Lee, Hyunjae Oh, Baeseong Park, and Dongsoo Lee. BV loss: Block verification-aware loss for block diffusion speculative decoding, 2026. URL <https://arxiv.org/abs/2609.34832v1>. arXiv:2609.34832v1, 28 Sep 2026.
- [40] Şuayp Talha Kocabay, Talha Rüzgar Akkuş, and Kamer Ali Yuksel. ARCHead: Activation-Metric Residual Correction for Large Language Model Output Heads, 2026. URL <https://arxiv.org/abs/2608.02703>. arXiv:2608.02703v1, 3 Aug 2026. Submitted to ACL Rolling Review.
- [41] Jahyun Koo, Sunghyeon Woo, Jaeun Kil, Jeongtae Lee, Sungjae Lee, Kyomin Jung, and Minsub Kim. Carryover drafting: Recycling rejected states for speculative decoding, 2026. URL <https://arxiv.org/abs/2609.14717>. Verified on the arXiv abstract page (v1).
- [42] Wouter Kool, Herke van Hoof, and Max Welling. Stochastic Beams and Where to Find Them: The Gumbel-Top-k Trick for Sampling Sequences Without Replacement. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3499–3508. PMLR, 2019. URL <https://arxiv.org/abs/1903.06059>. arXiv:1903.06059v2, 29 May 2019 (v1 14 Mar 2019).

- [43] Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi-Rui Tam, Keith Stevens, Abdullah Barhoum, Nguyen Minh Duc, Oliver Stanley, Richárd Nagyfi, Shahul ES, Sameer Suri, David Glushkov, Arnav Dantuluri, Andrew Maguire, Christoph Schuhmann, Huu Nguyen, and Alexander Mattick. OpenAssistant conversations – democratizing large language model alignment. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. URL <https://arxiv.org/abs/2304.07327>.
- [44] Siqi Kou, Lanxiang Hu, Zhezhi He, Zhijie Deng, and Hao Zhang. CLLMs: Consistency Large Language Models, 2024. URL <https://arxiv.org/abs/2403.00835>. arXiv:2403.00835v4, 13 Jun 2024 (v1 28 Feb 2024). ICML 2024 according to the arXiv comment.
- [45] Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2603.03251>. Verified on the arXiv abstract page (v3); OpenReview record aL1Wnml9Ef, ICLR 2026 poster.
- [46] Yashas Samaga B L, Varun Yerram, Chong You, Srinadh Bhojanapalli, Sanjiv Kumar, Prateek Jain, and Praneeth Netrapalli. HiRE: High Recall Approximate Top- k Estimation for Efficient LLM Inference, 2024. URL <https://arxiv.org/abs/2402.09360>. arXiv:2402.09360v1, 14 Feb 2024.
- [47] Layr Labs. Laguna LM-head prune (Sources/MLXFastModel/LagunaLmHeadPrune.swift). Source code, Layr-Labs/mlxfast-challenge, commit 4f4f68959cfe (28 July 2026), 2026. URL <https://github.com/Layr-Labs/mlxfast-challenge>. MIT licence. Read 30 September 2026.
- [48] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>. arXiv:2211.17192.
- [49] Hui Li, Tsz Nam Chan, Man Lung Yiu, and Nikos Mamoulis. FEXIPRO: Fast and exact inner product retrieval in recommender systems. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, pages 835–850, 2017. doi: 10.1145/3035918.3064009.
- [50] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 28935–28948. PMLR, 2024. URL <https://arxiv.org/abs/2401.15077>. arXiv:2401.15077v3, 4 Mar 2025 (v1 26 Jan 2024).
- [51] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7421–7432, 2024. doi: 10.18653/v1/2024.emnlp-main.422. URL <https://arxiv.org/abs/2406.16858>. arXiv:2406.16858v2, 30 Jun 2024 (v1 24 Jun 2024).
- [52] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://arxiv.org/abs/2503.01840>. arXiv:2503.01840v3, 23 Apr 2025 (v1 3 Mar 2025). <https://neurips.cc/virtual/2025/poster/119930>.
- [53] Luxi Lin, Zhanpeng Zeng, Shuang Peng, Songwei Liu, and Rongrong Ji. ReTrace: Rejected-trajectory conditioning for speculative decoding, 2026. URL <https://arxiv.org/abs/2608.29748>. Verified on the arXiv abstract page (v2).
- [54] Ze-Wei Liou and Tri Dao. ReplaySSM: Cache SSM inputs, not state. Dao AI Lab blog, June 2026. URL <https://dao-lab.ai/blog/2026/replayssm/>. Published 15 June 2026. Code: <https://github.com/Johnny-Liou/ReplaySSM>.
- [55] Aofan Liu, Jingxiang Meng, Fangxin Liu, and Yongbiao Chen. CURE: Local uncertainty repair for block-parallel speculative decoding, 2026. URL <https://arxiv.org/abs/2608.00531>. Verified on the arXiv abstract page (v1).

- [56] Dong Liu, Shu Wang, Yanxuan Yu, Haisheng Wang, and Ben Lengerich. CSV-Decode: Certifiable Sub-Vocabulary Decoding for Efficient Large Language Model Inference, 2025. URL <https://arxiv.org/abs/2511.21702>. arXiv:2511.21702v2, 26 Jul 2026 (v1 16 Nov 2025). Code: <https://github.com/FastLM/CSV-Decode> (no licence file).
- [57] Tianyu Liu, Yuhao Shen, Rui Cen, Junhan Shi, Jiebin Zhang, Guangshuo Qin, Hong Liu, Song Liu, Guanghua Yu, and Jianchen Zhu. D-cut: Adaptive verification depth pruning for batched speculative decoding, 2026. URL <https://arxiv.org/abs/2607.14647v1>. arXiv:2607.14647v1, 16 July 2026.
- [58] Xianzhen Luo, Yixuan Wang, Qingfu Zhu, Zhiming Zhang, Xuanyu Zhang, Qing Yang, and Dongliang Xu. Turning trash into treasure: Accelerating inference of large language models with token recycling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6816–6831, 2025. doi: 10.18653/v1/2025.acl-long.338. URL <https://aclanthology.org/2025.acl-long.338/>. Verified on the ACL Anthology page and the arXiv abstract page (v3).
- [59] Zhiyuan Ma. GDN Tree-Scan: Served Tree Verification for Recurrent-Hybrid Language Models, 2026. URL <https://arxiv.org/abs/2609.23900>. arXiv:2609.23900v1, 20 Sep 2026.
- [60] Chris J. Maddison, Daniel Tarlow, and Tom Minka. A* sampling. In *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, pages 3086–3094, 2014. URL https://proceedings.neurips.cc/paper_files/paper/2014/hash/937debc749f041eb5700df7211ac795c-Abstract.html. arXiv:1411.0030.
- [61] Stephen Mussmann and Stefano Ermon. Learning and inference via maximum inner product search. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 2587–2596. PMLR, 2016. URL <https://proceedings.mlr.press/v48/musmann16.html>.
- [62] Stephen Mussmann, Daniel Levy, and Stefano Ermon. Fast amortized inference and learning in log-linear models with randomly perturbed nearest neighbor search. In *Proceedings of the Thirty-Third Conference on Uncertainty in Artificial Intelligence (UAI 2017)*, 2017. URL <https://arxiv.org/abs/1707.03372>. arXiv:1707.03372.
- [63] NVIDIA. NVIDIA Hopper tuning guide. CUDA 13.0.2 documentation archive, 2025. URL <https://docs.nvidia.com/cuda/archive/13.0.2/hopper-tuning-guide/index.html>. Read 30 September 2026.
- [64] NVIDIA. AIPerf metrics reference. AIPerf documentation, 2026. URL <https://docs.nvidia.com/aiperf/reference/ai-perf-metrics-reference>. Read 30 September 2026.
- [65] NVIDIA. NVIDIA Blackwell tuning guide. CUDA 13.4 documentation, 2026. URL <https://docs.nvidia.com/cuda/blackwell-tuning-guide/index.html>. Read 30 September 2026.
- [66] NVIDIA. Floating point and IEEE 754. CUDA 13.4 documentation, 2026. URL <https://docs.nvidia.com/cuda/floating-point/index.html>. Read 30 September 2026.
- [67] NVIDIA. Nsight Compute profiling guide. Nsight Compute documentation, 2026. URL <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html>. Read 30 September 2026.
- [68] NVIDIA. Parallel thread execution ISA, version 9.4. CUDA documentation, 2026. URL <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html>. Sec. 9.7.17 (warpgroup MMA, requires sm_90a) and Sec. 9.7.18 (tcgen05); read 30 September 2026.
- [69] NVIDIA CUTLASS. CuTe DSL overview. CUTLASS documentation, 2026. URL <https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/overview.html>. Read 30 September 2026.
- [70] Yi Pan, Haocheng Xi, Kan Zhu, Xingyang Li, Yibo Wu, Mayank Mishra, Hongtao Zhang, William X. Zheng, Baris Kasikci, Song Han, Kurt Keutzer, Rishabh Iyer, and Ion Stoica. LeapQuant: Efficient Linear Attention with Accurate Recurrent State Quantization, 2026. URL <https://arxiv.org/abs/2609.38166>. arXiv:2609.38166v1, 29 Sep 2026.

- [71] George Papandreou and Alan L. Yuille. Perturb-and-MAP random fields: Using discrete optimization to learn and sample from energy models. In *2011 International Conference on Computer Vision (ICCV)*, pages 193–200, 2011. doi: 10.1109/ICCV.2011.6126242.
- [72] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. DeltaCNN: End-to-end CNN inference of sparse frame differences in videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12497–12506, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Parger_DeltaCNN_End-to-End_CNN_Inference_of_Sparse_Frame_Differences_in_Videos_CVPR_2022_paper.html. Verified on the CVF open access page.
- [73] Pragaash Ponnusamy, Shivam Sahni, Jue Wang, and Tri Dao. SonicSampler: Unified tile-aware kernels for LLM sampling and speculative verification, 2026. URL <https://arxiv.org/abs/2607.20475v1>. arXiv:2607.20475v1, submitted 24 May 2026.
- [74] PyTorch contributors. Numerical accuracy. PyTorch 2.14 documentation, 2026. URL https://docs.pytorch.org/docs/2.14/notes/numerical_accuracy.html. Read 30 September 2026.
- [75] Qwen. Qwen3.5-4B model card. Hugging Face, revision 851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a, 2026. URL <https://huggingface.co/Qwen/Qwen3.5-4B/tree/851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a>. Read 30 September 2026.
- [76] Qwen. Qwen3.5-4B model configuration (config.json). Hugging Face, revision 851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a, 2026. URL <https://huggingface.co/Qwen/Qwen3.5-4B/blob/851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a/config.json>. Read 30 September 2026.
- [77] qwen38-challenge_senpai contributors. E142: The verify readout streams 715 MB per round to answer a two-element question. GitHub pull request 142, morganmcg1/qwen38-challenge_senpai, opened 22 August 2026, 2026. URL https://github.com/morganmcg1/qwen38-challenge_senpai/pull/142. Final report comment of 23 August 2026 records the screens as refuted. Read 30 September 2026.
- [78] Parikshit Ram and Alexander G. Gray. Maximum Inner-Product Search Using Cone Trees. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 931–939, 2012. doi: 10.1145/2339530.2339677. URL <https://arxiv.org/abs/1202.6101>. arXiv:1202.6101v1, 28 Feb 2012. arXiv title: “Maximum Inner-Product Search using Tree Data-structures”.
- [79] Joseph Rowan, Buu Phan, and Ashish Khisti. List-Level Distribution Coupling with Applications to Speculative Decoding and Lossy Compression. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://arxiv.org/abs/2506.05632>. arXiv:2506.05632v3, 11 Jan 2026 (v1 5 Jun 2025).
- [80] Tomas Ruiz, Zhen Qin, Yifan Zhang, Xuyang Shen, Yiran Zhong, and Mengdi Wang. FlashSampling: Fast and memory-efficient exact sampling, 2026. URL <https://arxiv.org/abs/2603.15854v3>. arXiv:2603.15854v3, 25 September 2026 (v1 16 March 2026, v2 12 May 2026). Code: <https://github.com/FlashSampling/FlashSampling>.
- [81] Siegfried M. Rump. Verification methods: Rigorous results using floating-point arithmetic. *Acta Numerica*, 19: 287–449, 2010. doi: 10.1017/S096249291000005X.
- [82] Matan Rusanovsky, Yoav Miron, Roy Uziel, Omer Belhasin, Hao Guo, Ran Zilberstein, Maor Ashkenazi, and Michael Elad. LiLiCorr: Lightweight likelihood correlation of parallel drafts for speculative decoding, 2026. URL <https://arxiv.org/abs/2608.20530v2>. arXiv:2608.20530v2, 22 September 2026.
- [83] Andrea Santilli, Silvio Severino, Emilian Postolache, Valentino Maiorca, Michele Mancusi, Riccardo Marin, and Emanuele Rodolà. Accelerating transformer inference for translation via parallel decoding. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL <https://arxiv.org/abs/2305.10427>.

- [84] SGLang contributors. GDN ReplaySSM speculative fold (python/sglang/kernels/ops/attention/fla/gdn_replayssm_spec_fold.py). Source code, commit bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824, 2026. URL https://github.com/sgl-project/sglang/blob/bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824/python/sglang/kernels/ops/attention/fla/gdn_replayssm_spec_fold.py. Lines 1–95 read 30 September 2026.
- [85] SGLang contributors. [feature] dynamic verification for DFlash2. SGLang pull request 36136, opened 24 August 2026 by GitHub user EthanWang555, 2026. URL <https://github.com/sgl-project/sglang/pull/36136>. Open and unmerged on 30 September 2026; head c0235e700214f1de5044be25f795faf5b755e450.
- [86] SGLang contributors. DFlash model and candidate-head implementation (python/sglang/srt/models/dflash.py). Source code, commit bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824, 2026. URL <https://github.com/sgl-project/sglang/blob/bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824/python/sglang/srt/models/dflash.py>. Read 30 September 2026.
- [87] Kyuhong Shim, Minjae Lee, Iksoo Choi, Yoonho Boo, and Wonyong Sung. SVD-Softmax: Fast softmax approximation on large vocabulary neural networks. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/4e2a6330465c8ffcaa696a5a16639176-Abstract.html.
- [88] Junhyuk So, Hyunho Kook, Chaeyeon Jang, and Eunhyeok Park. Speculative coupled decoding for training-free lossless acceleration of autoregressive visual generation. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2510.24211>. Verified on the arXiv abstract page; v1 was titled MC-SJD: Maximal Coupling Speculative Jacobi Decoding; ICML 2026 per the arXiv comment and OpenReview.
- [89] Yang Song, Chenlin Meng, Renjie Liao, and Stefano Ermon. Accelerating feedforward computation via parallel nonlinear equation solving. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021. URL <https://arxiv.org/abs/2002.03629>.
- [90] sparkpipe contributors. Certified screened LM head (SparkLmHeadCertifiedFp8QuantizeKernel, SparkLmHeadCertifiedFp8ScoreKernel). Source code, model-families/common/include/sparkpipe/spark_lm_kernels.cuh, commit 84efd5b63d8d (16 August 2026); pull request 744 (merged 29 August 2026), 2026. URL <https://github.com/sparkpipe/sparkpipe/pull/744>. No licence file. Read 30 September 2026.
- [91] Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. SpecTr: Fast Speculative Decoding via Optimal Transport. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/6034a661584af6c28fd97a6f23e56c0a-Abstract-Conference.html. arXiv:2310.15141v2, 18 Jan 2024 (v1 23 Oct 2023).
- [92] Ziteng Sun, Uri Mendlovic, Yaniv Leviathan, Asaf Aharoni, Jae Hun Ro, Ahmad Beirami, and Ananda Theertha Suresh. Block Verification Accelerates Speculative Decoding. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2403.10444>. arXiv:2403.10444v3, 10 Apr 2025 (v1 15 Mar 2024).
- [93] Chaofan Tao, Qian Liu, Longxu Dou, Niklas Muennighoff, Zhongwei Wan, Ping Luo, Min Lin, and Ngai Wong. Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/cf5a019ae9c11b4be88213ce3f85d85c-Abstract-Conference.html. arXiv:2407.13623v3, 1 Nov 2024 (v1 18 Jul 2024).
- [94] Christina Teflioudi, Rainer Gemulla, and Olga Mykytiuk. LEMP: Fast retrieval of large entries in a matrix product. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 107–122, 2015. doi: 10.1145/2723372.2747647.
- [95] Yao Teng, Han Shi, Xian Liu, Xuefei Ning, Guohao Dai, Yu Wang, Zhenguo Li, and Xihui Liu. Accelerating auto-regressive text-to-image generation with training-free speculative Jacobi decoding. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2410.01699>. Verified on the arXiv abstract page; ICLR 2025 poster per OpenReview record LZfjxvqw0N.

- [96] The SGLang Team. Towards deterministic inference in SGLang and reproducible RL training. LMSYS Org blog, September 2025. URL <https://www.lmsys.org/blog/2025-09-22-sglang-deterministic/>. Published 22 September 2025, updated 24 September 2025.
- [97] TileLang contributors. TileLang documentation. Version 0.1.15, 2026. URL <https://tilelang.com/>. Read 30 September 2026.
- [98] Triton contributors. triton.autotune. Triton documentation, 2026. URL <https://triton-lang.org/main/python-api/generated/triton.autotune.html>. Read 30 September 2026.
- [99] Oszkár Urbán, Young D. Kwon, Stylianos I. Venieris, and Cecilia Mascolo. Margins, Not Windows: Training-Free Per-Step Lossy Speculative Decoding, 2026. URL <https://arxiv.org/abs/2609.02897>. arXiv:2609.02897v2, 28 Sep 2026 (v1 3 Jul 2026).
- [100] vLLM contributors. CUDA graphs. vLLM design documentation, 2026. URL https://docs.vllm.ai/en/latest/design/cuda_graphs/. Read 30 September 2026.
- [101] Quynh Vo, Cong-Duy Nguyen, Ponghvoan Srey, Luu Anh Tuan, and Thong Nguyen. TIGER: Text-conditioned visual gated routing with acceptance alignment for multimodal speculative decoding, 2026. URL <https://arxiv.org/abs/2607.11131v1>. arXiv:2607.11131v1, 13 Jul 2026.
- [102] Jie Wang, Peter Wonka, and Jieping Ye. Lasso screening rules via dual polytope projection. *Journal of Machine Learning Research*, 16:1063–1101, 2015. URL <https://jmlr.org/papers/v16/wang15a.html>.
- [103] Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. OPT-Tree: Speculative decoding with adaptive draft tree structure. *Transactions of the Association for Computational Linguistics*, 13:188–199, 2025. doi: 10.1162/tac1_a_00735. URL <https://aclanthology.org/2025.tacl-1.8/>. arXiv:2406.17276.
- [104] Junxiong Wang, Daniele Paliotta, Avner May, Alexander M. Rush, and Tri Dao. The Mamba in the Llama: Distilling and Accelerating Hybrid Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/723933067ad315269b620bc0d2c05cba-Abstract-Conference.html. arXiv:2408.15237v4, 27 Jun 2025 (v1 27 Aug 2024).
- [105] Li Wang, Yi Su, Xiabao Wu, Chiran You, Yongchao Liu, Zhan Qiu, Juelu Zhang, Jiajun Zheng, Fangxin Liu, Jie Zhang, Chen Tian, and Chengying Huan. Bole: Efficient Tree Speculation for Hybrid-Attention Language Models, 2026. URL <https://arxiv.org/abs/2608.01651>. arXiv:2608.01651v1, 3 Aug 2026.
- [106] Zhibin Wang, Xuying Han, Zhaohua Yang, Fuliang Liu, Xue Li, Rong Gu, Sheng Zhong, and Chen Tian. SpecLA: Efficient Speculative Decoding for Linear-Attention Models, 2026. URL <https://arxiv.org/abs/2607.16673>. arXiv:2607.16673v1, 18 Jul 2026.
- [107] Roger Weber, Hans-J. Schek, and Stephen Blott. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB 1998)*, 1998. URL <https://www.vldb.org/conf/1998/p194.pdf>.
- [108] Miles Williams, Young D. Kwon, Rui Li, Alexandros Kouris, and Stylianos I. Venieris. Speculative decoding with a speculative vocabulary. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. URL <https://arxiv.org/abs/2602.13836v2>. arXiv:2602.13836v2, 17 July 2026.
- [109] Mengdi Wu, Xinhao Cheng, Shengyu Liu, Chunan Shi, Jianan Ji, Man Kit Ao, Praveen Velliengiri, Xupeng Miao, Oded Padon, and Zhihao Jia. Mirage: A multi-level superoptimizer for tensor programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, 2025. URL <https://www.usenix.org/conference/osdi25/presentation/wu-mengdi>. arXiv:2405.05751v3.
- [110] Tianyu Wu, Yu Yao, Zhenting Qi, Han Zheng, Zhuohan Wang, Haoran Ma, Lawrence Liao, Himabindu Lakkaraju, Ju Li, and Yilun Du. D-PACE: Dynamic position-aware cross-entropy for parallel speculative drafting, 2026. URL <https://arxiv.org/abs/2605.18810v1>. arXiv:2605.18810v1, 12 May 2026.

- [111] Yangchao Wu, Zongyue Qin, Alex Wong, and Stefano Soatto. STree: Speculative Tree Decoding for Hybrid State-Space Models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://arxiv.org/abs/2505.14969>. arXiv:2505.14969v2, 27 Oct 2025 (v1 20 May 2025). <https://neurips.cc/virtual/2025/poster/117297>.
- [112] Tianhua Xia, Mugilan Ganesan, Yifei Feng, Haiyu Wang, Maximilian Egger, and Sai Qian Zhang. Acceptance-aware draft model training for speculative decoding, 2026. URL <https://arxiv.org/abs/2609.24150v1>. arXiv:2609.24150v1, 21 Sep 2026.
- [113] Shanli Xing, Zihao Ye, Bohan Hou, Luis Ceze, and Tianqi Chen. Sorting-free GPU kernels for LLM sampling. FlashInfer blog, March 2025. URL <https://flashinfer.ai/2025/03/10/sampling.html>. Published 10 March 2025.
- [114] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/d13a3eae72366e61dfdc7eea82eeb685-Abstract-Conference.html. arXiv:2406.06484v6, 15 Jan 2025 (v1 10 Jun 2024).
- [115] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>. arXiv:2412.06464v3.
- [116] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, pages 94–109, 2025. doi: 10.1145/3689031.3696098. URL <https://doi.org/10.1145/3689031.3696098>. Verified via the Crossref record and the arXiv abstract page (v3).
- [117] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of the Eighth Conference on Machine Learning and Systems (MLSys 2025)*, 2025. URL <https://arxiv.org/abs/2501.01005>. arXiv:2501.01005v2.
- [118] Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://arxiv.org/abs/2506.09501>. arXiv:2506.09501v2, 24 Oct 2025 (v1 11 Jun 2025). Version 1 was titled “Give Me FP32 or Give Me Death? Challenges and Solutions for Reproducible Reasoning”.
- [119] z-lab. Qwen3.5-4B-DFlash model card. Hugging Face, revision 9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf, 2026. URL <https://huggingface.co/z-lab/Qwen3.5-4B-DFlash/tree/9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf>. Apache-2.0; created 5 March 2026, updated 19 June 2026; mirrored at `modal-labs/Qwen3.5-4B-DFlash`. DFlash (v1) drafter for Qwen/Qwen3.5-4B with SGLang launch instructions and B200 benchmarks against MTP. Read 30 September 2026.
- [120] Chunran Zhang. Exact Adaptive Hybrid Retrieval Without Fixed Top-L Cutoffs, 2026. URL <https://arxiv.org/abs/2608.07152>. arXiv:2608.07152v1, 7 Aug 2026.
- [121] Jinbin Zhang, Nasib Ullah, Erik Schultheis, and Rohit Babbar. DynaSpec: Context-aware dynamic speculative sampling for large-vocabulary language models, 2025. URL <https://arxiv.org/abs/2510.13847v3>. arXiv:2510.13847v3, 3 February 2026 (v1 11 October 2025).
- [122] Shuai Zhang, Huachuan Qiu, Hongliang He, and Yong Dai. Cost-aware diffusion draft trees for speculative decoding, 2026. URL <https://arxiv.org/abs/2606.01813v1>. arXiv:2606.01813v1, 1 Jun 2026.
- [123] Tao Zhang, Jianchao Tan, Pingwei Sun, Yanqi Yu, Zixu Jiang, Yuchen Xie, Xunliang Cai, and Ziqian Zeng. DAMP: Decay-Aware Mixed-Precision Recurrent-State Quantization, 2026. URL <https://arxiv.org/abs/2608.27513>. arXiv:2608.27513v1, 27 Aug 2026.

- [124] Yaojie Zhang, Linfeng Zhang, Bin Cui, and Xupeng Miao. DFlow: Enabling Verifier Information Flow in Block Diffusion Speculative Decoding, 2026. URL <https://arxiv.org/abs/2609.06498>. arXiv:2609.06498v1, 6 Sep 2026.
- [125] Weilin Zhao, Tengyu Pan, Xu Han, Yudi Zhang, Ao Sun, Yuxiang Huang, Kaihuo Zhang, Weilun Zhao, Yuxuan Li, Jie Zhou, Hao Zhou, Jianyong Wang, Zhiyuan Liu, and Maosong Sun. FR-Spec: Accelerating Large-Vocabulary Language Models via Frequency-Ranked Speculative Sampling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3909–3921, 2025. doi: 10.18653/v1/2025.acl-long.198. URL <https://arxiv.org/abs/2502.14856>. arXiv:2502.14856v2, 11 Mar 2025 (v1 20 Feb 2025).
- [126] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. URL <https://arxiv.org/abs/2306.05685>.
- [127] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 62557–62583, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html. arXiv:2312.07104.
- [128] Enyu Zhou, Kai Sheng, Hao Chen, and Xin He. CARD: A cache-assisted parallel speculative decoding framework via query-and-correct paradigm for accelerating LLM inference, 2025. URL <https://arxiv.org/abs/2508.04462>. Verified on the arXiv abstract page (v2; v1 had a different title).
- [129] Xiandong Zou, Jianshu Li, Jing Huang, and Pan Zhou. Variational speculative decoding: Rethinking draft training from token likelihood to sequence acceptance, 2026. URL <https://arxiv.org/abs/2602.05774v6>. arXiv:2602.05774v6, 12 Aug 2026 (v1 5 Feb 2026).